AWS ROUTE 53

— Overview —

Highly Available and Scalable DNS and Domain Name Registration Service

"A reliable and cost-effective way to route users to internet applications."

1) What is Amazon Route 53 ?

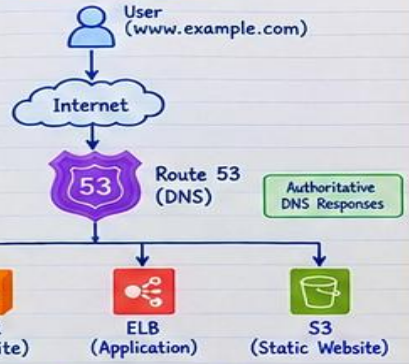
Amazon Route 53 is a scalable and highly available Domain Name System (DNS) web service that:

- Translates human-readable domain names (e.g., www.example.com) to IP addresses.
- Routes end users to internet applications. (e.g., websites, web apps, APIs).
- Provides health checking and traffic routing.
- Allows domain name registration.
- Integrates with other AWS services.

2) Why do we need Route 53 ?

- Users access applications using domain names.
- Helps route traffic to the best available endpoint.
- Provides high availability, low latency and fault tolerance.
- Supports advanced routing policies (latency, geolocation, failover, etc.).
- Easier domain management within AWS.
- Cost-effective and scales automatically.

3) How Route 53 Works ?

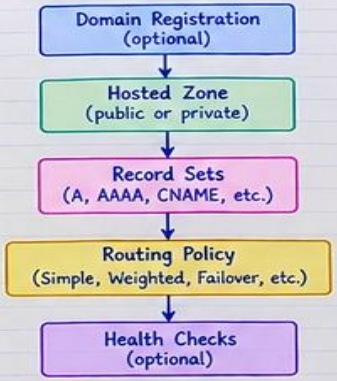


1. User types a domain name in browser.
2. Route 53 receives the DNS query.
3. Based on routing policy, it returns the appropriate IP address.
4. User is routed to the selected AWS resource (e.g., EC2, ELB, S3, etc.).

4) Important Terminology

Term	Meaning
Domain Name	Human-readable name (e.g., example.com).
Hosted Zone	Container for DNS records of a domain.
Record	DNS entry that maps a domain to a resource.
Record Set	A combination of a record name, type and value.
Routing Policy	Method to determine how traffic is routed.
Resolver	Resolves domain names to IP addresses.
Health Check	Monitors the health of resources.

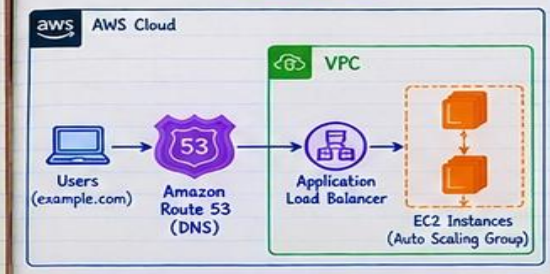
5) Route 53 Components



6) Routing Policies (How Traffic is Routed?)

Routing Policy	Use Case
Simple	Routes traffic to a single resource (default).
Weighted	Distribute traffic based on weights (e.g., A/B testing).
Latency-based	Routes users to region with lowest latency.
Geolocation	Routes based on user's geographic location.
Geoproximity	Routes based on location and also allows you to shift traffic (bias).
Failover	Routes traffic to a healthy resource (using health checks).
Multivalue Answer	Returns multiple IP addresses for high availability.

7) Architecture: Routing Internet Traffic to EC2 using Route 53



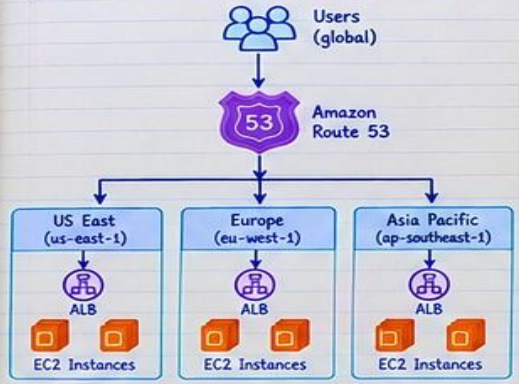
Flow of Traffic

1. User enters www.example.com in browser.
2. Route 53 resolves the domain and returns ALB DNS name / IP.
3. Traffic goes to Application Load Balancer.
4. ALB distributes traffic to healthy EC2 instances (Auto Scaling).
5. Health checks ensure unhealthy instances are removed.

8) Environment & Integration

	Purpose
EC2	Route traffic to EC2 instances.
Elastic Load Balancer	Route traffic to an ALB/NLB.
S3	Host static websites (Route 53 can point to S3).
CloudFront	Use Route 53 to point to CloudFront distribution.
Route 53 Resolver	Resolve internal domain names in VPC.
Health Checks	Monitor application and external endpoints.
Domain Registration	Register and manage domain names directly in Route 53.

9) Example Architecture: Global Application



Use Case:

- Latency-based routing policy routes users to the nearest region.
- Each region has its own ALB and EC2 instances.
- Provides low latency and high availability.
- If a region is unhealthy, failover policy can be used.

Real-World Example:
A global e-commerce website uses Route 53 with latency-based routing to provide best user experience to customers worldwide.

10) Common DNS Record Types

Record Type	Purpose
A	Maps a domain name to an IPv4 address (e.g., EC2, ALB, S3 website endpoint).
AAAA	Maps to an IPv6 address.
CNAME	Alias to another domain name.
Alias	AWS-specific CNAME that can point to AWS resources (e.g., ALB, CloudFront).
MX	Mail server records.
TXT	Text records (used for verification, SPF, etc.).
NS	Name server records (delegates a subdomain).
SOA	Start of authority record (zone information).
SRV	Specific services (e.g., SIP, XMPP).

11) Setting up a Domain with Route 53 (Steps)

1. Register a domain name (or use existing domain).
2. Create a hosted zone (public).
3. Add record sets (e.g., A, CNAME, Alias).
4. Choose a routing policy.
5. (Optional) Configure health checks.
6. Associate record with AWS resources (e.g., ALB, EC2, S3).
7. Wait for DNS propagation (few minutes).
8. Test the domain name.

12) Best Practices

- ✅ Use Alias records for AWS resources (no extra charges).
- ✅ Use health checks with failover routing.
- ✅ Use latency-based or geolocation routing for global users.
- ✅ Enable DNSSEC for improved security.
- ✅ Use low TTL values for faster changes (during testing).
- ✅ Monitor DNS queries using CloudWatch metrics.
- ✅ Use Route 53 Resolver for hybrid/on-premises environments.
- ✅ Tag hosted zones for better management.

13) Advantages and Limitations

Advantages

- ✔ Highly available and scalable.
- ✔ Wide range of routing policies.
- ✔ Integrates with many AWS services.
- ✔ Domain registration and management. Health checks for fault tolerance.
- ✔ Cost-effective (pay per query).
- ✔ Supports IPv6.
- ✔ Easy to use with AWS console, CLI, SDK.

Limitations

- ✖ Not a full DNS hosting solution for complex on-premises use cases.
- ✖ Only supports public and private DNS (no advanced DNS features like DNS templating).
- ✖ Charges for queries and health checks (though low).

14) Pricing (Simplified)

- $0.50 per hosted zone per month.
- $0.40 per million DNS queries (first 1 billion queries).
- Health checks: $0.50 per health check per month.
- Domain registration: varies by domain extension (e.g., .com).

15) Comparison

Feature	Route 53	Traditional DNS Provider
High Availability	Yes (AWS global network)	Varies
Routing Policies	Advanced (latency, failover, etc.)	Basic
Domain Registration	Yes	Yes
Integration with AWS	Native integration	Limited
Health Checks	Yes	Usually No
Cost	Pay per use (low cost)	Varies
Use Case	Cloud applications	General DNS hosting

16) Exam Tips (AWS Cloud Practitioner)

- Know that Route 53 is a DNS service, not just domain registration.
- Understand different routing policies.
- Know that Alias records are AWS-specific and free.
- Remember that health checks enable failover routing.
- Be familiar with common use cases (global applications, high availability, etc.).
- Identify the right routing policy for a given scenario.
- Know integration with ELB, CloudFront, S3, and EC2.
- Questions may ask which service is used for DNS → Answer: Route 53

Key Takeaway:
Route 53 helps you route users to the right application endpoint reliably, securely and efficiently, making your applications highly available and scalable.

AMAZON API GATEWAY

— Overview —



1) What is Amazon API Gateway ?

Amazon API Gateway is an AWS service for creating, publishing, maintaining, monitoring and securing REST, HTTP and WebSocket APIs.

Key Functions

1. Create and publish APIs.
2. Route requests to backend services.
3. Secure APIs (authentication and authorization).
4. Monitor and log API traffic.
5. Throttle and control access to APIs.

Types of APIs in API Gateway

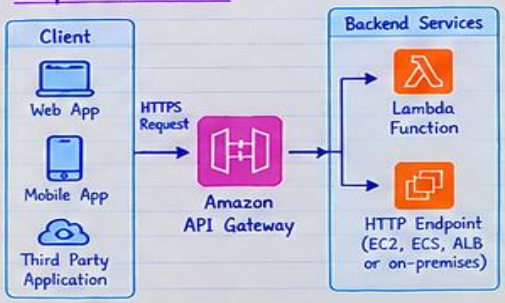
- **REST API** – Feature rich, supports more AWS features.
- **HTTP API** – Lightweight, high performance, supports OAuth 2.0 / OpenID Connect and CORS.
- **WebSocket API** – For real-time, two-way communication (eg. chat applications).

1) What is Amazon API Gateway ?

An API (Application Programming Interface) is an entry point between a client and a backend service. It allows different applications to communicate over HTTP/HTTPS.

Amazon API Gateway acts as a managed front door for your backend services, handling incoming requests, authorization, routing and more.

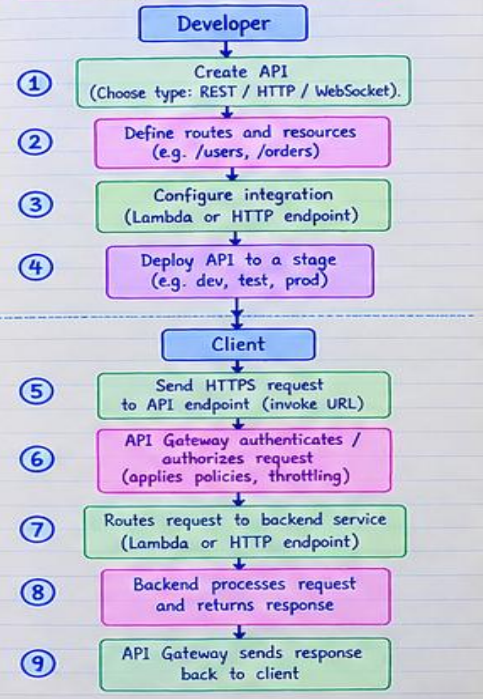
Simple Architecture



Why use API Gateway ?

- Single managed entry point for multiple backend services.
- Routes client requests to the right service.
- Supports authentication and authorization.
- Provides monitoring, logging and throttling.
- Helps decouple frontend and backend services.

2) Basic Working of API Gateway



3) Important Terminology

Term	Meaning
API	A collection of routes, methods and integrations (e.g. REST, HTTP, WebSocket).
Route / Resource	Defines the path (e.g. /users) that accepts requests.
Method	HTTP method like GET, POST, PUT, DELETE on a route.
Integration	The backend service that processes the request (Lambda, HTTP endpoint, ALB via VPC Link, etc.).
Stage	A logical environment (e.g. dev, prod) with its own configuration and endpoint URL.
Deployment	A snapshot of the API configuration that is deployed to a stage.
VPC Link	Enables private integration with resources in a VPC (VPC Link V2 supports ALB or NLB).
Endpoint	API endpoint URL used by clients to invoke the API (public, private or custom domain).
Authorization	Controls who can access the API (IAM, Lambda authorizer, Cognito, OAuth 2.0).

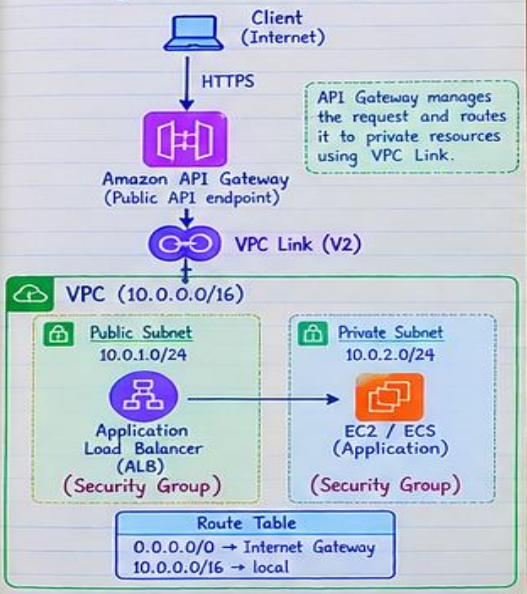
REST API vs HTTP API

Feature	REST API	HTTP API
Use case	Feature rich, complex APIs	Simpler, high performance
Authorization	IAM, Cognito, Lambda, etc.	JWT, OAuth 2.0, OIDC
CORS	Supported	Built-in and simpler
Cost	Higher	Lower
Best for	Legacy or complex needs	Modern, simple APIs

Both support integration with Lambda and HTTP endpoints.

4) Scenario – Client to Private Backend

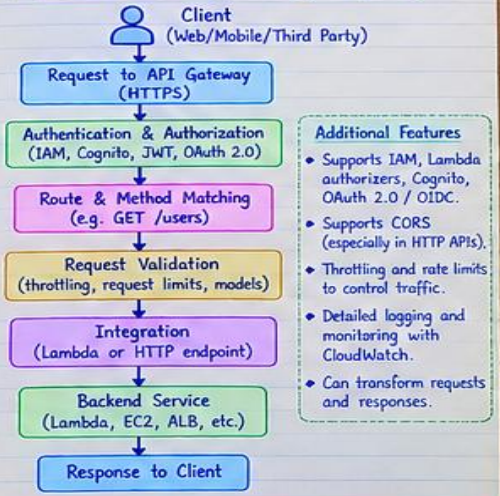
Using VPC Link (Private Integration)



API Gateway manages the request and routes it to private resources using VPC Link.

VPC Link V2 supports private integrations with Application Load Balancers (ALB) or Network Load Balancers (NLB).

5) How API Gateway manages requests



Additional Features

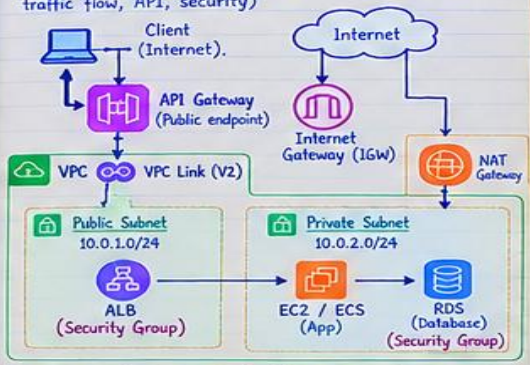
- Supports IAM, Lambda authorizers, Cognito, OAuth 2.0 / OIDC.
- Supports CORS (especially in HTTP APIs).
- Throttling and rate limits to control traffic.
- Detailed logging and monitoring with CloudWatch.
- Can transform requests and responses.

Q: Explain how API Gateway can help manage request between client and backend service.

A: API Gateway acts as a managed front door, validates and authenticates requests, routes them to the correct backend service, controls traffic (throttling/rate limits), monitors and logs requests, and returns the response to the client.

6) VPC + Routing + Security Scenario

Design an architecture for any scenario: (public/private subnet, routing table, gateway, traffic flow, API, security)



Route Table (Public Subnet)	
Destination	Target
0.0.0.0/0	Internet Gateway
10.0.0.0/16	local

Route Table (Private Subnet)	
Destination	Target
0.0.0.0/0	NAT Gateway
10.0.0.0/16	local

Analyze how routing table influence traffic flow

- Route tables decide where subnet traffic goes (e.g. Internet or NAT).
- Public subnet routes 0.0.0.0/0 to IGW.
- Private subnet routes 0.0.0.0/0 to NAT Gateway.
- All subnets have a local route for VPC CIDR.

Analyze how security group can be configured

- ALB SG: allow inbound 443 from 0.0.0.0/0, allow to backend on 80.
- EC2/ECS SG: allow inbound from ALB SG only, restrict others.
- RDS SG: allow inbound from application SG only.
- Security groups are stateful (return traffic allowed automatically).

7) API Gateway vs Alternatives

Feature	API Gateway	Application LB (ALB)	Direct EC2 Endpoint
Primary use	Manage and secure APIs	Load balance HTTP/HTTPS traffic	Expose single server
Protocol support	REST, HTTP, WebSocket	HTTP, HTTPS	Varies
Authorization	IAM, Cognito, Lambda, OAuth2	Basic (via) security groups)	Self managed
Routing	Routes to Lambda/HTTP	Routes to multiple targets (EC2, ECS)	No routing (single instance)
Private integration	Yes (VPC Link V2)	Yes (within VPC)	N/A
Monitoring	CloudWatch (detailed)	CloudWatch	Manual
Best for	APIs, serverless, modern apps	Web applications	Simple / testing

Deployment & Management

- API Gateway provides stages (dev/test/prod) and deployments.
- Custom domain names can be used with Route 53.
- Integrates with CloudWatch for logs, metrics and alarms.
- Supports throttling, request validation and transformation.

Exam Traps

1. API Gateway is NOT a database.
2. VPC Link is used for private integration with ALB or NLB.
3. Private REST API can be called only from within a VPC (through an interface VPC endpoint)
4. HTTP APIs support OAuth 2.0 / OpenID Connect and CORS.
5. Route tables control subnet traffic flow, not security groups.
6. Security groups allow rules and are stateful.
7. API Gateway can integrate with Lambda.

Quick Revision (8 points)

- ✔ Managed service to create and publish APIs.
- ✔ Supports REST, HTTP and WebSocket APIs.
- ✔ Routes requests to Lambda or HTTP endpoints.
- ✔ VPC Link V2 integrates with ALB/NLB.
- ✔ Private APIs are accessible only within a VPC.
- ✔ HTTP APIs support OAuth 2.0/OIDC and CORS.
- ✔ Provides throttling.
- ✔ Helps secure, throttle and manage API traffic.

AMAZON VPC

– Overview –

Amazon Virtual Private Cloud (VPC)

"A logically isolated private network in the AWS cloud where you can launch and manage resources securely."

1) What is Amazon VPC ?

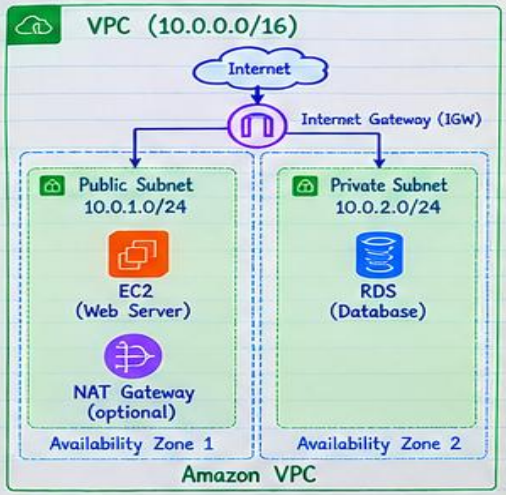
Amazon VPC (Virtual Private Cloud) is a logically isolated network in AWS that lets you launch AWS resources in a virtual network that you define.

- Provides complete control over virtual networking environment.
- Allows you to define IP address ranges, subnets, route tables, network gateways and security settings.
- Resources in a VPC are isolated from other VPCs and the public internet (unless explicitly configured).
- Functions like a virtual data center in the cloud.
- Helps in building secure, scalable and highly available applications.

2) Why do we need VPC ?

- Network isolation and security.
- Custom IP address planning.
- Control over traffic routing.
- Connect to on-premises networks.
- Fine-grained access control (Security Groups, NACLs).
- Multi-tier architecture (public and private subnets).
- Compliance and data protection.

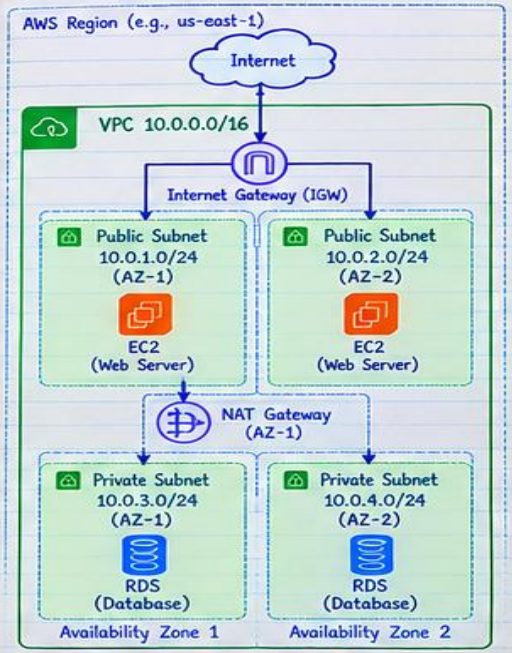
3) Key Components of VPC



4) Important Terminology

Term	Meaning
VPC	Logically isolated virtual network in AWS.
CIDR Block	IP address range for the VPC (e.g., 10.0.0.0/16).
Subnet	A range of IP addresses in a VPC.
Availability Zone	Physically separate data center within a region.
Internet Gateway (IGW)	Allows communication between VPC and the internet.
NAT Gateway	Allows private subnet instances to access the internet.
Route Table	Controls traffic routing in the VPC.
Security Group	Stateful firewall at instance level.
Network ACL (NACL)	Stateless firewall at subnet level.

5) VPC Architecture – Multi AZ (Example)



6) Features of Amazon VPC

1. **Isolation & Security** – logically isolated network.
2. **Custom IP Addressing** – define your own CIDR block.
3. **Subnets** – divide network into public and private subnets.
4. **Routing** – control traffic using route tables.
5. **Multiple AZs** – build highly available architectures.
6. **Connectivity** – connect to on-premises using VPN/Direct Connect.
7. **Secure Access** – Security Groups and Network ACLs.
8. **Scalable** – easily expand network and add resources.
9. **IPv6 Support** – dual-stack (IPv4 and IPv6).

7) Subnets

A subnet is a range of IP addresses in your VPC.
You can create subnets in each Availability Zone.

Public Subnet

- Has a route to the Internet Gateway (IGW).
- Instances can have public IP and are accessible from internet.
- Used for web servers, load balancers, bastion hosts, etc.

Private Subnet

- No direct route to IGW.
- Used for backend resources (e.g., databases, application servers).
- Can access internet using NAT Gateway (for outbound traffic).

8) Route Table

A route table contains a set of rules called routes that determine where network traffic is directed.

Destination	Target (Example)
10.0.0.0/16	local (VPC internal communication)
0.0.0.0/0	Internet Gateway (for public subnet)
0.0.0.0/0	NAT Gateway (for private subnet)
172.16.0.0/12	Virtual Private Gateway (VPN)

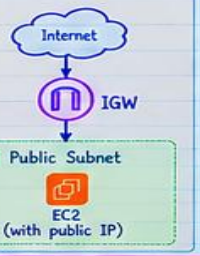
9) Security Group vs Network ACL

Security Group (SG)	Network ACL (NACL)
Stateful firewall (allow return traffic automatically).	Stateless firewall (need explicit rules for both).
Instance level (attached to ENI).	Subnet level (associated with subnets).
Only ALLOW rules.	Both ALLOW and DENY rules.
Evaluated before traffic is allowed.	Evaluated in order (lowest number first).
Default: Denies all inbound, allows all outbound.	Default: Allows all inbound and outbound.

10) Internet & NAT Gateway

Internet Gateway (IGW)

- Connects VPC to the internet.
- Horizontally scaled, highly available.
- Allows communication between instances in public subnets and the internet.
- One IGW per VPC.
- Must be explicitly attached to the VPC.



NAT Gateway

- Allows instances in private subnets to access the internet. (for outbound traffic only).
- Blocks unsolicited inbound traffic from internet.
- Managed service, highly available within an AZ.
- Create one per AZ for high availability.
- Use case: software updates, calling external APIs, etc.



Note: NAT Gateway allows outbound internet access only.
For inbound access to private instances, use bastion host, VPN or Direct Connect.

11) VPC Connectivity Options

1) VPC Peering

- Connects two VPCs (same or different regions).
- Allows private IP communication.
- No transitive peering (VPC A ↔ VPC B ↔ VPC C not allowed).



2) Site-to-Site VPN

- Establishes a secure encrypted connection between on-premises network and VPC.



3) AWS Direct Connect

- Dedicated private network connection from data center to AWS (more reliable and fast).



12) Use Cases

1. **Web applications** – public subnets for web tier, private subnets for database tier.
2. **Enterprise network extension** – connect on-premises using VPN/Direct Connect.
4. **Multi-tier applications** – secure and scalable architecture.
4. **Development and testing environments** – isolated networks.
5. **Data security and compliance** – control traffic and access.
5. **Hybrid cloud architectures** – integrate with on-premises networks.
7. **Disaster recovery** – replicate resources across AZs/regions.
8. **Microservices and containerized applications** – using VPC, subnets, and security groups.

Best Practices

- ✓ Use at least two AZs for high availability.
- ✓ Create separate public and private subnets.
- ✓ Use NAT Gateway for private subnet internet access.
- ✓ Keep subnets small and purpose-specific.
- ✓ Use Security Groups (least privilege access).
- ✓ Use Network ACLs for additional subnet-level security.
- ✓ Enable VPC Flow Logs for monitoring and troubleshooting.
- ✓ Plan IP addressing to avoid overlap with on-premises networks.
- ✓ Use VPC endpoints to access AWS services privately.
- ✓ Regularly review and update route tables, SGs and NACLs.

13) Limitations (Important for Exam)

- ⚠ Each VPC must have a unique CIDR block (cannot overlap with other VPCs or on-premises).
- ⚠ You cannot change the size of the primary CIDR block after creation (you can add secondary CIDR blocks).
- ⚠ Default limits: 5 VPCs per region (can increase), 5 CIDR blocks per VPC (1 primary + 4 secondary).
- ⚠ VPC peering is not transitive.
- ⚠ NAT Gateway is AZ-specific (create in each AZ for HA).
- ⚠ Internet Gateway is region-specific (not cross-region).
- ⚠ Security Groups are stateful, NACLs are stateless.
- ⚠ Charge applies for NAT Gateway, VPN, and Direct Connect.

Key Takeaway

- ✔ VPC is the foundation of your AWS network.
- ✔ It provides isolation, security, and control over resources.
- ✔ Use public and private subnets, route tables, gateways, and security groups to design secure architectures.
- ✔ Connect with on-premises using VPN or Direct Connect when needed.
- ✔ Always design for high availability and follow best practices.

ROUTE 53 + API GATEWAY + VPC – SCENARIOS (Q & A)



Key Points:

- **Route 53** → DNS (domain → API endpoint)
- **API Gateway** → Public endpoint (HTTPS)
- API Gateway → Integrates with private resources in VPC (via VPC Link)
- **VPC** → Private subnets, route tables, security
- Use PrivateLink (VPC Link) for private integrations

1 Basic Integration Scenario

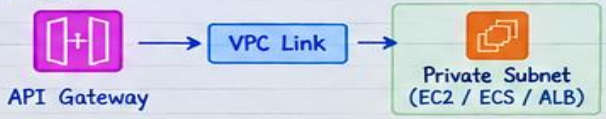
Q: A company wants to expose an internal microservice running on EC2 (in private subnet) to the internet using a custom domain (api.example.com). Which AWS services should be used?



Ans: Use Route 53 (for custom domain) + API Gateway (for HTTPS endpoint) + VPC Link (to connect to private EC2 in private subnet).

2 VPC Link Requirement

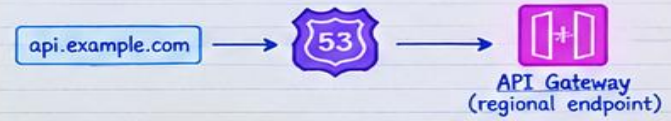
Q: Your API in API Gateway needs to access a service running in a private subnet (EC2/ECS). Which feature should be used to securely connect API Gateway to a VPC?



Ans: Use **VPC Link** (AWS PrivateLink) to connect API Gateway to private resources in the VPC.

3 Routing with Route 53

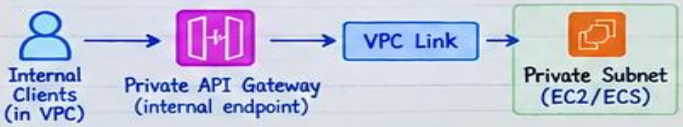
Q: You have created an API in API Gateway with a regional endpoint. You want users to access it using https://api.example.com. What should you do in Route 53?



Ans: Create an **A record** (Alias) in Route 53 pointing to the API Gateway regional endpoint or custom domain name provided by API Gateway.

4 Private Integration (No Public Exposure)

Q: A company wants the API to be accessible only from within their VPC (internal applications), not from the internet. What should they use?

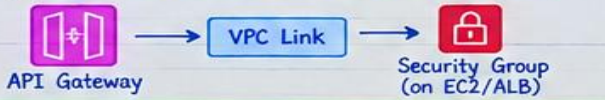


Ans: Use a Private API Gateway (internal endpoint) with **VPC Link** and access it using VPC DNS / Private Hosted Zone in Route 53.

5 Security Configuration

Q: Which security component controls inbound/outbound traffic to the backend EC2 instance accessed via API Gateway?

API Gateway does not use Security Groups. It connects via VPC Link to backend resources.



Ans: Security Group (attached to EC2/ALB) controls traffic. NACL is subnet-level (stateless), but SG is the primary control here.

6 Route Table and NAT Gateway

Q: Your backend service in private subnet needs to call an external API on the internet (e.g., third-party service). Which component and route table configuration is required?



Ans: Use a **NAT Gateway** in a public subnet and add a route (0.0.0/0 → NAT Gateway) in the private subnet route table.

7 Custom Domain with HTTPS

Q: You want to enable HTTPS for your custom domain (api.example.com) in Route 53 and API Gateway. What additional service is used?



Ans: Use AWS Certificate Manager (**ACM**) to provision SSL/TLS certificate and attach it to the API Gateway custom domain, then create an Alias record in Route 53.

8 Architecture Design Question

Q: Design an architecture for a public API that uses a custom domain, routes through API Gateway, and connects to an internal service in a VPC (multi-AZ).



Ans: Use **Route 53** + API Gateway (regional/custom domain) + **VPC Link** + private subnets across multiple AZs for high availability.

9 Troubleshooting Scenario

Q: Your API is returning 504 (Gateway Timeout). Backend EC2 is in private subnet and security group allows traffic from the VPC Link. What could be the issue?

504 usually means API Gateway could not reach the backend.

A) Incorrect VPC Link configuration.
B) Missing route in the private subnet route table.
C) Security group blocking traffic.
D) EC2 instance is stopped.
E) All of the above.

Ans: E) **All of the above.**
(Any of these can cause 504 error.)

Key Takeaways:

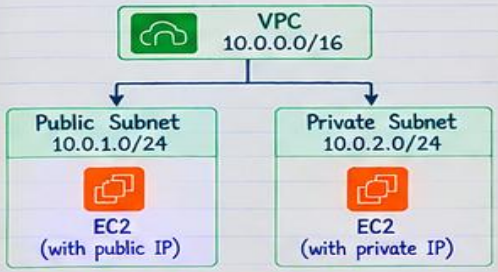
- Route 53 is used for DNS and custom domains.
- API Gateway provides a managed, secure HTTP interface.
- **VPC Link** (PrivateLink) integrates API Gateway with private resources.
- Security Groups control instance traffic; NACL is subnet-level.
- Use NAT Gateway for outbound internet access from private subnets.

Common Exam Points:

- ✓ Use VPC Link for private integrations.
- ✓ Use Alias record in Route 53 for API Gateway.
- ✓ Security Group is stateful (allow), NACL is stateless (allow/deny).
- ✓ Private API is accessible only within VPC.
- ✓ NAT Gateway is needed for private subnet to access internet.

VPC SUBNETS & ROUTE TABLES

— Overview —



1 What are Subnets ?

A subnet is a range of IP addresses in your VPC. You can create subnets in each Availability Zone (AZ). Subnets divide your VPC into smaller, manageable network segments.

Key Points:

- Each subnet must belong to one AZ.
- Subnets use a subset of the VPC CIDR block.
- A subnet can be public or private.
- You can create multiple subnets per AZ.
- Subnets help in high availability and better resource organization.

2 Types of Subnets

Public Subnet	Private Subnet
• Has a route to the Internet Gateway (IGW). • Instances can have public IP and are accessible from internet. • Used for web servers, load balancers, bastion hosts.	• No direct route to IGW. • Used for backend resources (e.g., databases, application servers). • Access internet using NAT Gateway (for outbound traffic) if needed.

Subnet Example (from source content)

VPC CIDR	: 10.0.0.0/16	
Public Subnet	: 10.0.1.0/24	(us-east-1a)
Private Subnet	: 10.0.2.0/24	(us-east-1a)
Another Private	: 10.0.3.0/24	(us-east-1b)

Subnets in different AZs provide high availability.

3 Subnet CIDR Rules

- Subnet CIDR must be a subset of the VPC CIDR.
- Subnets cannot overlap.
- Minimum subnet size: /28 (16 IP addresses).
- Maximum subnet size: /16 (but cannot be larger than VPC CIDR).
- AWS reserves 5 IP addresses in each subnet (e.g., 10.0.1.0 – 10.0.1.4).

4 Reserved IP Addresses in a Subnet

IP Address	Purpose	Example (for 10.0.1.0/24)
Network address	Identifies the network	10.0.1.0
VPC router	Default gateway for the subnet	10.0.1.1
DNS resolver	Amazon provided DNS	10.0.1.2
Future use	Reserved by AWS	10.0.1.3
Broadcast address	Network broadcast	10.0.1.255

So total 5 IPs are reserved. Usable IPs in /24 = 251

5 What are Route Tables ?

A route table contains a set of rules (routes) that determine where network traffic is directed.

Key Points:

- Every subnet must be associated with a route table.
- You can create custom route tables.
- A route table can be associated with multiple subnets (usually in same VPC).
- Each VPC has a main route table (used by default).
- Routes define the destination and target (where to send traffic).

Route Table Components

Destination	The IP range (CIDR) to match.
Target	The destination for the traffic (e.g., IGW, NAT Gateway, VPC Peering, local, etc.)
Subnet Association	Which subnets use this route table.

6 Common Route Table Entries

Destination	Target	Use Case
10.0.0.0/16	local	VPC internal communication
0.0.0.0/0	Internet Gateway (IGW)	Internet access (from public subnet)
0.0.0.0/0	NAT Gateway	Internet access (from private subnet)
172.16.0.0/12	Virtual Private Gateway	Connect to on-premises (VPN)
10.1.0.0/16	VPC Peering	Connect to another VPC

7 Main Route Table vs Custom Route Table

Main Route Table	Custom Route Table
• Created automatically with VPC. • Used by default for subnets (with no explicit association). • Cannot be deleted. • Can be modified.	• Created by you. • Associate with specific subnets. • Can be modified or deleted (if not in use). • Useful for public/private separation.

Tip: You can have multiple route tables in a VPC, but each subnet can be associated with only one route table at a time.

8 Route Table Example

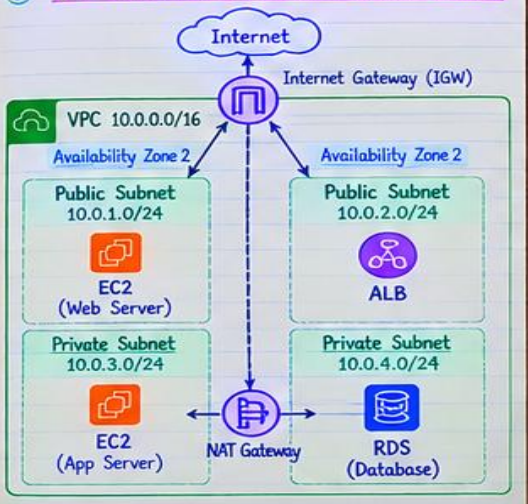
Public Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxx (Internet Gateway)

Private Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-xxxx (NAT Gateway)

9 VPC Architecture with Subnets & Route Tables



10 How Traffic Flows ?

a) From Public Subnet (to Internet)

1. EC2 in public subnet sends traffic to 0.0.0.0/0.
2. Route table points to Internet Gateway (IGW).
3. Traffic goes to internet.

b) From Private Subnet (to Internet – Outbound only)

1. EC2 in private subnet sends traffic to 0.0.0.0/0.
2. Route table points to NAT Gateway (in public subnet).
3. NAT Gateway sends traffic to internet via IGW.
4. Response comes back through NAT Gateway.
5. Instances in private subnet cannot be directly accessed from internet.

c) Internal VPC Communication

1. Destination 10.0.0.0/16 with target local.
2. Allows communication between all subnets in VPC.

11 Key Use Cases

1. Web applications – public subnets for web tier, private subnets for app & database tier.
2. Multi-AZ deployments – subnets in multiple AZs.
3. Secure architectures – keep backend resources private.
4. Hybrid cloud – connect to on-premises using VPN/Direct Connect.
5. Microservices – separate subnets for different services.
6. Compliance – control network access using route tables, NACLs and security groups.

12 Best Practices

- ☑ Use at least two AZs for high availability.
- ☑ Separate public and private subnets.
- ☑ Use NAT Gateway (not NAT instance) for outbound internet.
- ☑ Keep subnet CIDR blocks small and purpose-specific.
- ☑ Use custom route tables instead of default when needed.
- ☑ Use security groups and NACLs for layered security.
- ☑ Regularly review and update routes.
- ☑ Enable VPC Flow Logs for monitoring.
- ☑ Follow least privilege network access.

13 Common Exam Questions (with Answers)

Q1. You have an EC2 instance in a private subnet. It needs internet access for software updates. Which component allows this?
A: NAT Gateway

Q2. Which route table entry is required for a public subnet to access the internet?
A: 0.0.0.0/0 → Internet Gateway (IGW)

Q3. How many IP addresses are reserved in each subnet by AWS?
A: 5

Q4. Can a subnet's CIDR block overlap with another subnet in the same VPC?
A: No

Q5. Which target is used in a route table for communication within the VPC?
A: local

Q6. You want to connect your VPC to an on-premises network using a VPN. What should be the route table target?
A: Virtual Private Gateway (VGW)

Q7. Can EC2 instances in a private subnet be directly accessed from the internet?
A: No

Q8. Which service is used to define a range of IP addresses within a VPC?
A: Subnet

Q9. If a subnet is not explicitly associated with a route table, which route table is used?
A: Main route table

Q10. You want to restrict inbound and outbound traffic at the subnet level. Which component will you use?
A: Network ACL (NACL)

Key Takeaway:

- Subnets divide your VPC into smaller networks.
- Route tables control where traffic goes.
- Public subnet → IGW, Private subnet → NAT (for outbound).
- Use multiple AZs for high availability.
- Combine Route Tables, Security Groups and NACLs for security.
- Subnet CIDR must be a subset of VPC CIDR (no overlap).

AWS SECURITY GROUPS

— Overview —



"Security Groups act as a virtual firewall for your AWS resources, controlling inbound and outbound traffic at the instance level."

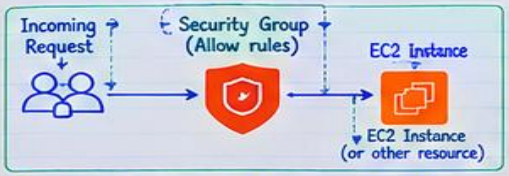
1) What is a Security Group ?

A Security Group (SG) is a stateful virtual firewall that controls inbound and outbound traffic to AWS resources such as EC2 instances, RDS, Lambda (function URLs), Elastic Load Balancers, etc.

Key Points:

- Works at the instance/resource level (not subnet).
- Stateful – return traffic is automatically allowed.
- Only ALLOW rules (no explicit DENY).
- Can reference IP addresses, CIDR ranges, or other SGs.
- Applied to resources in a VPC.
- Helps in securing resources by controlling traffic based on protocols, ports and source/destination.

2) How Security Groups Work ?



1. A request comes to as AWS resource (e.g., EC2).
2. Security Group checks the inbound rules.
3. If the rule allows the traffic, it reaches the instance.
4. For response traffic, SG automatically allows it (stateful).
5. Outbound rules control what the instance can initiate.

3) Stateful vs Stateless

Feature	Security Group	Network ACL
Type	Stateful	Stateless
Apply at	Instance level	Subnet level
Rules	Only ALLOW	ALLOW and DENY
Return traffic	Automatically allowed	Must be explicitly allowed
Rule evaluation	All rules evaluated	Evaluated in order (rule number)
Use case	Instance security	Subnet-level control
Association	Multiple SGs per instance	One NACL per subnet

" Security Groups are stateful, so you don't need to create outbound rules for response traffic.

4) Security Group Components

Component	Description
Inbound Rules	Control incoming traffic to the instance. Define protocol, port range and source (IP, CIDR or SG).
Outbound Rules	Control outgoing traffic from the instance. Define protocol, port range and destination.
Type	Traffic type (e.g., HTTP, SSH, MySQL, Custom TCP/UDP, All traffic).
Protocol	TCP, UDP, ICMP, or All.
Port Range	Specific port or range (e.g., 80, 22, 3306, 0-65535).
Source/Destination	IP address, CIDR range (e.g., 0.0.0.0/0) or another Security Group.
Security Group Reference	Allows traffic from another SG instead of an IP (helps in dynamic environments).
Rule ID	Unique ID for each rule.
Description	Optional note about the rule purpose.

Important:

- You can attach multiple security groups to an instance.
- Rules are evaluated all together (no priority).
- You can reference another security group as a source (e.g., ALB SG to EC2 SG).
- Changes take effect immediately.

5) Security Group Rule Examples

Example 1 – Allow SSH from specific IP

Type	Protocol	Port	Source
SSH	TCP	22	203.0.113.5/32

Example 2 – Allow HTTP from anywhere

Type	Protocol	Port	Source
HTTP	TCP	80	0.0.0.0/0

Example 3 – Allow traffic from another SG

Type	Protocol	Port	Source
Custom TCP	TCP	3306	sg-xxxxxxxx (RDS SG)

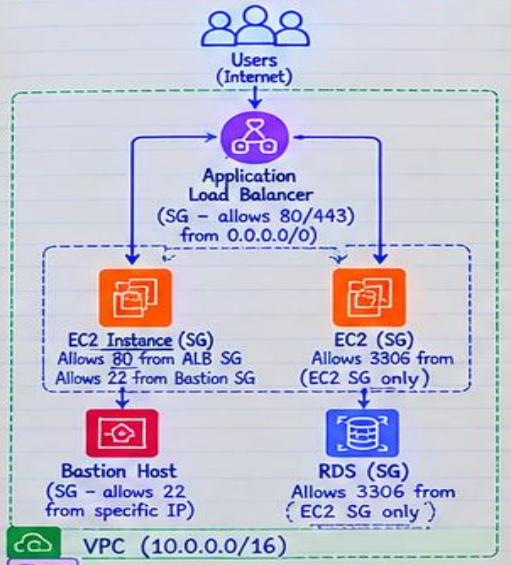
Example 4 – Allow all outbound traffic

Type	Protocol	Port	Source
All traffic	All	All	0.0.0.0/0

Note:

- Use 0.0.0.0/0 carefully (open to internet).
- For production, restrict to specific IP ranges.
- Use security group references instead of IPs where possible (e.g., ALB SG → EC2 SG).
- Outbound rules are usually kept as "All traffic" unless restricted by compliance.

6) Architecture Example (Web App)



Flow:

1. Users access ALB on port 80/443.
2. ALB forwards traffic to EC2 (allowed from ALB SG).
3. EC2 connects to RDS (allowed from EC2 SG).
4. Bastion host allows SSH from specific IP only.

7) Common Scenarios & Correct Answers

#	Scenario	Correct Answer
1	Allow EC2 to accept HTTP traffic from internet.	Inbound rule: TCP 80, Source: 0.0.0.0/0
2	Allow SSH from your office IP 203.0.113.0/24.	Inbound rule: TCP 22, Source: 203.0.113.0/24
3	Allow EC2 to connect to RDS on port 3306.	In EC2 SG: Outbound 3306 to RDS SG (or RDS SG ref). In RDS SG: Inbound 3306 from EC2 SG.
4	Allow communication between two EC2 instances in same SG.	Already allowed (stateful).
5	Restrict outbound internet access from EC2.	Modify outbound rules to specific IP/ports (remove 0.0.0.0/0).
6	Allow traffic between instances in different SGs.	Add inbound rule in target SG with source as the other SG.
7	You want to block a specific IP from accessing EC2.	Not possible in SG (no DENY). Use NACL or restrict allowed IPs.
8	After adding an inbound rule, it's not working.	Check: correct SG attached, correct port/protocol, source IP, NACL, route table, and instance OS firewall.

8) Best Practices

- ✅ Use least privilege access (allow only required ports/IPs).
- ✅ Use security group references instead of IP addresses.
- ✅ Avoid using 0.0.0.0/0 in production.
- ✅ Separate security groups for different tiers (web, app, db).
- ✅ Use descriptive names and rule descriptions.
- ✅ Regularly review and remove unused rules.
- ✅ Combine with NACLs for subnet-level control.
- ✅ Use Infrastructure as Code (CloudFormation/Terraform).
- ✅ Monitor changes using AWS CloudTrail and Config.
- ✅ Follow organizational security policies and compliance requirements.

9) Limitations

- Only ALLOW rules (no explicit DENY).
- Stateful, so not suitable for blocking specific IPs.
- Applied at instance level, not subnet level.
- Rule evaluation is not in order (all rules are evaluated).
- Does not control traffic between subnets (use NACLs).
- Used only within a VPC (cannot be used outside AWS).

Key Takeaways:

- ✔ Security Groups = stateful, instance-level firewall.
- ✔ Only allow rules, no deny.
- ✔ Use SG references for dynamic environments.
- ✔ Works with VPC resources (EC2, RDS, ALB, etc.).
- ✔ Combine with NACLs for complete network security.

10) Exam Traps (MCQ with Answers)

Q1. Security Groups are:

A) Stateless B) Stateful
C) Subnet level D) Support deny rules

Answer: B

Q2. Which of the following can be used as a source in a security group rule?

A) IP address B) CIDR range
C) Another security group D) All of the above

Answer: D

Q3. You want to block a specific IP from accessing an EC2 instance. What should you use?

A) Security Group B) Network ACL
C) Route Table D) Internet Gateway

Answer: B

Q4. Can you attach multiple security groups to an instance?

A) Yes B) No
C) Only one per VPC D) Only for ALB

Answer: A

Q5. By default, security groups allow:

A) All inbound and outbound traffic
B) No inbound traffic and all outbound traffic
C) All inbound traffic and no outbound traffic
D) No inbound and no outbound traffic

Answer: B

AWS NAT GATEWAY

— Overview —



① What is NAT Gateway ?

A NAT (Network Address Translation) Gateway is a managed AWS service that allows instances in a private subnet to connect to the internet for outbound traffic, while blocking unsolicited incoming traffic from the internet.

Key Points:

- Provides internet access to private resources.
- Only allows outbound traffic (no inbound).
- Managed by AWS (no maintenance).
- Uses Elastic IP (public IP) for outbound traffic.
- Highly available within an Availability Zone.
- Used for software updates, external APIs, etc.

"NAT Gateway enables private resources to access the internet, securely and easily."

② How NAT Gateway Works ?



1. Private instance initiates an outbound request (e.g. to download software).
2. Request goes to the NAT Gateway.
3. NAT Gateway replaces the private source IP with its Elastic IP (public IP).
4. Internet receives the request and responds to the NAT Gateway.
5. NAT Gateway forwards the response back to the private instance.

NAT Gateway tracks the connection and only allows responses to the outbound requests. It drops any new connections initiated from the internet."

③ When to Use NAT Gateway ?

Use NAT Gateway when:

- ✅ Instances in private subnets need internet access for outbound traffic.
- ✅ You want a fully managed, highly available solution.
- ✅ You don't want to manage NAT instances.
- ✅ You need to access external services (e.g. software updates, package repositories, external APIs).
- ✅ You want better availability and scalability.

Do NOT use NAT Gateway when:

- ❌ You need inbound traffic from the internet (use Internet Gateway + public subnet).
- ❌ You want a lower-cost option and can manage NAT instances yourself.
- ❌ Your resources should not access the internet at all (use VPC endpoints instead).

Common Use Cases

- Downloading OS updates.
- Accessing external APIs (e.g. payment gateway).
- Pulling container images from Docker Hub.
- Accessing S3/other AWS services (alternatively use VPC Endpoints).
- Connecting to third-party services.

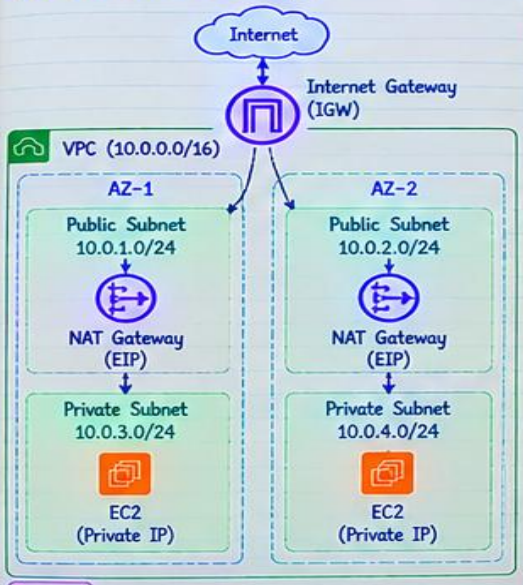
④ NAT Gateway vs NAT Instance

Feature	NAT Gateway	NAT Instance
Type	Managed service	EC2 instance (self-managed)
Availability	Highly available (within AZ)	You must manage high availability
Maintenance	No maintenance	You manage patching, updates
Scalability	Automatic scaling	Manual scaling
Cost	Higher	Lower
Bandwidth	Up to 45 Gbps	Depends on instance type
Use case	Recommended for most cases	When you need custom control or lower cost

Conclusion:

Use NAT Gateway for production and ease of use.
Use NAT Instance only if you have specific requirements to control the NAT instance or reduce cost.

⑤ Architecture Example



Note:

- Place NAT Gateway in a public subnet.
- Each AZ should have its own NAT Gateway for high availability.
- Private subnets route 0.0.0.0/0 to the NAT Gateway.

⑥ Route Table Configuration

Public Subnet Route Table

Destination	Target	Use Case
10.0.0.0/16	local	VPC internal communication
0.0.0.0/0	Internet Gateway (IGW)	Access to internet

Private Subnet Route Table

Destination	Target	Use Case
10.0.0.0/16	local	VPC internal communication
0.0.0.0/0	NAT Gateway	Outbound internet access (no inbound)

Important Points:

- NAT Gateway must be in a public subnet.
- Private subnet route table should use NAT Gateway as the target for 0.0.0.0/0.
- Public subnet must have a route to Internet Gateway.
- Each private subnet in an AZ should use the NAT Gateway in the same AZ (best practice).

⑦ Security Considerations

- NAT Gateway allows only outbound traffic.
- Security Groups on private instances control which outbound traffic is allowed.
- You can restrict outbound access using:
  - Specific ports (e.g. 443 for HTTPS)
  - Specific destination IPs or CIDR ranges
- NACLs at subnet level are stateless, so you must allow both outbound and inbound (return) traffic.
- Use VPC Endpoints (e.g. S3, DynamoDB) to access AWS services without going through NAT Gateway.
- NAT Gateway does not perform deep inspection or filtering. Use AWS Network Firewall if you need advanced filtering.

⑧ Limitations & Important Facts

- Works only for IPv4 (not for IPv6).
- Must be in a public subnet with a route to IGW.
- Charges per hour and per GB of data processed.
- No inbound connections (drops unsolicited traffic).
- Bandwidth up to 45 Gbps (scales automatically).
- AZ specific (create one per AZ for high availability).
- If you delete the NAT Gateway, private instances lose internet access.

⑨ Common Exam Questions (with Answers)

Q1. Which AWS service allows instances in a private subnet to access the internet?
A: NAT Gateway

Q2. Does NAT Gateway allow inbound traffic from the internet?
A: No

Q3. In which subnet should you create a NAT Gateway?
A: Public subnet

Q4. Is NAT Gateway region-specific or AZ-specific?
A: AZ-specific

Q5. Which route table entry is required in a private subnet to access the internet via NAT Gateway?
A: 0.0.0.0/0 → NAT Gateway

Q6. Can NAT Gateway be used for IPv6 traffic?
A: No (IPv4 only)

Q7. What is a better option to access S3 from a private subnet without using NAT Gateway?
A: VPC Endpoint (Gateway Endpoint for S3)

Key Takeaways:

- ✔ NAT Gateway = outbound internet access for private subnets.
- ✔ Managed, highly available, no maintenance.
- ✔ Place in public subnet, use in same AZ.
- ✔ Only for IPv4, no inbound, charges apply.
- ✔ Use VPC Endpoints for AWS services to save cost.

CSE3015 – AWS Cloud Practitioner
VIT Bhopal

END-TO-END ARCHITECTURE

Route 53 + API Gateway + VPC

(with Subnets, Route Tables, Security Groups, NAT GW)

1 Scenario (from your notes)

Design an architecture for:

1. Public/Private subnets
2. Route tables
3. Gateway (Internet/NAT)
4. Traffic flow
6. API (API Gateway)
6. Security (Security Groups)
7. How they work together

1) What is each service ?

Route 53

- DNS service.
- Maps domain (e.g. api.example.com)
- to API Gateway.

API Gateway

- Managed service to create, deploy, and manage APIs (HTTP/REST).
- Integrates with private resources using VPC Link.

VPC

- Isolated network to launch AWS resources.
- You define CIDR, subnets, route tables.

Subnet

- Range of IP addresses in a VPC.
- Public – access to internet (via IGW).
- Private – no direct internet access.

Route Table

- Set of rules (routes) to determine where traffic is sent.

Security Group

- Stateful firewall (instance/ENI level).
- Controls inbound and outbound traffic.

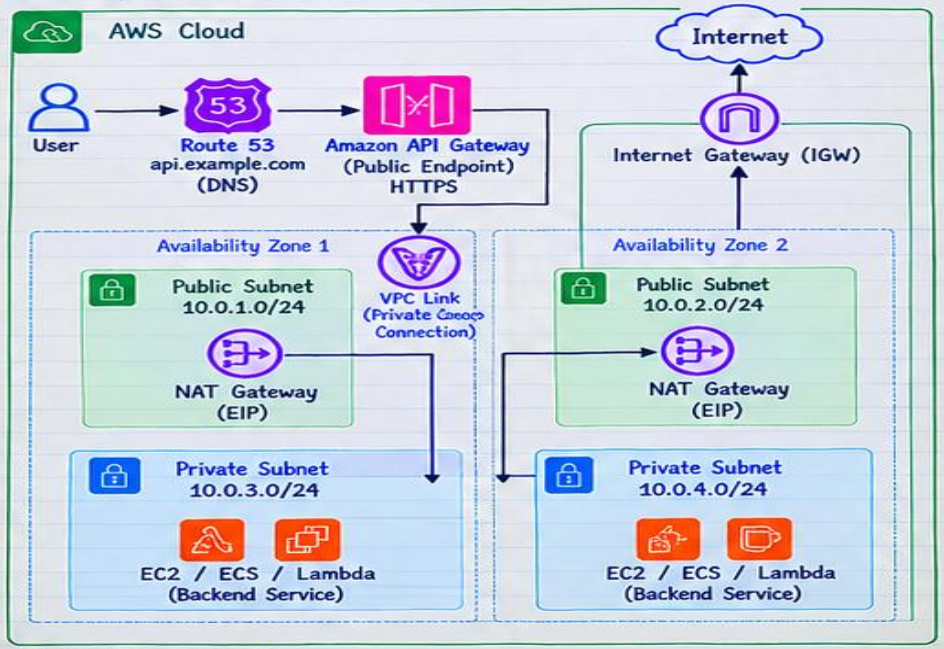
NAT Gateway

- Managed service for outbound internet access from private subnets (no inbound).
- Uses Elastic IP (public IP).

2) High Level Flow

1. User calls https://api.example.com
2. Route 53 resolves domain to API Gateway.
3. API Gateway receives request (public endpoint where applicable).
4. API Gateway uses VPC Link (private) to reach backend services (e.g. ALB/Lambda/EC2) in private subnets.
5. Backend services (in private subnet) access internet (for updates) via NAT Gateway.
6. Responses flow back through the same path.

3) Complete Architecture Diagram (3-AZ)



4) Subnet Details

Subnet Type	CIDR (Example)	Purpose	Route to
Public Subnet (AZ-1)	10.0.1.0/24	NAT Gateway, Bastion (optional)	0.0.0.0/0 -> IGW
Public Subnet (AZ-2)	10.0.2.0/24	NAT Gateway	0.0.0.0/0 -> IGW
Private Subnet (AZ-1)	10.0.3.0/24	Backend services	0.0.0.0/0 -> NAT GW
Private Subnet (AZ-2)	10.0.4.0/24	Backend services	0.0.0.0/0 -> NAT GW

5) Route Table Configuration

Public Subnet Route Table

Destination	Target	Purpose
10.0.0.0/16	local	VPC internal communication
0.0.0.0/0	Internet Gateway (IGW)	Internet access

Private Subnet Route Table

Destination	Target	Purpose
10.0.0.0/16	local	VPC internal communication
0.0.0.0/0	NAT Gateway	Outbound internet access

6) Security Group Configuration (Example)

Resource	Type	Inbound Rules	Outbound Rules
API Gateway (public)	Managed	HTTPS (443) from 0.0.0.0/0	N/A
ALB (internal)	SG	HTTPS (443) from API Gateway SG (or VPC Link)	All (or specific)
EC2/ECS (backend)	SG	App port (e.g. 8080) from ALB SG	All (or specific)
NAT Gateway	Managed	No inbound	All outbound
RDS (if used)	SG	DB port (e.g. 3306) from app SG	No outbound (or specific)

7) Configuration Steps (Summary)

1. Create a VPC (e.g. 10.0.0.0/16) with public and private subnets across 2+ AZs.
2. Create and attach an Internet Gateway (IGW) to the VPC.
3. Create NAT Gateways in public subnets (one per AZ for high availability) with Elastic IPs.
4. Create route tables:
   - Public RT -> 0.0.0.0/0 to IGW
   - Private RT -> 0.0.0.0/0 to NAT Gateway
5. Configure Security Groups for API, ALB, EC2/RDS.
6. Create an Application Load Balancer (internal) in private subnets.
7. Create a VPC Link in API Gateway and attach it to the internal ALB.
8. Create API in API Gateway and integrate with ALB using VPC Link.
9. Register domain in Route 53 and create an Alias record pointing to API Gateway.
10. Test the flow and monitor using CloudWatch.

8) Traffic Flow (Detailed)

1. User sends request to https://api.example.com
2. Route 53 resolves domain to API Gateway.
3. API Gateway receives the request.
4. API Gateway forwards request via VPC Link (private connection) to internal ALB in VPC.
5. ALB routes to targets (EC2/ECS/Lambda) in private subnets.
6. Backend services, if need internet (e.g. download updates), send traffic to NAT Gateway.
7. NAT Gateway performs SNAT using its Elastic IP and sends to Internet via IGW.
8. Response returns to the user through the same path.

9) Use Cases

- Expose private backend services securely via API Gateway.
- Microservices running in private subnets.
- Web applications with custom domain.
- Secure access to databases (RDS) from application tier.
- Hybrid architectures and on-premises connectivity (via VPN/Direct Connect).
- Applications that need outbound internet access but no inbound access.

10) Common Questions & Answers

Q1: Which AWS service is used to map a custom domain to API Gateway?
A: Route 53 (Alias record).

Q2: How can API Gateway access a private backend in VPC?
A: Using VPC Link (private integration with ALB/NLB).

Q3: Which component provides outbound internet access for instances in private subnet?
A: NAT Gateway.

Q4: Can NAT Gateway be used for inbound traffic?
A: No. It only allows outbound traffic.

Q5: Do we need a NAT Gateway in every AZ?
A: Best practice is one NAT Gateway per AZ for high availability.

11) Key Takeaways

- Route 53 -> DNS for custom domain.
- API Gateway -> managed API with private integration via VPC Link.
- VPC -> isolated network with public/private subnets.
- Public subnet -> route to IGW.
- Private subnet -> route to NAT Gateway for outbound internet.
- Security Groups -> stateful, instance level.
- NAT Gateway -> outbound internet access only.
- Use multiple AZs for high availability.
- Follow least privilege security.
- Use VPC Endpoints (e.g. S3, DynamoDB) to reduce NAT costs.

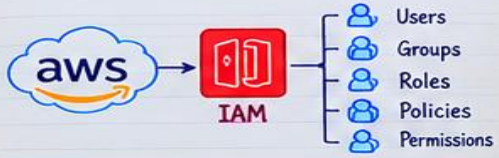
12) Exam Tips

- NAT Gateway is not a replacement for a firewall.
- Security Groups are stateful; NACLs are stateless.
- Private resources are not directly accessible from the internet.
- Use VPC Link for API Gateway private integration.
- Use Alias record in Route 53 for API Gateway.
- Each subnet must be within the VPC CIDR block.
- Plan for high availability (NAT GW per AZ).
- Know route table entries (0.0.0.0/0 -> IGW or NAT).
- Use HTTPS (443) for secure communication.

"Design secure, scalable and highly available applications using AWS networking building blocks."

AWS IDENTITY AND ACCESS MANAGEMENT (IAM)

— Overview —



"IAM lets you securely control access to AWS resources."

① What is IAM ?

AWS Identity and Access Management (IAM) is a service that helps you securely control access to AWS resources. You can create and manage users, groups, roles and policies to allow or deny permissions.

Key Points:

- Global service (not region specific).
- Used to manage identities and permissions.
- Supports least privilege access.
- Free service (no additional cost).
- Used for AWS Management Console, CLI and SDKs.

② IAM Components

Component	Description
User	An identity with long-term credentials (password, access keys). Used for people or applications.
Group	A collection of users. Permissions are attached to groups (not directly to users).
Role	An identity that can be assumed by trusted entities (users, AWS services, other AWS accounts). No long-term credentials.
Policy	A document (JSON) that defines permissions (Allow or Deny). Can be attached to users, groups, roles or resources.
Permissions	Allow or Deny access to specific AWS actions on resources.
Identity Provider (IdP)	External identity source (e.g., corporate directory via SAML or OIDC).

③ Types of Policies

- **Identity-based policies** – attached to users, groups or roles.
- **Resource-based policies** – attached to resources (e.g., S3 bucket).
- **IAM permissions boundary** – set maximum permissions for an identity.
- **AWS Organizations SCP** – set maximum permissions at the organization level.
- **Session policies** – apply permissions to a session (temporary).

④ How IAM Works ?



1. An authenticated user/service makes a request (e.g., s3:ListBucket).
2. IAM evaluates all applicable policies (identity-based, resource-based, permissions boundary, SCP, session).
3. If an explicit Deny exists → request is denied.
4. If allowed and no Deny → request is allowed.

Explicit Deny always overrides Allow.

⑤ IAM Policy Structure (JSON)

```
{
  "Version": "2012-10-17",                    ← Policy version
  "Statement": [
    {
      "Effect": "Allow",                      ← Allow or Deny
      "Action": "s3:ListBucket",              ← AWS action(s)
      "Resource": "arn:aws:s3:::my-bucket",   ← Resource ARN
      "Condition": {                          ← Optional conditions
        "StringEquals": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
}
```


⑥ IAM Users vs Groups vs Roles

Feature	User	Group	Role
Represents	Person or application	Collection of users	Temporary identity
Credentials	Long-term (password, access keys)	No credentials	Temporary credentials (using STS)
Permissions directly attached	Yes	Yes (then to users)	Yes
Use case	Human users, applications	Manage multiple users	Cross-account access, AWS services (EC2, Lambda), temporary access

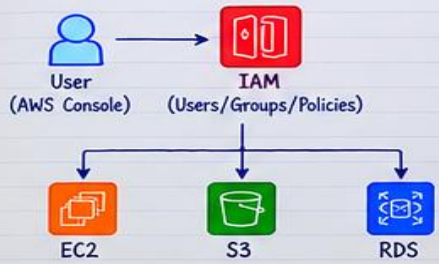
⑦ IAM Roles – Common Ues Cases

- EC2 instance role (access S3, DynamoDB, etc. without access keys).
- Lambda function role.
- Cross-account access (e.g., account A allows account B to assume a role).
- Federation with external IdP (SAML/OIDC).
- Temporary access for CLI/SDK.

Example: EC2 Role to access S3

1. Create an IAM role.
2. Attach AmazonS3ReadOnlyAccess policy.
3. Attach the role to EC2 instance.
4. EC2 can now access S3 without access keys.

⑧ Architecture Example



Example: Flow – S3 Access

1. User logs in with IAM user credentials.
2. User requests s3:ListBucket on my-bucket.
3. IAM checks attached policies.
4. If allowed → S3 returns the bucket list.
5. If denied → AccessDenied error.

Integration with Other AWS Services

- IAM Roles with EC2, Lambda, ECS, EKS, etc.
- Federation with AWS SSO / Identity Center.
- Use with CloudTrail for auditing.
- Use with KMS for key access control.
- Use with Services like API Gateway, CodePipeline, etc.
- Use condition keys for extra security (IP, MFA, time, etc.).

⑨ Best Practices

- ☑ Follow least privilege access.
- ☑ Use groups instead of attaching policies to users.
- ☑ Use roles for AWS services and cross-account access.
- ☑ Enable MFA for all human users.
- ☑ Rotate and avoid long-term access keys.
- ☑ Use permission boundaries and SCPs.
- ☑ Use IAM Access Analyzer to find over-permissive access.
- ☑ Monitor activity using CloudTrail.
- ☑ Use meaningful naming and tagging.
- ☑ Regularly review and remove unused users/roles.

⑩ Common IAM Condition Keys

Condition Key	Use Case	Example
aws:SourceIp	Restrict by IP address	"203.0.113.0/24"
aws:MultiFactorAuthPresent	Require MFA	true
aws:CurrentTime	Allow only in time range	"2024-01-01T00:00:00Z"
aws:PrincipalArn	Restrict to specific user/role	ARN of principal
aws:RequestedRegion	Allow specific region	"us-east-1"

⑪ IAM Security Tools

- IAM Access Analyzer – identify unintended access.
- IAM Credentials Report – list all users and key status.
- IAM Policy Simulator – test permissions.
- CloudTrail – track API calls.
- AWS Config – monitor IAM configuration changes.

⑫ Scenario Questions (with Answers)

Q1. A developer needs programmatic access to AWS using CLI. Which IAM credential should be used?
A: IAM User with Access Keys (and MFA).

Q2. An EC2 instance needs to access an S3 bucket. What should be used instead of access keys?
A: IAM Role attached to EC2 instance.

Q3. You want to allow a third-party account to access your S3 bucket. Which IAM component should you use?
A: Resource-based policy (bucket policy) or a role.

Q4. A user is denied access even though they have an allow policy. What could be the reason?
A: An explicit Deny (in any policy, permissions boundary, or SCP) overrides the allow.

Q5. You want users to access AWS only from your corporate IP range. Which IAM feature is used?
A: Condition key aws:SourceIp in the policy.

Q6. A company wants to manage access for employees using their corporate Active Directory. Which service is used with IAM?
A: AWS IAM Identity Center (AWS SSO) or SAML federation.

Q7. You need to restrict permissions for a role so that it cannot be given more than S3 read access. Which feature?
A: Permissions Boundary.

⑬ Exam Tips

- IAM is a global service.
- Users have long-term credentials; roles have temporary credentials.
- Policies are written in JSON with Allow or Deny.
- Explicit Deny always wins.
- Use groups to manage multiple users.
- Use roles for AWS services (EC2, Lambda, etc.).
- Resource-based policies are used for S3, SNS, SQS, etc.
- MFA is recommended for all human users.
- Use least privilege and review access regularly.
- SCPs are used in AWS Organizations (not for single accounts).
- IAM is free, but other services you access may cost.

Key Takeaways

- ✓ IAM controls who can do what on AWS resources.
- ✓ Use the right identity type: User / Group / Role.
- ✓ Follow least privilege and enable MFA.
- ✓ Use conditions, permissions boundaries and SCPs for better security.
- ✓ Integrate with other AWS services for secure access.
- ✓ Always monitor and audit using CloudTrail and Access Analyzer.

Remember:

"Give the minimum permissions,
to the right user, for the right time, for the right reason."

→ BOOK ←

AWS CERTIFICATE MANAGER (ACM)

— Overview —



"Provision, manage and deploy SSL/TLS certificates for your AWS resources easily."

① What is ACM ?

AWS Certificate Manager (ACM) is a managed service that provisions, manages and renews SSL/TLS certificates for use with AWS services.

② Why is ACM used ?

- Provides SSL/TLS certificates to enable HTTPS.
- Encrypts data in transit between clients and AWS resources.
- Helps establish identity (authentication) using certificates.
- Improves security, compliance and trust.

SSL/TLS Certificates:

Used to encrypt data in transit and verify the identity of a website or service.

Key Features of ACM

1. **Free public SSL/TLS certificates** for AWS services
   - Elastic Load Balancing (ALB/NLB)
   - Amazon CloudFront
   - Amazon API Gateway
2. **Automatic renewal** of certificates (managed by AWS).
3. **Private Certificate Authority (Private CA)** for internal applications, private networks, IoT devices and internal APIs.
4. **Managed issuance, renewal and revocation** of certificates.
5. **Import third-party certificates** (e.g., from external CAs).
   → Imported certificates require manual renewal.
6. Supports wildcard certificates (e.g., *.example.com).
7. Integrates seamlessly with AWS services.

Types of Certificates in ACM

Public Certificates	Private Certificates	Imported Certificates
• Issued by AWS public CA (trusted by browsers)	• Issued by ACM Private CA	• Import certificates from third-party CAs
• Free of cost	• Used for internal resources, private networks, IoT devices	• Used when you already have a cert
• Used with ALB, CloudFront, API Gateway, etc.	• Not publicly trusted	• Manual renewal required

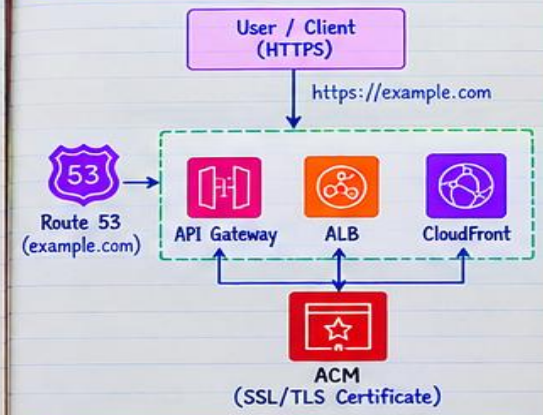
Certificate Lifecycle in ACM

1. **Request / Provision** — Request a certificate (public/private) or import a certificate.
2. **Domain Validation** — Verify domain ownership (DNS validation – recommended or Email validation).
3. **Issue** — After successful validation, ACM issues the certificate.
4. **Attach / Deploy** — Attach the certificate to AWS services (ALB, CloudFront, API Gateway, etc.).
5. **Renew / Redeploy** — Public certificates are automatically renewed by AWS. (Imported certificates require manual renewal).
6. **Revoke / Replace** — Revoke the certificate if needed or replace with a new one.

Note:

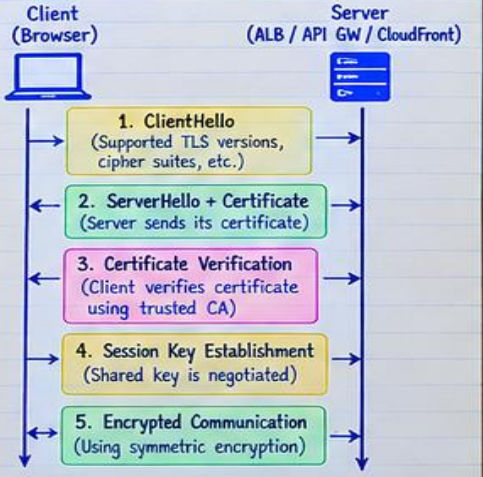
- ACM handles certificate management (lifecycle).
- You don't manage the private key for ACM-issued certs.
- Public certificates are renewed automatically as long as validation remains in place.

Architecture & How it Works ?



- ACM provides the certificate to the AWS service (ALB / API Gateway / CloudFront).
- The service uses the certificate to terminate HTTPS and encrypt traffic between client and service.
- Route 53 maps the domain (e.g., example.com) to the service endpoint.
- ACM fits in by providing and managing the SSL/TLS certificate used by these services.

TLS Handshake (Simplified)



Key Point:

- ACM provides and manages the TLS certificate.
- TLS handles the actual encryption and authentication during the handshake.
- Data in transit is encrypted after successful handshake.

Configuration Steps (AWS Console)

1. Open AWS Certificate Manager (ACM) console.
2. Choose Request a certificate.
3. Select certificate type:
   - Public certificate (for internet-facing services)
   - Private certificate (using ACM Private CA)
   - Import certificate (existing certificate)
4. Add domain names (e.g., example.com, www.example.com)
   (You can also use wildcard like *.example.com).
5. Select validation method:
   - DNS validation (recommended)
   - Email validation
6. Wait for certificate to be issued (status: Issued).
7. Attach the certificate to your AWS service:
   - API Gateway (custom domain)
   - Application Load Balancer (HTTPS listener)
   - CloudFront (distribution – add custom SSL certificate)

Important:

- For CloudFront, the ACM certificate must be in us-east-1 (N. Virginia) region.
- Imported certificates require manual renewal.
- Keep the validation records (DNS) intact for automatic renewal.

Scenario Examples (with Answers)

Q1. You want to protect an API Gateway custom domain with HTTPS. Which AWS service should you use?

A: AWS Certificate Manager (ACM)

(Request a public certificate and attach to API Gateway custom domain).

Q2. You have a web application behind an Application Load Balancer (ALB) and want to enable HTTPS. Which service provides the SSL/TLS certificate?

A: AWS Certificate Manager (ACM)

(Attach the ACM certificate to the ALB HTTPS listener).

Q3. You want to use HTTPS for a CloudFront distribution with a custom domain. What should you do?

A: Use an ACM certificate in the us-east-1 region and attach it to the CloudFront distribution.

Q4. You have an internal application in a private network and need a TLS certificate. Which option should you use?

A: ACM Private Certificate Authority (Private CA)

Q5. You already have a certificate from a third-party CA. You want to use it with ALB. What should you do?

A: Import the certificate into ACM (but manual renewal is required).

Comparison & Exam Focus

Feature	Public Cert	Private Cert	Imported Cert
Issued by	AWS public CA	ACM Private CA	External CA
Trusted by browsers	Yes	No (internal use)	Depends
Cost	Free	(Private CA + cert)	As per external CA
Auto renewal	Yes	Yes	No (manual)
Use cases	ALB, CloudFront, API Gateway.	Internal apps, private networks, IoT	Existing certs

Common Mistakes / Exam Traps

1. Using ACM certificate for CloudFront in the wrong region (must be in us-east-1).
2. Thinking imported certificates are auto-renewed (they are not).
3. Deleting DNS validation records (can break automatic renewal).
4. Assuming private certificates are publicly trusted (they are not).
5. Not attaching the certificate to the correct listener/service.

Key Takeaways

- ✅ ACM provides and manages SSL/TLS certificates.
- ✅ Free public certificates for ALB, CloudFront, API Gateway.
- ✅ Private CA for internal use cases.
- ✅ Automatic renewal (public and private), manual for imported.
- ✅ ACM does not handle traffic encryption – TLS does.
- ✅ For CloudFront, certificate must be in us-east-1.
- ✅ Use DNS validation for reliable automatic renewal.
- ✅ Keep certificates attached to the correct AWS service.

AWS KEY MANAGEMENT SERVICE (KMS)

— Overview —



"AWS KMS makes it easy for you to create and control the encryption keys used to encrypt your data."

① What is AWS KMS ?

AWS Key Management Service (KMS) is a managed service that lets you create, manage, and use cryptographic keys to encrypt and decrypt data across AWS services and in your applications.

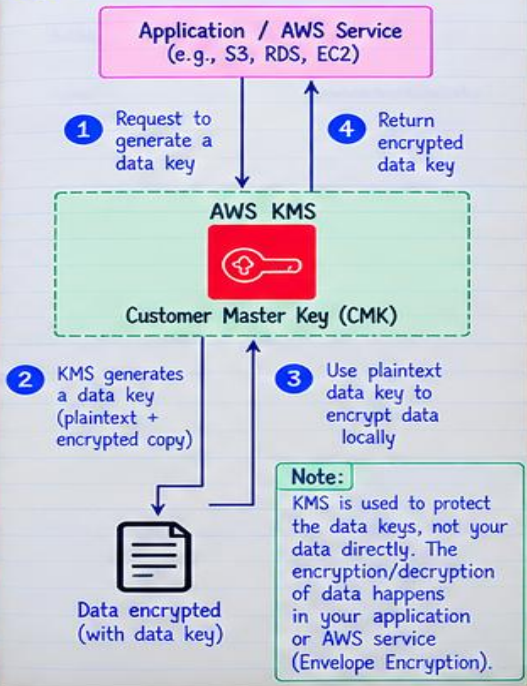
Key Points:

- Fully managed and highly available.
- Integrates with many AWS services.
- Helps you meet compliance and security requirements.
- You don't manage the underlying hardware.
- Pay per use (no upfront cost).

② Key Concepts and Terminology

Term	Description
Customer Master Key (CMK)	A logical key in KMS. Used for encryption, decryption and generating data keys.
AWS Managed Key	Created and managed by AWS (e.g., aws/s3, aws/ebs). Free to use.
Customer Managed Key (CMK)	Created and managed by you. You control policies, rotation, enable/disable, deletion.
Data Key	A temporary, unique key generated by KMS for encrypting data. (Envelope encryption)
Key Policy	Resource-based policy that defines who can use and manage the key.
Grants	Allow temporary access to a key (used by AWS services).
Alias	A friendly name for a key (e.g., alias/my-key).
Key Rotation	Automatically creates new key material (default 1 year for CMK).

③ How AWS KMS Works ?



Note:
KMS is used to protect the data keys, not your data directly. The encryption/decryption of data happens in your application or AWS service (Envelope Encryption).

④ Types of Keys in KMS

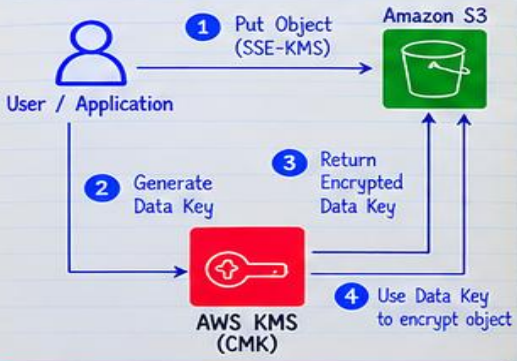
Key Type	Created By	Use Case	Cost
AWS Managed Key	AWS	Used by AWS services (e.g., aws/s3, aws/ebs)	Free
Customer Managed Key (CMK)	You	Full control, custom policies, rotation, etc.	Pay per use
AWS Owned Key	AWS	Used internally by AWS (not visible to you)	Free

⑤ Integration with AWS Services

- S3 – Server-side encryption (SSE-KMS)
- EBS – Encrypt volumes
- RDS – Encrypt database instances
- Lambda – Encrypt environment variables
- CloudWatch Logs – Encrypt log data
- Secrets Manager – Encrypt secrets
- API Gateway – Encrypt data at rest
- ECS/EKS – Encrypt task definitions/secrets
- Many more (DynamoDB, SQS, SNS, etc.)

KMS integrates seamlessly with AWS services using key policies and grants.

⑥ Architecture Example (S3 Encryption)



Flow:

1. Application uploads object to S3 with SSE-KMS.
2. S3 asks KMS to generate a data key using the CMK.
3. KMS returns an encrypted data key and plaintext key (to S3).
4. S3 uses the plaintext data key to encrypt the object and stores the encrypted data key with the object.

⑦ Key Policies vs IAM Policies

Key Policy (Resource-based)	IAM Policy (Identity-based)
Attached to the key (CMK)	Attached to IAM user, group, or role
Defines who can manage and use the key	Defines what actions an identity can perform
Required for cross-account access	Used in combination with key policy
Evaluated by KMS	Evaluated by AWS IAM
Can allow access to other AWS accounts	Typically used within the same account

Both key policy and IAM policy must allow access for a principal to use a KMS key.

⑧ Security Best Practices

- ✅ Follow least privilege access.
- ✅ Use customer managed keys (CMKs) for sensitive data.
- ✅ Enable automatic key rotation.
- ✅ Use key aliases for easy management.
- ✅ Use grants for AWS services where needed.
- ✅ Monitor KMS usage with AWS CloudTrail.
- ✅ Restrict key access using key policies and IAM policies.
- ✅ Enable multi-factor authentication (MFA) for key administration.
- ✅ Regularly review and audit access.

⑨ Scenario Questions (with Answers)

Q1. A company wants to encrypt data in an S3 bucket and manage the encryption key. Which service should be used?
A: AWS KMS (Customer Managed Key)

Q2. Which type of key is free and created by AWS for S3 encryption?
A: AWS managed key (aws/s3)

Q3. An application needs to encrypt data before storing it in a database. Which key should be used to generate a data key?
A: Customer Master Key (CMK) in AWS KMS

Q4. How can you allow another AWS account to use your KMS key?
A: Update the key policy to allow the other account (or use a grant).

Q5. Does KMS automatically rotate keys?
A: Yes, for customer managed keys (every 1 year by default).

Q6. You need to encrypt environment variables in AWS Lambda. Which service is used?
A: AWS KMS

⑩ Limitations and Considerations

- KMS has a request rate limit (can be increased).
- There may be a small latency for cryptographic operations.
- KMS is a regional service (keys are region-specific).
- You cannot export the key material from AWS KMS (for CMKs and AWS managed keys).
- Use multi-Region keys if you need the same key across multiple regions.
- Imported keys require manual rotation.
- KMS is not used for large data encryption directly (use data keys).

⑪ Common Exam Tips

- ACM is for SSL/TLS certificates, not encryption keys.
- KMS encrypts data keys, not your data directly.
- AWS managed keys are free.
- Customer managed keys give full control.
- Key policy and IAM policy both must allow access.
- Use CloudTrail to monitor KMS API calls.
- Multi-Region keys are used for cross-region scenarios.
- KMS does not store or manage your data.

Key Takeaways:

- ✅ KMS helps you create, manage and use encryption keys.
- ✅ Uses envelope encryption (data keys).
- ✅ Integrates with many AWS services.
- ✅ Follow least privilege and monitor usage.
- ✅ Understand key policies, IAM policies, and key types.
- ✅ Important for data security and compliance.

AWS IDENTITY AND ACCESS MANAGEMENT (IAM)

① Overview

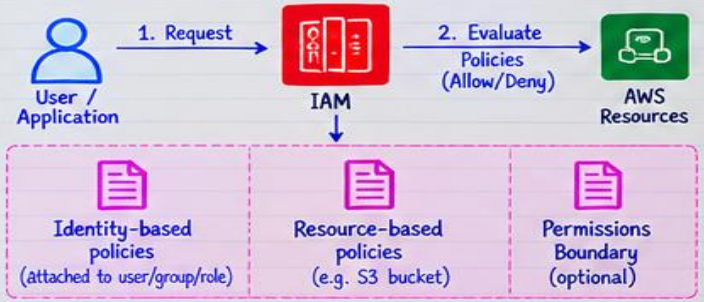
- IAM is a global service that helps you securely control access to AWS resources.
- You can create and manage users, groups, roles, policies and permissions.
- No additional cost.
- Principle of Least Privilege → give only the minimum permissions required.

AWS IAM (Global Service)

② IAM Components

Component	Description
User	Individual person or application with long-term credentials (password/access keys).
Group	Collection of users.
Role	Temporary identity that can be assumed by users or AWS services (no long-term keys).
Policy	JSON document that defines permissions (Allow or Deny).
Permissions Boundary	Maximum permissions an identity can have.
Session	Temporary credentials (via STS).

③ How IAM Works ?



④ IAM Architecture Example



⑤ Best Practices

- ✅ Follow least privilege access.
- ✅ Use groups and roles (not individual permissions).
- ✅ Use IAM roles for AWS services (e.g., EC2, Lambda).
- ✅ Enable MFA for human users.
- ✅ Use temporary credentials (STS).
- ✅ Regularly audit with IAM Access Analyzer and CloudTrail.
- ✅ Use permission boundaries and SCPs in organizations.

⑥ Common Use Cases

- Control access to S3, EC2, RDS, etc.
- Federate with external IdP (e.g., corporate AD via SAML).
- Provide temporary access (assume role).
- Manage access for CI/CD pipelines, applications, and services.
- Implement cross-account access.

⑦ Key Takeaway

IAM answers → Who can access what in AWS? It authenticates and authorizes.

AWS CERTIFICATE MANAGER (ACM)

① Overview

- ACM is a managed service that provisions, manages and deploys SSL/TLS certificates for AWS services.
- Automatically renews certificates (no manual renewal).
- Free for AWS services (e.g., ALB, CloudFront, API Gateway).
- Supports public certificates, private certificates (and import of third-party certificates).

AWS ACM (Regional Service)

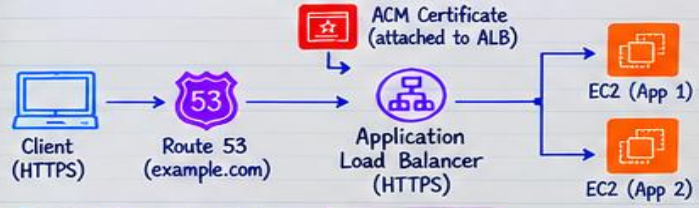
② Types of Certificates

Type	Description	Use Cases	Cost
Public Certificate	Issued by AWS public CA.	ALB, CloudFront, API Gateway.	Free
Private Certificate (ACM Private CA)	Issued by your private CA.	Internal apps, private domains.	Pay per certificate and CA.
Imported Certificate	Import existing third-party certificate.	Use with AWS services that support import.	You pay CA, ACM usage is free.

③ How ACM Works ? (Certificate Lifecycle)



④ Architecture Example (HTTPS with ALB)



⑤ Configuration Steps (Example: ALB)

1. Open ACM console and request a public certificate.
2. Add domain (e.g., example.com) and choose validation method (DNS recommended).
3. Complete validation (ACM issues certificate).
4. Create an Application Load Balancer and add HTTPS listener (port 443).
5. Attach the ACM certificate to the listener.
6. Update Route 53 to point your domain to the ALB.

⑥ Common Use Cases

- Enable HTTPS for ALB, CloudFront, API Gateway.
- Secure custom domain for API Gateway.
- Internal TLS using ACM Private CA.
- Import and use third-party certificates.

Key Takeaway

ACM simplifies SSL/TLS certificate management with automatic renewal and seamless AWS integration.

AWS KEY MANAGEMENT SERVICE (KMS)

① Overview

- KMS is a managed service to create, manage and use cryptographic keys for encryption and decryption.
- You control the keys, AWS manages the infrastructure.
- Integrates with many AWS services.
- Highly available and scalable.
- You are charged for key usage (except AWS owned keys).
- Helps you meet compliance and security requirements.

AWS KMS (Regional Service)

② Key Concepts

Term	Description
Customer Master Key (CMK)	A logical key in KMS. Used to encrypt/decrypt data keys. Can be AWS managed or customer managed.
AWS Managed Key	Created and managed by AWS (e.g., aws/s3). Free to use. Limited control.
Customer Managed Key (CMK)	Created and managed by you. Full control over policies, rotation, deletion.
Data Key	A temporary key generated by KMS to encrypt data (envelope encryption).
Key Policy	Resource-based policy that controls who can use the key.
Grants	Allows temporary access to a key (used by AWS services).
Key Rotation	Automatically rotate key material (default 1 year for CMK).

③ How KMS Works ? (Envelope Encryption)



④ Types of Keys

Type	Created By	Use Case	Cost
AWS Managed Key	AWS	Used by AWS services (e.g., aws/s3).	Free
Customer Managed Key (CMK)	You	Full control, custom policies, rotation, cross-account.	Pay per use
AWS Owned Key	AWS	Used internally by AWS (not visible to you).	Free

⑤ Integration with AWS Services

- S3 (SSE-KMS) – encrypt objects
- EBS – encrypt volumes
- RDS – encrypt databases
- Lambda – encrypt environment variables
- CloudWatch Logs
- Secrets Manager
- ECS/EKS (secrets)
- Many more...

⑥ Best Practices

- ✅ Use customer managed keys (CMK) for sensitive data.
- ✅ Enable key rotation.
- ✅ Use key policies and IAM policies together (least privilege).
- ✅ Monitor usage with CloudTrail and AWS KMS logs.
- ✅ Use grants for temporary access (e.g., AWS services).
- ✅ Follow compliance requirements (e.g., HIPAA, PCI-DSS).

Key Takeaway

KMS provides secure, scalable key management. It protects your data using encryption keys and integrates with almost all AWS services.

1. Networking & Content Delivery

1 Amazon Route 53 (DNS)

- Highly available and scalable DNS service.
- Routes users to resources (ALB, API Gateway, EC2, S3, etc.).
- Supports routing policies:
  - Simple, Weighted, Latency-based
  - Failover, Geolocation, Multi-value
  - Health checks
- Domain registration and DNS management.



Example Use Cases:

- Route users to nearest region (latency).
- Failover to backup site.
- Route API traffic to API Gateway.

2 Amazon API Gateway

- Fully managed service to create, publish, maintain and secure APIs.
- Supports REST, HTTP and WebSocket APIs.
- Integrates with AWS services (Lambda, EC2, HTTP endpoints) using VPC Link.
- Features: throttling, authentication (IAM/Cognito), request/response transformation, logging (CloudWatch).

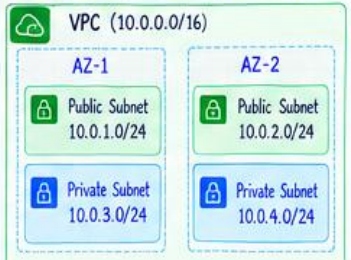


Example Use Cases:

- Expose backend services securely.
- Integrate with private resources using VPC Link.
- Handle authentication, rate limiting, montioring.

3 Amazon VPC (Virtual Private Cloud)

- Logically isolated network in AWS.
- You define IP range (CIDR), subnets, route tables, security groups, etc.
- Provides isolation, security and control.
- Can span multiple Availability Zones (AZs).



Example: 10.0.0.0/16 = 65,536 IPs (you can create subnets from this range).

4 VPC Subnets

- Subdivide VPC into smaller CIDR ranges.
- Types:
  - **Public Subnet** – has route to Internet Gateway (IGW).
  - **Private Subnet** – no direct internet access (use NAT Gateway).
- Subnets are created in specific Availability Zones.
- Each subnet has a route table.



5 Route Tables

- Set of rules (routes) that determine where traffic is sent.
- Each subnet is associated with a route table.
- Routes can point to: Local, Internet Gateway (IGW), NAT Gateway, VPC Peering, Transit Gateway, etc.

Example Route Table (Public Subnet)

Destination	Target	Purpose
10.0.0.0/16	local	VPC internal
0.0.0.0/0	igw-xxxx	Internet access

Example Route Table (Private Subnet)

Destination	Target	Purpose
10.0.0.0/16	local	VPC internal
0.0.0.0/0	nat-xxxx	Outbound internet via NAT

6 Security Groups (SG)

- Stateful firewall at the instance/ENI level.
- Controls inbound and outbound traffic.
- Allows rules based on protocol, port, source (IP/SG).
- Default: all outbound allowed, no inbound.
- Multiple SGs can be attached to a resource.

Example Inbound Rules (Web Server)

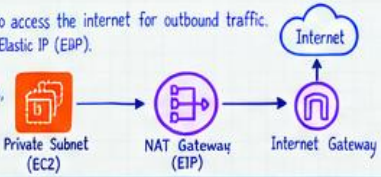
Type	Protocol/Port	Suroce
HTTP	TCP 80	0.0.0.0/0
HTTPS	TCP 443	0.0.0.0/0
SSH	TCP 22	Your IP

Example Outbound Rules

Type	Protocol/Port	Source
All traffic	All	0.0.0.0/0

7 NAT Gateway

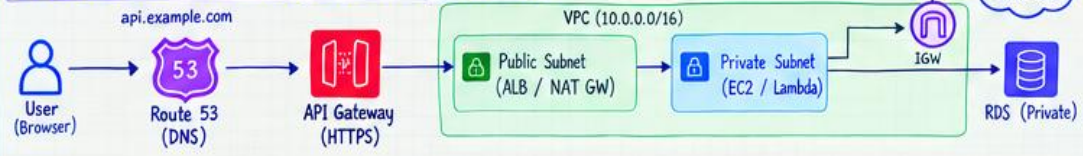
- Allows instances in private subnets to access the internet for outbound traffic.
- Created in a public subnet with an Elastic IP (EIP).
- Managed service (highly available).
- Used for software updates, downloads, external APIs.
- No inbound connections from internet.



NAT Gateway vs NAT Instance

Feature	NAT Gateway	NAT Instance
Managed	Yes	No
High availability	Yes	You manage
Performance	High	Depends on instance
Cost	Higher	Lower
Use case	Recommended	Legacy / custom

End-to-End Architecture Example (All Networking Components)



2. Security, Identity & Compliance

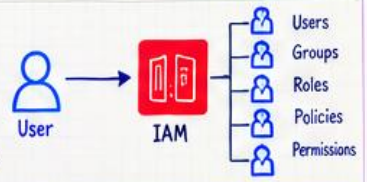
8 AWS Identity and Access Management (IAM)

- Global service to securely control access to AWS resources.
- Manage users, groups, roles, policies and permissions.
- Follow principle of least privilege.
- Supports MFA, access keys, temporary credentials (STS).
- Integrates with other AWS services.

Components	
User	Person or application with credentials
Group	Collection of users
Role	Temporary identity (assumed by users/services)
Policy	JSON document (Allow/Deny)
Permissions boundary	Maximum permissions for an identity
Federation	External identities (SAML/OIDC)

How IAM Works ?

1. User/Service makes a request
2. IAM evaluates policies
3. Allows or denies the request
4. If allowed, temporary credentials may be issued (STS)



Example Policy (Allow S3 Read)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

Best Practices

- Use least privilege access.
- Enable MFA.
- Use roles for services.
- Avoid long-term access keys.
- Regularly audit with IAM Access Analyzer.
- Use IAM groups and policies.
- Use permissions boundaries and SCPs.

9 AWS Certificates Manager (ACM)

- Managed service to provision, manage, and deploy SSL/TLS certificates.
- Automatic renewal (for ACM managed certificates).
- Free for AWS services (ALB, CloudFront, API Gateway).
- Supports public, private and imported certificates.

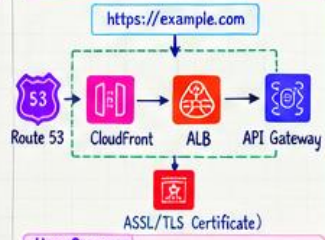
Types of Certificates

Public Certificate	Issued by AWS CA, free, used with ALB/CloudFront/API Gateway
Private Certificate	Issued by ACM Private CA, for internal use
Imported Certificate	Import existing third-party certificate

How ACM Works ? (Lifecycle)

1. Request or import certificate
2. Domain validation (DNS or Email)
3. ACM issues certificate
4. Attach to AWS service (ALB, CloudFront, API Gateway)
5. Automatic renewal (for ACM managed)

Architecture Example



Use Cases

- Enable HTTPS for applications
- Secure custom domains
- Internal TLS with Private CA

10 AWS Key Management Service (KMS)

- Managed service to create, manage and use cryptographic keys.
- Used for encryption and decryption.
- Highly available and scalable.
- Integrates with many AWS services.
- You never handle or store the actual keys (for AWS managed keys).

Key Types	
AWS Managed Key	Created by AWS, used by AWS services, free
Customer Managed Key (CMK)	Created and managed by you, full control, pay per use
AWS Owned Key	Created by AWS for internal use, not visible to you
Data Key	Temporary key generated by KMS for data encryption (envelope encryption)

How KMS Works ? (Envelope Encryption)



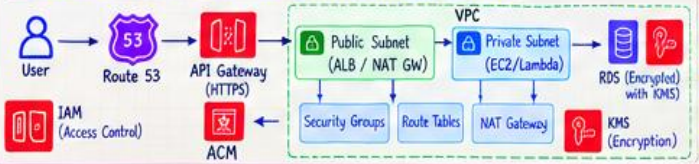
Use Cases

- Encrypt data at rest (S3, EBS, RDS)
- Encrypt data in transit (TLS)
- Encrypt environment variables (Lambda)
- Encrypt secrets (Secrets Manager)
- Cross-account and cross-region key usage

Best Practices

- Use customer managed keys for sensitive data.
- Enable key rotation.
- Use IAM policies and key policies.
- Monitor with CloudTrail and CloudWatch.
- Follow compliance (HIPAA, PCI-DSS, etc.)

Combined View – Secure and Scalable AWS Architecture



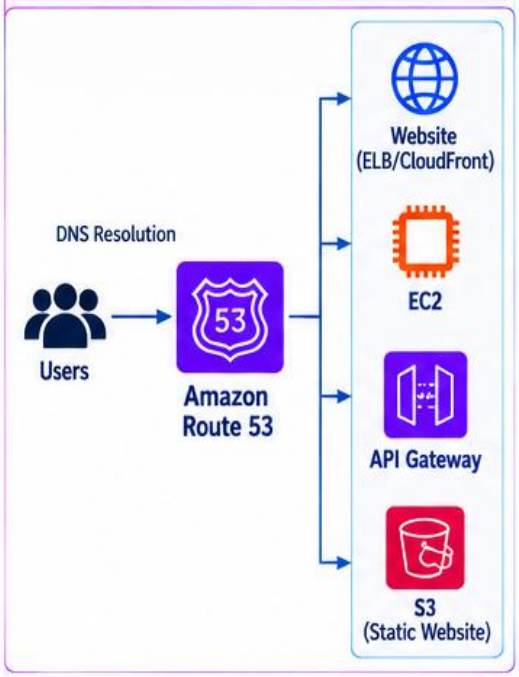
Key Takeaways

- Use Route 53 for DNS and global routing.
- Secure APIs with API Gateway + IAM + ACM.
- Design VPC with public/private subnets and proper route tables.
- Use Security Groups to control traffic.
- Use NAT Gateway for private subnet internet access.
- Use IAM for access control, ACM for certificates, and KMS for encryption.
- Follow practices for security and compliance.

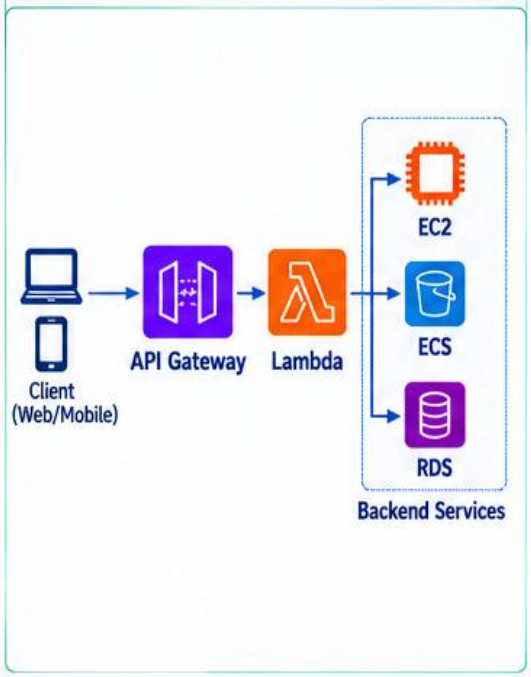
Networking & Content Delivery

Build • Connect • Secure • Deliver

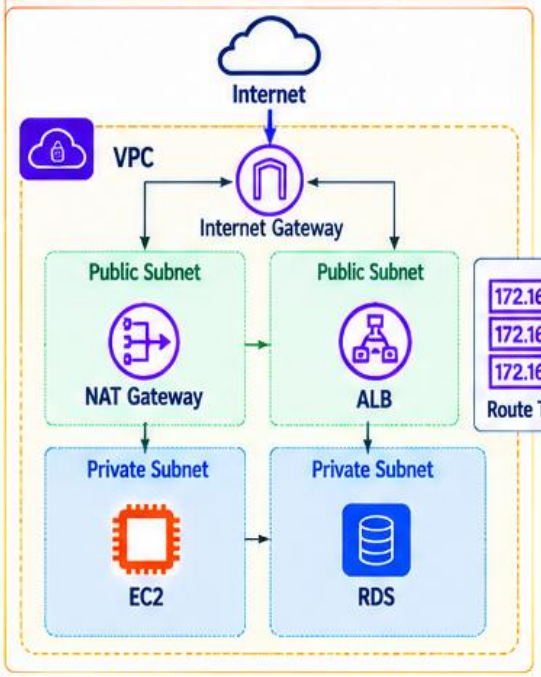
Route 53 – Architecture



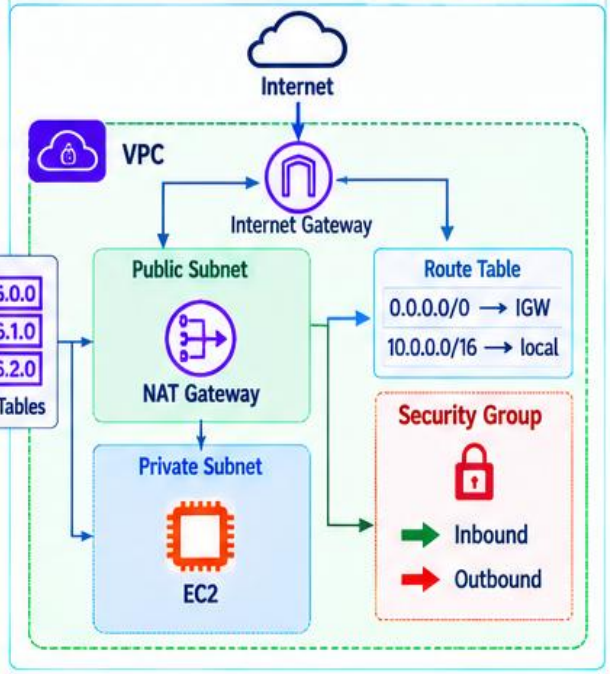
Amazon API Gateway – Architecture



Amazon VPC – Architecture

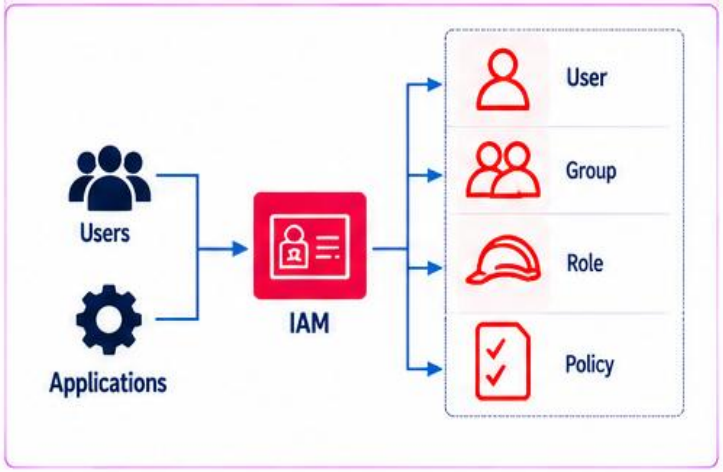


Subnets, Route Tables, Security Groups & NAT Gateway

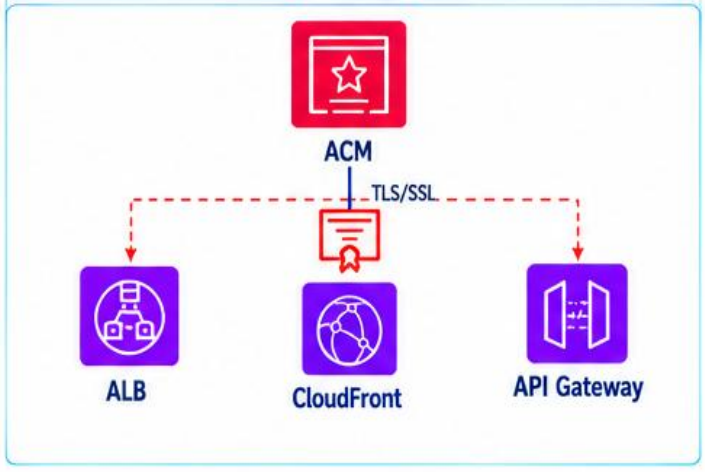


Security, Identification and Compliance

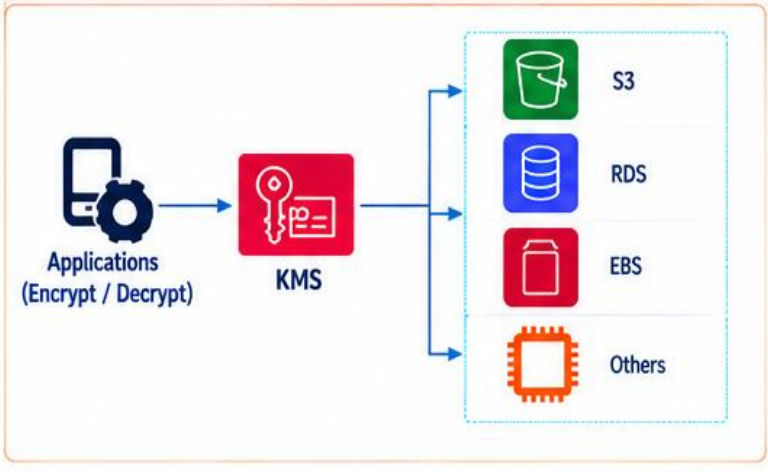
AWS Identity and Access Mamagement (IAM) – Architecture



AWS Certificate Manager (ACM) – Architecture



AWS Key Management Service (KMS) – Architecture



Secure Foundations for a Scalable Cloud

Amazon Relational Database Service (RDS)

Managed Relational Database Service for Easy, Scalable and Reliable Databases

1 What is Amazon RDS ?

- Amazon RDS (Relational Database Service) is a managed service to set up, operate, and scale relational databases in the AWS Cloud.
- It automates time-consuming administrative tasks such as provisioning, patching, backups, monitoring, and high availability.
- Supports popular database engines.

"Focus on your application, not on database management."

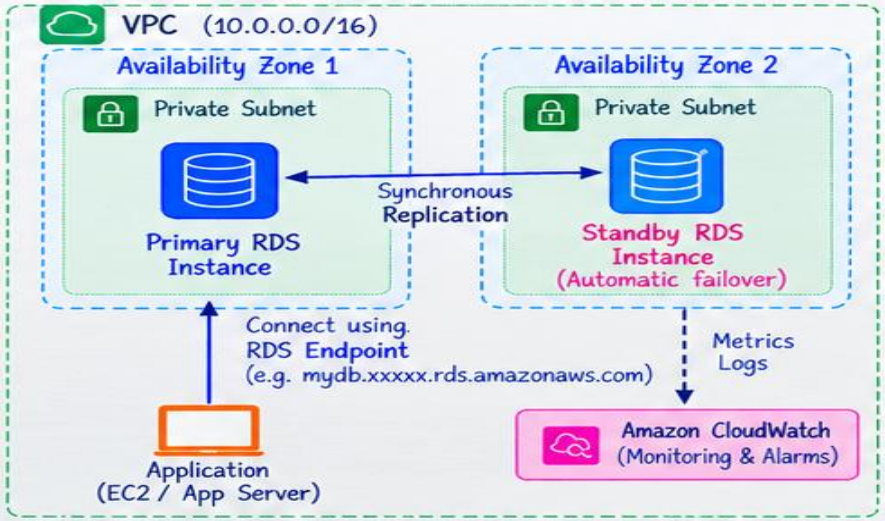
2 Supported Database Engines

Engine	Use Case / Notes
Amazon Aurora	AWS-optimized, high performance, MySQL & PostgreSQL compatible
MySQL	Popular open-source database
PostgreSQL	Advanced open-source database
MariaDB	Community-driven fork of MySQL
Oracle	Enterprise workloads
Microsoft SQL Server	Windows-based enterprise apps
IBM Db2	Enterprise workloads

3 Key Features

- ✓ Managed service (automatic provisioning, patching, backups)
- ✓ High availability with Multi-AZ deployments
- ✓ Read replicas for read scaling.
- ✓ Automatic backups and point-in-time recovery
- ✓ Snapshots (manual and automated)
- ✓ Easy scaling (compute and storage)
- ✓ Encryption at rest (using AWS KMS) and in transit (SSL/TLS)
- ✓ Monitoring with Amazon CloudWatch
- ✓ Integration with IAM (authentication & authorization)
- ✓ Supports VPC for network isolation
- ✓ Multi-AZ for disaster recovery.
- ✓ Cost-efficient (pay for what you use)

4 RDS Architecture (Multi-AZ - Highly Available)



5 Important Components

Component	Description
DB Instance	A single database environment (runs the DB engine).
DB Cluster (Aurora)	Collection of DB instances (1 writer + multiple readers).
Multi-AZ Deployment	Standby instance in another AZ for high availability (failover).
Read Replica	Read-only copy for scaling. (used for read-heavy workloads).
DB Subnet Group	Specifies subnets (at least 2 AZs).
Security Group	Controls inbound/outbound traffic to the DB instance.
Parameter Group	Configures engine-specific settings.
Option Group	Additional features (e.g., Oracle options, SQL Server features).
Backup	Automated backups (retention period) and manual snapshots.
RDS Endpoint	DNS name to connect to the DB.

6 Common Use Cases

- Web and mobile applications
- Enterprise applications (ERP, CRM)
- E-commerce platforms
- Content management systems
- Analytics and reporting databases
- Lift and shift of on-prem databases to cloud
- Disaster recovery and business continuity

7 Best Practices

- ✓ Use Multi-AZ for production databases.
- ✓ Use Read Replicas for read scaling.
- ✓ Enable automatic backups and set appropriate retention period.
- ✓ Use encryption (KMS) for data at rest.
- ✓ Place RDS in private subnets (not publicly accessible).
- ✓ Use IAM authentication (where supported).
- ✓ Monitor with CloudWatch and set alarms.
- ✓ Use Parameter Groups and Option Groups as needed.
- ✓ Use snapshots before major changes.
- ✓ Follow least privilege access using Security Groups.

8 Backup, Snapshot and Recovery

Feature	Description
Automated Backup	Daily full backup + transaction logs (point-in-time recovery).
Manual Snapshot	User-initiated backup (retained until deleted).
Point-in-Time Recovery	Restore to any time within the retention period.
Backup Retention Period	1 to 35 days (configurable).
Cross-Region Snapshot	Copy snapshot to another region for disaster recovery.

9 Scaling Options

- Vertical Scaling.
  - Change instance type (compute/memory).
- Storage Scaling.
  - Automatically increases storage when needed. (if enabled).
- Read Replicas
  - Create read replicas to handle read traffic.
  - Can be in same region or cross-region.
- Aurora Auto Scaling.
  - Automatically scales read replicas based on demand.

10 Security

- Control access using Security Groups (allow only required IP/instances).
- Use encryption at rest with AWS KMS.
- Use SSL/TLS for data in transit.
- Place RDS in private subnets (no public access).
- Use IAM database authentication (available for MySQL and PostgreSQL).
- Regularly rotate credentials.
- Enable logging and monitor using CloudWatch.
- Use least privilege principle.

11 Example: Create an RDS Instance

1. Open RDS console.
2. Choose database engine (e.g., MySQL).
3. Select instance type.
4. Configure storage, Multi-AZ (if needed).
5. Choose VPC, subnet group, and security group.
6. Configure backup and maintenance window.
7. Set DB name, username, password.
8. Review and create.
9. Connect using the RDS endpoint.

12 Scenario Example (Answering Your Questions)

- **Requirement**: Design a highly available, secure database for a web application with automatic backups.
- **Solution**: Use Amazon RDS with Multi-AZ in private subnets, configure security groups, enable automatic backups, and use an RDS endpoint. Optionally create read replicas for read scaling.
- **Benefits**: High availability, reduced downtime, automated backups, secure access, and easy scaling.

13 Key Takeaways

- ✓ RDS is a managed relational database service.
- ✓ Use Multi-AZ for high availability and automatic failover.
- ✓ Read replicas help scale read workloads.
- ✓ Automated backups, snapshots and point-in-time recovery protect your data.
- ✓ Integrate with VPC, Security Groups, IAM and KMS for security.
- ✓ RDS reduces operational overhead and improves reliability.

Create an Amazon RDS (MySQL) – Step by Step Guide

A managed relational database in the AWS Cloud

1 What is Amazon RDS ?

- Amazon RDS (Relational Database Service) is a managed service to set up, operate and scale relational databases in AWS.
- Supports multiple database engines (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Amazon Aurora, etc.).
- Automates time-consuming tasks like provisioning, patching, backups, monitoring and high availability.

"You focus on your application, not on database management."

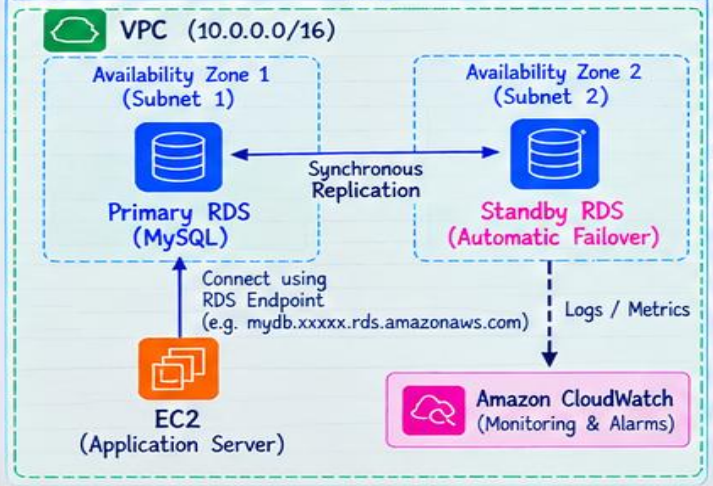
2 Prerequisites

- ✅ An AWS account.
- ✅ A VPC with at least two subnets (preferably in different AZs).
- ✅ A DB subnet group (or create during setup).
- ✅ A Security Group allowing access (e.g., from EC2).
- ✅ An EC2 instance or your local machine to connect.
- ✅ Required IAM permissions (`AmazonRDSFullAccess` or granular permissions).

3 Key Components Used

Component	Purpose
VPC	Provides isolated network for RDS.
Subnets	Place RDS in private subnets (across AZs).
DB Subnet Group	Collection of subnets for RDS.
Security Group	Controls inbound/outbound traffic to RDS.
RDS Instance	Managed database (e.g., MySQL).
Parameter Group	Configures database engine settings.
Option Group	Additional features (e.g., native backups).
RDS Endpoint	DNS name to connect to the database.

4 RDS Architecture (Typical – Multi-AZ)



5 Step-by-Step: Create Amazon RDS (MySQL)

1 Open RDS Console

- Go to AWS Management Console → RDS → Create database.

2 Choose a Database Creation Method

- Choose **Standard create** (for full configuration).
- (You can also use Easy create for quick setup).

Amazon RDS

3 Select Engine and Version

- Engine: MySQL
- Choose the desired version (e.g., MySQL 8.0).

4 Configure Templates

- Choose Free tier (for learning) or Production.

5 Set DB Instance Details

- DB instance identifier: `mydb`
- Master username: `admin`
- Master password: (choose a strong password)
- Instance class: `db.t3.micro` (free tier) or as per need
- Storage: e.g., 20 GB (General Purpose SSD – gp3)
- Enable Multi-AZ deployment (for high availability).

6 Configure Connectivity

- VPC: Select your VPC
- Subnet group: Choose existing or create new (with at least 2 subnets)
- Public access: No (keep in private subnet)
- Security group: Choose existing or create new
  - Allow inbound MySQL (port 3306) from your EC2 (or your IP for testing).

7 Additional Configuration

- Database name: `mydatabase` (optional)
- Parameter group: `default.mysql8.0` (or custom)
- Backup retention: e.g., 7 days
- Enable automated backups
- Enable deletion protection (recommended)
- Monitoring: Enable Enhanced monitoring (optional)
- Choose maintenance window
- Tags: Add tags (e.g., Project = Demo).

8 Review and Create

- Review all settings and click **Create database**.
- It will take a few minutes to provision the RDS instance.

6 Example Security Group Rule (for RDS)

Type	Protocol	Port	Source	Description
MYSQL/Aurora	TCP	3306	sg-xxxxxxxx (EC2 SG)	Allow MySQL access from EC2
MYSQL/Aurora	TCP	3306	your-ip/32 (for testing)	Allow MySQL access from local machine (temporary)

Note: For production, allow access only from trusted sources (e.g., EC2 security group or application servers), not from 0.0.0.0/0.

7 Connect to RDS (from EC2 or Local)

From an EC2 instance (Linux)

1. SSH into your EC2 instance.
2. Install MySQL client (if not already installed).

```
sudo yum install mysql     # For Amazon Linux
sudo apt install mysql-client   # For Ubuntu
```

3. Connect using the RDS endpoint:

```
mysql -h mydb.xxxxx.rds.amazonaws.com \\
      -u admin -p
```

4. Enter the password and start using the database.

From your local machine (optional – for testing)

1. Add your IP in the RDS security group (port 3306).
2. Use a MySQL client (e.g., MySQL Workbench).
3. Host: `mydb.xxxxx.rds.amazonaws.com`
   Port: `3306`
   Username: `admin`
   Password: `******`

8 Monitoring and Management

- Amazon CloudWatch – Monitor metrics (CPU, connections, etc.).
- RDS Performance Insights – Analyze database performance.
- Automated Backups – Point-in-time recovery.
- Snapshots – Manual backup and restore.
- Multi-AZ – Automatic failover for high availability.
- Read Replicas – For read scaling.
- RDS Events – Track events and notifications.

9 Verify – RDS Instance is Created

RDS > Databases

Databases (1) | Create database

Find resources

DB identifier	Status	Engine	Region	Multi-AZ
mydb	Available	MySQL	ap-south-1	Yes

10 Key Takeaways

- ✅ RDS simplifies database management.
- ✅ Use Multi-AZ for high availability.
- ✅ Place RDS in private subnets (not publicly accessible).
- ✅ Use Security Groups to control access.
- ✅ Enable automated backups and monitor using CloudWatch.
- ✅ Use read replicas for scaling read workloads.
- ✅ Apply least privilege access.
- ✅ RDS is cost-effective and reduces operational overhead.

Common Use Cases

- Web and mobile applications
- Enterprise applications (ERP, CRM)
- E-commerce platforms
- Analytics and reporting databases

"Managed Databases for a Better Tomorrow"

1. Amazon DynamoDB – Overview

- Amazon DynamoDB is a fully managed, serverless, NoSQL database service.
- Provides fast and predictable performance with single-digit millisecond latency.
- Automatically scales to handle any amount of traffic.
- High availability and durability across multiple Availability Zones (AZs).
- No server provisioning, patching or maintenance.
- Supports key-value and document data models.
- Integrates with AWS services like Lambda, API Gateway, Cognito, etc.

> "DynamoDB is designed for scale, speed and serverless simplicity."

2. Key Concepts in DynamoDB

Concept	Description
Table	A collection of items (similar to a table in relational DB).
Item	A single record (a set of attributes).
Attribute	A key-value pair within an item.
Primary Key	Uniquely identifies each item. • Partition key (Hash key) – required. • Sort key (Range key) – optional (for composite primary key).
Throughput Capacity	Defines read/write capacity. • Provisioned (R/W capacity units) • On-Demand (auto scales, pay per request)
Indexes	LSI (Local Secondary Index) GSI (Global Secondary Index)
Streams	Capture changes to items (for real-time processing).
TTL (Time to Live)	Automatically deletes items at a specified time.
DynamoDB Accelerator (DAX)	In-memory cache for microsecond latency (optional).

3. Create a DynamoDB Table (Step-by-Step)



1 Open AWS Management Console
Go to DynamoDB → Tables → Create table.

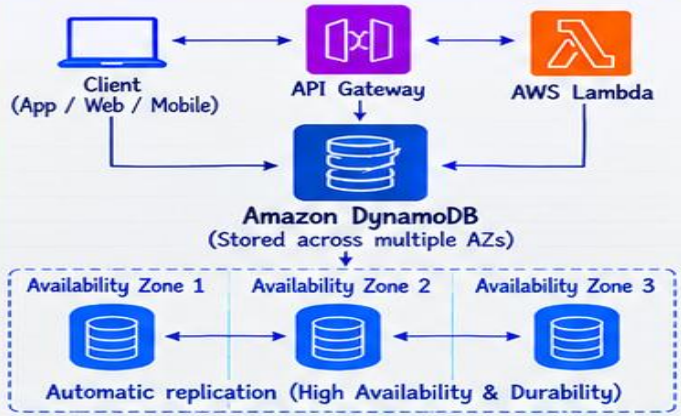
2 Enter table details
Table name: Students
Primary key: Partition key (e.g., StudentID) [String]
(Optionally add a sort key, e.g., CourseID)

3 Choose capacity mode
- On-Demand (recommended for most cases) – no need to set capacity
- Provisioned – specify Read/Write capacity units

4 Configure additional settings (optional)
- Enable Auto Scaling (for provisioned mode)
- Enable TTL (e.g., attribute: ExpiryTime)
- Enable encryption at rest (default AWS owned key or KMS key)
- Configure indexes (GSI/LSI) if needed
- Add tags (e.g., Project = Demo)

5 Review and create
Click **Create table**. It will be created in a few seconds (status: Active).

4. DynamoDB Architecture



DynamoDB automatically replicates your data across multiple AZs within a region.

5. Example: Create a Table (Student)

Table Configuration	
Table name	Students
Partition key	StudentID (String)
Sort key (optional)	CourseID (String)
Capacity mode	On-Demand
TTL (optional)	ExpiryTime
Additional settings	Default (encryption enabled)

Sample Item (JSON format)

```
{
  "StudentID": "S001",
  "CourseID": "CSE101",
  "Name": "Ravi Kumar",
  "Branch": "CSE",
  "Year": 2,
  "CGPA": 9.2,
  "Email": "ravi@example.com"
}
```

6. Access the Table (Operations)

Operation	AWS Console	AWS CLI	Example
Create table	Tables → Create table	aws dynamodb create-table	Create Students table
Put item (Insert)	Explore table → Create item	aws dynamodb put-item	Add new student record
Get item (Read)	Explore table → Get item	aws dynamodb get-item	Retrieve record by key
Update item	Edit item	aws dynamodb update-item	Update CGPA
Delete item	Delete item	aws dynamodb delete-item	Delete a record
Query (Using key)	Explore table → Query	aws dynamodb query	Get all courses of a student
Scan (All items)	Explore table → Scan	aws dynamodb scan	Retrieve all items

7. Use Cases & Best Practices

Common Use Cases
- User profiles (web/mobile apps)
- Session management
- Gaming leaderboards
- IoT device data
- Real-time analytics
- E-commerce catalogs
- Serverless applications

Best Practices
- ✅ Use appropriate primary key design
- ✅ Choose On-Demand for variable traffic
- ✅ Use GSI for additional access patterns
- ✅ Enable TTL for temporary data
- ✅ Use encryption (KMS) for sensitive data
- ✅ Monitor with CloudWatch
- ✅ Use IAM policies for fine-grained access
- ✅ Consider DAX for high read performance

8. Amazon RDS vs DynamoDB (Comparison)

Feature	Amazon RDS	Amazon DynamoDB
Type	Relational (SQL)	NoSQL (Non-relational)
Data model	Tables with fixed schema	Key-value / Document (flexible schema)
Query language	SQL (e.g., MySQL, PostgreSQL)	NoSQL API (GetItem, PutItem, Query, Scan)
Scalability	Vertical (scale up/down), read replicas	Automatic, seamless horizontal scaling
Use case	Transactional applications, complex queries, ACID compliance	High-scale, low-latency, serverless applications
Management	Manage instance (patching, minor upgrades)	Fully serverless (no infrastructure management)
Performance	Good, but scales with instance size	Single-digit millisecond latency, designed for scale
Schema	Fixed schema (needs changes for modification)	Flexible schema (no schema required)
Pricing	Pay for instance (compute + storage)	Pay per request or provisioned capacity
Best for	Applications needing complex queries, transactions, joins	Applications needing massive scale, simple access patterns, low latency

9. Key Takeaways

- ✅ Use Amazon RDS when you need a relational database with complex queries, transactions and SQL support.
- ✅ Use DynamoDB when you need a scalable, serverless, low-latency NoSQL database.
- ✅ Choose the right service based on your application requirements, data model and scaling needs.
- ✅ Both services integrate well with other AWS services.
- ✅ DynamoDB is ideal for modern serverless and high-scale applications, while RDS is suitable for traditional relational workloads.
- ✅ Follow AWS best practices for security, monitoring and cost optimization.

> "Right database for the right problem."

Amazon Redshift – Data Warehouse for Analytics

Fast, scalable, and fully managed data warehouse in the cloud

1 What is Amazon Redshift ?

- Amazon Redshift is a fully managed, petabyte-scale data warehouse service in AWS.
- Designed for fast analytic queries on large datasets using columnar storage and massive parallel processing (MPP).
- Suitable for business intelligence, reporting and data analytics workloads.
- Integrates with other AWS services (S3, Glue, Athena, QuickSight, etc.).

Amazon Redshift

"Redshift turns your data into insights at any scale."

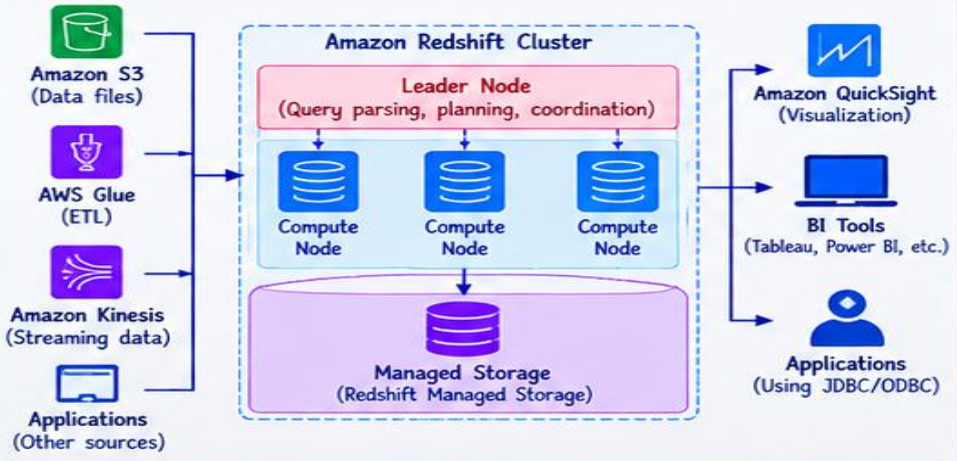
2 Key Features

- ✅ Columnar storage for high compression and faster queries
- ✅ Massively Parallel Processing (MPP) architecture
- ✅ Scales from GBs to PBs of data
- ✅ SQL-based (PostgreSQL compatible)
- ✅ Supports concurrency and workload management
- ✅ Integration with AWS services (S3, Glue, Kinesis, QuickSight)
- ✅ Data encryption (at rest using KMS, in transit with SSL)
- ✅ Automatic backups and snapshots
- ✅ Supports RA3, DC2 and Serverless architectures
- ✅ Materialized views, result caching and query optimization

3 Use Cases

- Business Intelligence
- Data Analytics
- Data Lake Analytics
- Reporting & Dashboards
- Customer Analytics
- E-commerce Analytics
- Operational Reporting
- Machine Learning (Data Preparation)

4 Amazon Redshift Architecture



Load data from multiple sources → Run fast analytical queries → Visualize insights

5 Steps to Create an Amazon Redshift Cluster

1. **Open AWS Console**
   Go to Amazon Redshift → Create cluster.
2. **Choose Cluster Type**
   - Provisioned (RA3, DC2) or
   - Serverless (recommended for new users).
3. **Configure Cluster Settings**
   - Cluster identifier (e.g., my-redshift)
   - Node type (e.g., ra3.xlplus)
   - Number of nodes (or use Serverless capacity)
   - Database name (e.g., dev)
   - Master username and password
   - Port (default: 5439)
4. **Network and Security**
   - Choose VPC and subnets
   - Create/choose a Security Group (allow port 5439)
   - (Optional) Choose publicly accessible or keep private.
5. **Additional Settings**
   - Enable automated backups (default: 1 day)
   - Enable encryption (KMS)
   - Configure maintenance window, parameter group.
6. **Review and Create**
   Click "Create cluster". It will take a few minutes.

6 Connect to Redshift

Using Query Editor v2 (in AWS Console)

1. Open Redshift → Query editor v2
2. Select cluster and enter database credentials
3. Start writing SQL queries

Using a SQL Client (e.g., DBeaver, DataGrip)

1. **Host:** <endpoint>.redshift.amazonaws.com
2. **Port:** 5439
3. **Database:** dev (your database)
4. **Username:** your user
5. **Password:** your password

Ensure your client IP or application security group is allowed in the Redshift security group.

7 Example: Create a Table and Load Data

1. **Create a table**

```
CREATE TABLE sales (
  id INT,
  product VARCHAR(100),
  amount DECIMAL(10,2),
  sale_date DATE
);
```

2. **Load data from S3 (using COPY command)**

```
COPY sales
 FROM 's3://my-bucket/sales-data/'
 IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftRole'
 FORMAT AS CSV
 IGNOREHEADER 1;
```

3. **Query data**

```
SELECT * FROM sales
 LIMIT 10;
```

8 Comparison: AWS Database Services

Feature	Amazon RDS	Amazon DynamoDB	Amazon Redshift
Type	Relational (SQL)	NoSQL (Key-Value / Document)	Data Warehouse (Analytical)
Use case	Transactional applications	High-scale, serverless applications	Analytics, BI, large-scale reporting
Data model	Tables (SQL)	Key-value / Document	Columnar storage (tables)
Scalability	Vertical (instance size, read replicas)	Automatic, serverless, virtually unlimited	Massive parallel processing (MPP)
Query language	SQL	NoSQL API (PartiQL supported)	SQL (PostgreSQL compatible)
Best for	OLTP, web apps, ERP, CRM	Real-time apps, session store, IoT	Data analytics, BI, data lake queries
Pricing	Pay per instance and storage	Pay per request or provisioned capacity	Pay per node/hour (or serverless)
Example engines	MySQL, PostgreSQL, Oracle, SQL Server	DynamoDB	Redshift (proprietary)

9 Best Practices for Redshift

- ✅ Use appropriate node type (RA3 for most cases)
- ✅ Use automated backups and snapshots
- ✅ Encrypt data at rest (KMS) and in transit (SSL)
- ✅ Use sort keys and distribution keys for better performance
- ✅ Use compression encoding (e.g., ZSTD, LZO)
- ✅ Load data from S3 using COPY (faster than INSERT)
- ✅ Monitor performance using Amazon CloudWatch
- ✅ Use materialized views for frequent queries
- ✅ Integrate with AWS Glue and QuickSight
- ✅ Delete unused data and clusters to save cost

10 Key Takeaways

- ✅ RDS → managed relational database (OLTP)
- ✅ DynamoDB → serverless NoSQL (high-scale, low-latency)
- ✅ Redshift → managed data warehouse (analytics)
- ✅ Choose the right service based on your workload.
- ✅ Redshift is optimized for large-scale analytic queries.
- ✅ Use AWS integration (S3, Glue, QuickSight) for end-to-end analytics solutions.
- ✅ Follow best practices for security, performance and cost.

Amazon Redshift – Data Warehouse for Analytics

Fast, Scalable, and Fully Managed Data Warehouse in the Cloud

"Turn your data into insights at any scale."

1. What is Amazon Redshift ?

- Amazon Redshift is a fully managed, petabyte-scale data warehouse service in AWS.
- Designed for fast analytical queries on large datasets using columnar storage and massively parallel processing (MPP).
- Optimized for OLAP (Online Analytical Processing) workloads (not for OLTP).
- Supports integration with other AWS services (S3, Glue, Kinesis, QuickSight, etc.).
- Available in two options:
  - Provisioned clusters
  - Serverless (auto-scaling, no infrastructure mgmt)
- Pay for what you use.

Amazon Redshift

"Redshift is built for high-performance analytics, not transactional workloads."

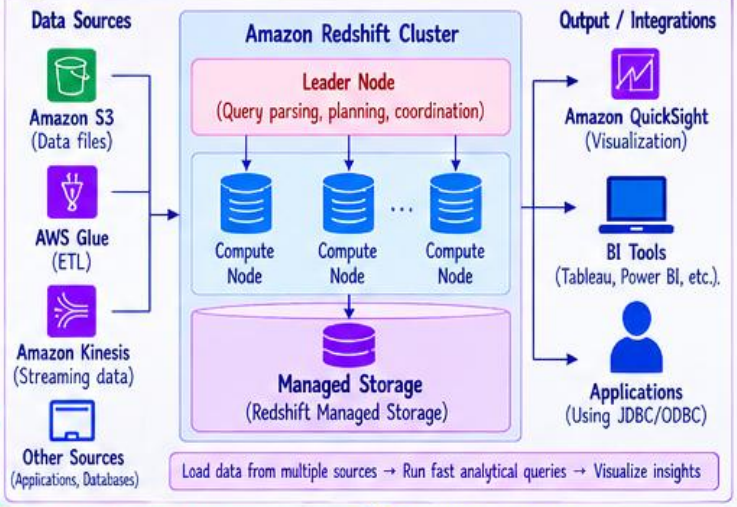
2. Key Features

- ✓ Columnar storage – high compression, faster queries
- ✓ Massively Parallel Processing (MPP)
- ✓ Scales from GBs to PBs of data
- ✓ Compatible with PostgreSQL (SQL-based)
- ✓ Supports concurrency and workload management
- ✓ Integration with S3, Glue, Kinesis, QuickSight, etc.
- ✓ Data encryption at rest (KMS) and in transit (SSL)
- ✓ Automated backups and snapshots
- ✓ Supports RA3, DC2 node types and Serverless
- ✓ Materialized views, result caching and query optimization
- ✓ Cost-effective for large analytical workloads

3. Common Use Cases

- Business Intelligence
- Data Analytics
- Reporting & Dashboards
- Customer Analytics
- E-commerce Analytics
- Machine Learning (Data Preparation)

4 Redshift Architecture



5 Create an Amazon Redshift Cluster (Step-by-Step)

1. **Open AWS Console**
   Search for Amazon Redshift → Click Create cluster.
2. **Choose Cluster Type**
   - Provisioned (RA3, DC2) or Serverless.
   - Choose node type and cluster size.
3. **Configure Cluster Settings**
   - Cluster identifier (e.g., my-redshift)
   - Database name (e.g., dev)
   - Master username and password
   - Node type (e.g., ra3.xlplus) and number of nodes
   - Select VPC, subnets and security group (port 5439)
   - Choose public accessibility (optional).
4. **Additional Settings**
   - Enable automated backups (default 1 day)
   - Enable encryption (AWS KMS)
   - Configure maintenance window and parameter group (if needed).
5. **Review and Create**
   - Click Create cluster. It will take a few minutes.

Note: For learning, you can also use Amazon Redshift Serverless (no cluster management).

6 Connect to Redshift

Using Query Editor v2 (in AWS Console)

1. Go to Redshift → Query editor v2.
2. Select your cluster and enter database credentials.
3. Start writing SQL queries.

Using a SQL Client (e.g., DBeaver, DataGrip)

1. Host: <endpoint>.redshift.amazonaws.com
2. Port: 5439
3. Database: dev
4. Username: your_user
5. Password: your_password

Ensure your client IP or application security group is allowed in the Redshift security group (port 5439).

7 Example: Create Table and Load Data

1. **Create a table**

```
CREATE TABLE sales (
  id INT,
  product VARCHAR(100),
  amount DECIMAL(10,2),
  sale_date DATE
);
```

2. **Load data from S3 (using COPY command)**

```
COPY sales
FROM 's3://my-bucket/sales-data/'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftRole'
FORMAT AS CSV
IGNOREHEADER 1;
```

3. **Query data**

```
SELECT * FROM sales
LIMIT 10;
```

8. Security and Management

Network Security
- Place cluster in private subnets (recommended).
- Use Security Groups (allow port 5439).
- Use IAM authentication (optional).

Encryption
- Encryption at rest using AWS KMS.
- Encryption in transit using SSL/TLS.

Backup & Recovery
- Automated backups (1–35 days).
- Snapshots for point-in-time recovery.

Monitoring
- Use Amazon CloudWatch for metrics and alarms.
- Monitor query performance, storage, connections.

Performance Optimization
- Use appropriate node type (RA3 for most cases).
- Use sort keys and distribution keys.
- Use compression encoding (e.g., ZSTD, LZO).
- Use spectrum to query data directly from S3 (no load).

9 Best Practices

- ✓ Use RA3 or Serverless for modern workloads.
- ✓ Choose right distribution and sort keys.
- ✓ Compress data to reduce storage and improve query performance.
- ✓ Load data from S3 using COPY (faster than INSERT).
- ✓ Use materialized views for frequent queries.
- ✓ Monitor using CloudWatch and set alarms.
- ✓ Use WLM (Workload Management) for query prioritization.
- ✓ Keep data in columnar format for analytics.
- ✓ Integrate with AWS Glue, S3, and QuickSight.
- ✓ Delete unused clusters to save cost.

10. Key Takeaways

- ✓ Redshift is a managed data warehouse for analytics.
- ✓ Best for large-scale, read-heavy (OLAP) workloads.
- ✓ Use Redshift when you need fast analytical queries on large datasets.
- ✓ Integrates with many AWS services.
- ✓ Choose between Provisioned and Serverless based on your needs.
- ✓ Follow best practices for security, performance and cost.

"Store. Analyze. Visualize. Scale. That's Amazon Redshift."

11. Comparison: Amazon RDS vs Amazon DynamoDB vs Amazon Redshift

Feature	Amazon RDS	Amazon DynamoDB	Amazon Redshift
Type	Relational Database (SQL)	NoSQL (Key-Value / Document)	Data Warehouse (Analytical)
Use case	Transactional applications (OLTP)	High-scale, low-latency, serverless apps	Analytics, BI, large-scale reporting (OLAP)
Data model	Tables (fixed schema)	Key-value / Document (flexible schema)	Columnar storage (tables)
Scaling	Vertical (instance size), read replicas	Automatic, serverless, on-demand	Massively parallel (MPP), scales to PBs
Query language	SQL (e.g., MySQL, PostgreSQL, Oracle, SQL Server)	NoSQL API, PartiQL (SQL-like)	SQL (PostgreSQL compatible)
Performance	Good for transactions, moderate scale	Single-digit millisecond latency	High performance for complex analytical queries
Pricing	Pay per instance and storage	Pay per request or provisioned capacity	Pay per node/hour (or serverless)
Best for	Web apps, ERP, CRM (relational data)	Real-time apps, IoT, mobile apps (massive scale)	Data analytics, reporting, BI, data lakes

AWS CloudFormation – Infrastructure as Code (IaC)

Automate • Reuse • Scale • Be Consistent

"Write once, Deploy anywhere"

1 What is AWS CloudFormation ?

- AWS CloudFormation is Infrastructure as Code (IaC) service for automated provisioning and management of AWS resources.
- You define your infrastructure in a template (JSON or YAML) and CloudFormation creates and manages the resources for you.
- Stacks are collections of AWS resources deployed as a single unit.
- Helps you provision, update and delete resources in a predictable and repeatable manner.

"Focus on your application, not on infrastructure management."

2 Key Components

Component	Description
Template	JSON or YAML file that defines AWS resources and their configuration.
Stack	A collection of AWS resources created and managed as a unit.
StackSet	Helps deploy and manage stacks across multiple accounts and regions.
Change Set	Previews the changes before updating a stack.
Stack Update	Updates the resources in the stack when template changes.
Rollback	Automatically reverts changes if an update fails.

3 How CloudFormation Solves Repeated Deployment ?

Without CloudFormation (Manual)

- ✗ Create resources manually for Dev, Test, Prod
- ✗ Time consuming and error prone
- ✗ Inconsistent configurations
- ✗ Difficult to track changes
- ✗ Hard to recover from failures

With CloudFormation (Template-based)

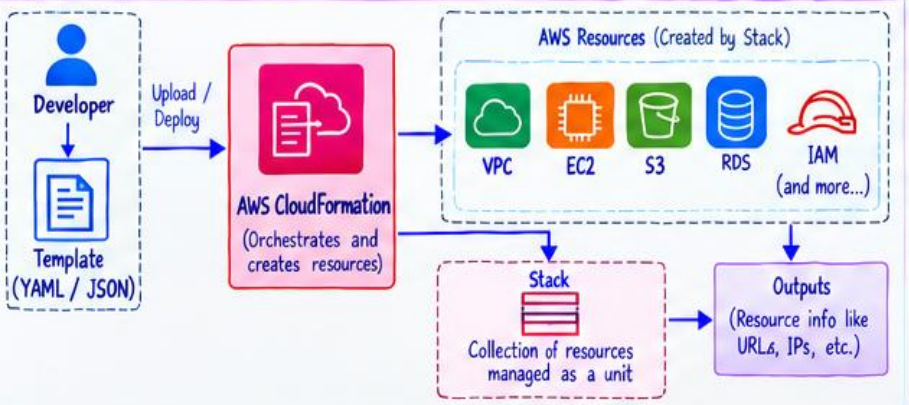
- ✓ Write one template and deploy multiple times
- ✓ Consistent and repeatable infrastructure
- ✓ Easy to version control
- ✓ Automatic or simple updates
- ✓ Rollback on failure
- ✓ Use StackSets for multiple accounts/regions

"Same template. Same infrastructure. Everywhere."

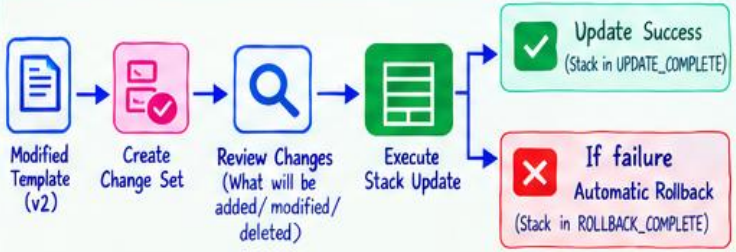
4 Use Cases

- Automated infrastructure deployment (for applications, environments)
- Deploy consistent infrastructure across multiple regions (using StackSets)
- Disaster recovery (quickly recreate infrastructure)
- Security compliance (standardized templates)
- Repeatable deployment for development, testing and production environments

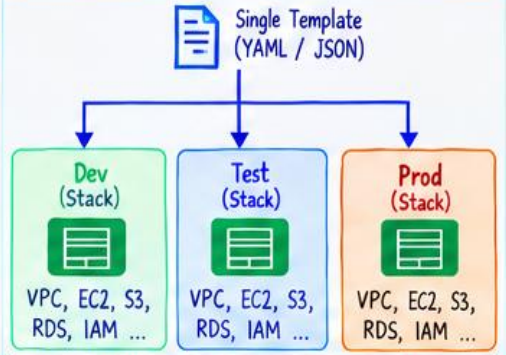
5 Architecture Diagram



6 Stack Update Flow with Change Set



7 Repeated Deployment Scenario (Dev / Test / Prod)



8 How Templates Help Recovery and Consistency ?

- ✓ Templates act as a single source of truth for your infrastructure.
- ✓ If resources are accidentally deleted or environment is lost, you can redeploy the entire stack using the same template.
- ✓ Ensures consistent configurations across Dev, Test, Prod.
- ✓ Version control (e.g., Git) helps track changes and maintain history.
- ✓ Rollback capability helps recover from failed updates.
- ✓ Use StackSets to maintain consistency across multiple accounts and regions.

"Templates + Version Control + Rollback = Reliable and Consistent Infrastructure"

9 Step-by-Step: Create a CloudFormation Stack

1. **Create a template** - Define resources in YAML or JSON.
2. **Validate template** - Check for syntax errors.
3. **Create stack** - Upload template in CloudFormation console.
4. **(Optional) Create Change Set** - Review changes before deployment.
5. **Deploy stack** - Execute the stack creation.
6. **Monitor events** - Check progress in Events tab.
7. **Update stack** - Modify template and create a change set.
8. **Rollback** - If update fails, stack automatically roll back.

10 Example Exam Scenario

Q. A company needs to deploy the same web application in Dev, Test and Prod environments. The deployments should be consistent, repeatable and easy to recover if something goes wrong. Which AWS service should be used and how would you achieve this?

Model Answer:

Use AWS CloudFormation. Create a template (YAML/JSON) that defines all required resources (e.g., VPC, EC2, S3, RDS, IAM). Deploy the template as a stack for each environment (Dev, Test, Prod). Use version control to track changes, use Change Sets to preview updates, and rely on automatic rollback for failed deployments. This ensures consistent, repeatable infrastructure and easy recovery by redeploying the stack using the same template.

11 Benefits (Important for Exam)

- ✓ Automated provisioning
- ✓ Consisting reporiment
- ✓ Consistent and repeatable deployments
- ✓ Version control and change tracking
- ✓ Rollback on failure
- ✓ Template reuse (save time and reduce errors)
- ✓ Deploy across multiple accounts/regions (StackSets)
- ✓ Cost optimization (deploy only what you need)

12 Key Takeaways

- ✓ CloudFormation = Infrastructure as Code (IaC)
- ✓ Templates (YAML/JSON) define resources
- ✓ Stacks manage resources as a single unit
- ✓ Change Sets preview changes
- ✓ Automatic rollback helps recovery
- ✓ Use templates for consistency and repeatability
- ✓ Use StackSets for multi-account and multi-region deployment

"Automate Today, Scale Tomorrow"

AWS Elastic Beanstalk – Overview

The easiest way to deploy, manage, and scale web applications in AWS

"Focus on your application not the infrastructure."

1 What is AWS Elastic Beanstalk ?

- AWS Elastic Beanstalk is a fully managed service that makes it easy to deploy, manage, and scale web applications and services in the AWS Cloud.
- You simply upload your application code, and Elastic Beanstalk automatically handles the infrastructure provisioning, configuration, load balancing, scaling, and monitoring.
- Supports popular platforms like Java, .NET, Node.js, Python, PHP, Ruby, Go, Docker, etc.
- Uses other AWS services under the hood (e.g., EC2, Auto Scaling, Elastic Load Balancing, RDS, CloudWatch, S3).

"Upload your code, and let Elastic Beanstalk take care of the rest."

2 Key Features

- **Easy Deployment** – Deploy code in minutes without managing infrastructure.
- **Automatic Provisioning** – Creates and configures AWS resources (EC2, ELB, Auto Scaling, etc.).
- **Auto Scaling** – Handles traffic spikes automatically.
- **Load Balancing** – Distributes traffic across multiple instances.
- **Health Monitoring** – Integrated with Amazon CloudWatch.
- **Configuration Management** – Configure via console, CLI, or .ebextenstions.
- **Versioning & Rollback** – Easily deploy previous versions if needed.
- **Supports Multiple Platforms** – Java, Python, Node.js, .NET, PHP, Ruby, Go, Docker, etc.
- **Cost-Effective** – Pay only for the underlying resources you use.

3 How Elastic Beanstalk Works ?

Developer
(Upload Code)

↓

AWS Elastic Beanstalk
(Manages deployment, scaling, monitoring)

Automatically provisions and manages

- **Amazon EC2** (Application instances)
- **Elastic Load Balancing** (Distributes traffic)
- **Auto Scaling** (Handles scale up/down)
- **Amazon RDS** (Optionally for database)
- **Amazon CloudWatch** (Monitoring & Logs)
- **Amazon S3** (Store application versions)

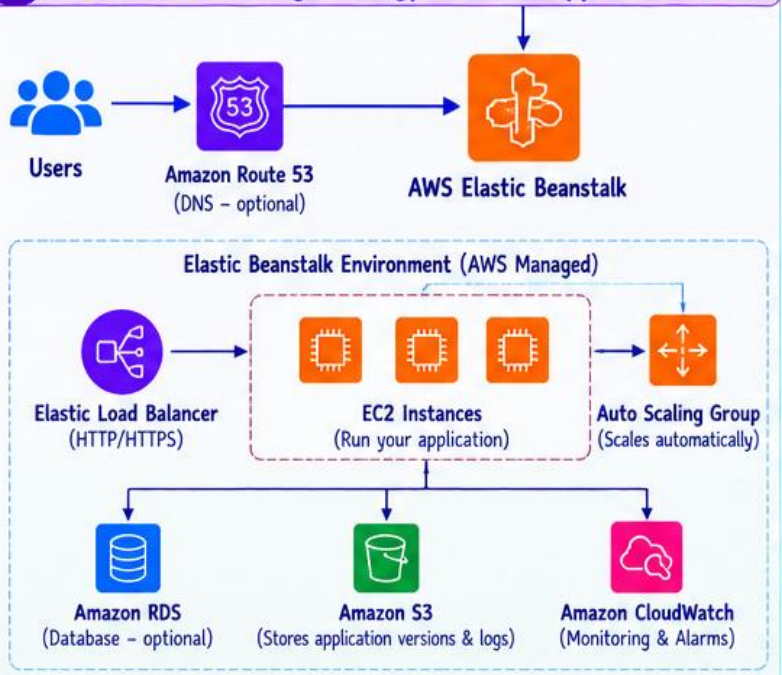
You manage your code. Elastic Beanstalk manages the infrastructure."

4 Supported Platforms

Platform	Examples / Versions
Java	Corretto, OpenJDK, Tomcat
.NET	.NET (Windows/Linux)
Node.js	Node.js (LTS versions)
Python	Python (Django, Flask, etc.)
PHP	PHP (with Apache/Nginx)
Ruby	Ruby (Passenger)
Go	Go (Linux)
Docker	Custom Docker containers
Others	Supports platform customizations

You can also use a custom platform if your platform is not listed.

5 Architecture Diagram (Typical Web Application)



6 Step-by-Step: Deploy Application on Elastic Beanstalk

1. **Prepare Your Application**
   - Create your application (e.g., Node.js, Python, Java, etc.).
   - Zip the source code (e.g., app.zip).
2. **Open Elastic Beanstalk Console**
   Go to AWS Management Console → Elastic Beanstalk → Create application.
3. **Configure Application**
   Enter application name and choose platform (e.g., Node.js).
4. **Configure Environment**
   Choose environment type (web server) and configuration (e.g., single instance or load balanced, VPC settings, database if needed).
5. **Upload Application Code**
   Upload your zip file or connect to a code repository (GitHub, AWS CodeCommit).
6. **Review and Create**
   Review settings and click Create environment.
7. **Access Your Application**
   Once the environment is ready, you'll get a URL (e.g., http://myapp.elasticbeanstalk.com).
8. **Update or Rollback**
   To update, upload a new version. To rollback, select a previous version.

7 Environment Types

Web Server Environment	Worker Environment
• For web applications (handles HTTP/HTTPS traffic). • Includes a load balancer and auto scaling (optional). • Suitable for most use cases.	• For background processing (e.g., queue consumers, batch jobs). • Does not include a load balancer. • Useful for long-running tasks (e.g., video processing, data processing, scheduled jobs).

8 Use Cases

- Deploy web applications quickly.
- Host APIs and backend services.
- Deploy applications with minimal DevOps effort.
- Educational projects and prototypes.
- Handle variable or unpredictable traffic.
- Run background processing jobs.
- Use with RDS for database-backed applications.
- Quickly recreate environments for disaster recovery.

9 Key Takeaways

- Elastic Beanstalk simplifies application deployment and management.
- It automatically handles infrastructure provisioning, scaling, load balancing, and monitoring.
- Supports multiple programming languages and platforms.
- Ideal for developers who want to focus on code, not infrastructure.
- Integrates with other AWS services like EC2, ELB, Auto Scaling, RDS, S3, and CloudWatch.
- Use versioning and rollback for safe deployment.
- Choose the right environment type (Web Server or Worker) based on your application needs.

"Build. Deploy. Scale.
Without the complexity."

AWS Elastic Beanstalk

AWS Cloud Development Kit (CDK) – Infrastructure as Code with Familiar Languages

"Higher level abstractions for a better developer experience."

1 What is AWS CDK ?

- AWS Cloud Development Kit (CDK) is an open source framework to define cloud infrastructure using familiar programming languages.
- It provides higher-level abstractions (constructs) that map to AWS resources.
- Your CDK code is synthesized into an AWS CloudFormation template, which is then deployed by AWS CloudFormation.
- Helps you use software development best practices (IDE, version control, testing, reuse) for infrastructure.

"Write infrastructure in code, not in JSON/YAML."

2 Core Concepts and Hierarchy

- **App** – The root of a CDK application. It contains one or more stacks.
- **Stack** – A deployment unit. Maps to a CloudFormation stack. Contains constructs and resources.
- **Constructs** – Building blocks for your application. Can be high-level (e.g., s3.Bucket) or low-level (L1, L2, L3 constructs).
- **AWS Resources** – Actual AWS resources (e.g., S3, Lambda, VPC, etc.) that will be provisioned via CloudFormation.

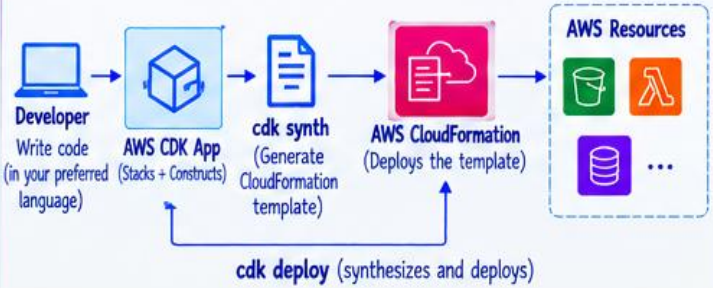
Hierarchy: App → Stack → Constructs → AWS Resources

3 Supported Programming Languages (AWS CDK v2)

- TypeScript (Recommended)
- JavaScript
- Python
- Java
- C# (.NET)
- Go

All languages use the same AWS CDK libraries and construct model.

4 CDK Architecture and Deployment Flow



AWS CDK does not deploy resources directly. It generates a CloudFormation template, and AWS CloudFormation performs the actual deployment.

5 Common AWS CDK CLI Commands

Command	Purpose
cdk init	Create a new CDK project (app).
cdk bootstrap	Set up required resources in your AWS account/region (CDKToolkit).
cdk synth	Synthesize your CDK app into a CloudFormation template.
cdk diff	Show differences between deployed stack and current code.
cdk deploy	Synthesize and deploy the stack to AWS (using CloudFormation).
cdk destroy	Delete the stack and its resources (using CloudFormation).

6 Steps to Get Started and Deploy

1. **Install prerequisites** – Install Node.js, AWS CDK CLI (npm install -g aws-cdk), configure AWS credentials (AWS CLI).
2. **Initialize project** – cdk init app --language <language>
3. **Bootstrap environment** – cdk bootstrap aws://<account-id>/<region>
4. **Develop your stack** – Define constructs and resources in code.
5. **Synthesize template** – cdk synth (generates CloudFormation template)
6. **Deploy** – cdk deploy (deploys using CloudFormation)
7. **Update / Destroy** – Modify code and run cdk deploy or use cdk destroy to remove resources.

7 CDK Bootstrap (What happens?)

- Before deploying, you must run cdk bootstrap once per AWS account/region.
- This creates a set of resources (CDKToolkit) used by CDK to deploy your applications, including:

CDKToolkit (created by bootstrap)

- **S3 bucket** (stores assets like Lambda code, Docker images)
- **ECR repository** (stores container assets, if needed)
- **IAM roles** (used by CDK (for deployment)

- Bootstrap resources are created per account/region and can be shared by multiple CDK apps.
- The exact resources may vary based on CDK version and options, but typically include S3, ECR and IAM roles.

8 Example: Simple CDK App (Python)

```python
from aws_cdk import App, Stack
from aws_cdk import aws_s3 as s3

class MyStack(Stack):
    def __init__(self, scope, id, **kwargs):
        super().__init__(scope, id, **kwargs)

        # Create an S3 bucket
        bucket = s3.Bucket(self, "MyBucket",
            versioned=True,
            removal_policy=s3.RemovalPolicy.DESTROY)

app = App()
MyStack(app, "MyStack")
app.synth()
```

This code creates a CDK app with one stack that provisions an S3 bucket. CDK will synthesize this into a CloudFormation template and deploy it using cdk deploy.

9 Scenario: Repeated Deployment Across Environments

Q A company wants to deploy the same infrastructure (VPC, S3, Lambda, etc.) to Dev, Test and Prod environments repeatedly, How can AWS CDK help?

A

- ✓ Define reusable constructs (e.g., a custom VpcConstruct, StorageConstruct, etc.).
- ✓ Create a stack that uses these constructs.
- ✓ Use different configuration (context, environment variables, or CDK Stages) for Dev, Test and Prod (e.g., different account/region or parameter values).
- ✓ Synthesize and deploy the same code to each environment.
- ✓ This ensures consistency, repeatability and easier maintenance.

10 CDK vs CloudFormation

Feature	AWS CDK	AWS CloudFormation
What it is	Open source framework (software development kit)	AWS service
Language	TypeScript, JavaScript, Python, Java, C#, Go	JSON or YAML templates
Purpose	Define infrastructure using familiar programming languages	Provision and manage resources using templates
Abstraction	Higher-level constructs (L1, L2, L3)	Low-level resource definitions
Deployment	Uses CloudFormation to deploy	Deploys stacks directly
Best for	Developers who want code-based infrastructure	Managing the actual deployment and stack lifecycle

11 Key Takeaways (Exam Focus)

- ✓ AWS CDK is an Infrastructure as Code (IaC) framework, not a cloud service.
- ✓ CDK generates AWS CloudFormation templates (using cdk synth).
- ✓ AWS CloudFormation performs the actual deployment (using cdk deploy).
- ✓ Main concepts: App, Stack, Constructs.
- ✓ Stacks are the smallest deployment unit in CDK (maps to a CloudFormation stack).
- ✓ Constructs are building blocks that represent AWS resources or higher-level components.
- ✓ You must run cdk bootstrap before deploying to a new account/region.
- ✓ CDK supports TypeScript, JavaScript, Python, Java, C# and Go (v2).
- ✓ Useful for repeatable, consistent deployments across multiple environments.
- ✓ Does not replace CloudFormation – it works on top of it.
- ✓ Supports best practices: version control, code reuse, testing and CI/CD.

"Same infrastructure. Better code. Greater possibilities."

AWS CodePipeline (CI/CD) – Automate Build, Test and Deploy

"From code commit to running application – fully automated."

1 What is AWS CodePipeline ?

- AWS CodePipeline is a fully managed continuous integration and continuous delivery (CI/CD) service.
- It automates the release process for your applications whenever there is a code change.
- Coordinates the build, test and deploy stages using other AWS services (e.g., CodeCommit, CodeBuild, CodeDeploy, S3, Elastic Beanstalk, ECS, etc.).
- Provides a visual workflow, version history, approval actions and easy rollback.
- Pay only for what you use.

"Automate your software delivery, from commit to production."

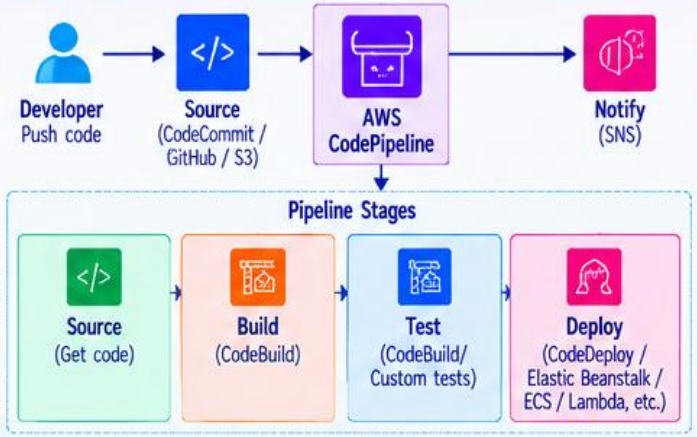
2 Key Features

- ✅ Fully managed CI/CD service
- ✅ Automates build, test and deployment
- ✅ Integrates with AWS and third-party tools (GitHub, GitLab, Bitbucket, etc.)
- ✅ Supports manual approval actions
- ✅ Tracks release history and provides visibility
- ✅ Easy rollback to previous versions
- ✅ Works with multiple deployment targets (EC2, ECS, Elastic Beanstalk, Lambda, etc.)
- ✅ Secure with IAM and encryption
- ✅ Scales with your application

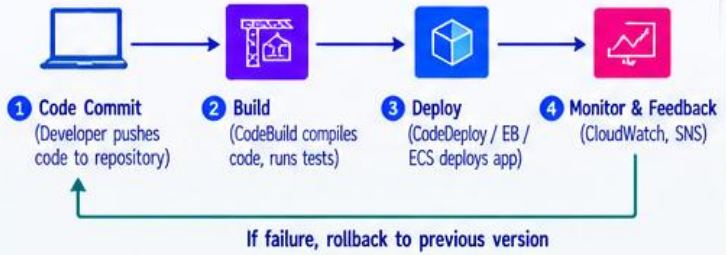
3 CI/CD Terminology

Term	Meaning
Continuous Integration (CI)	Automatically build and test code when changes are pushed.
Continuous Delivery (CD)	Automatically deploy to staging/production (with optional approval).
Pipeline	A workflow of stages and actions that define the release process.
Source	Where the code is stored (CodeCommit, GitHub, S3, etc.).
Build	Compiles the code and runs tests (CodeBuild).
Deploy	Deploys the application (CodeDeploy, Elastic Beanstalk, etc.).
Artifact	Output (e.g., build files) passed between stages.

4 High-Level Architecture



5 CI/CD Workflow



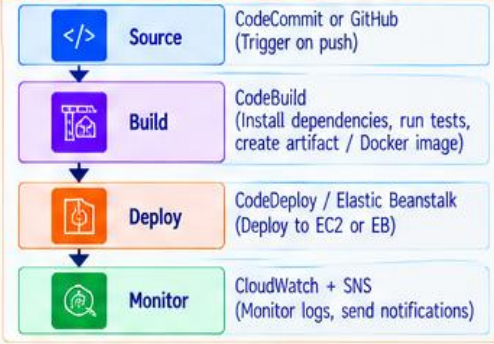
6 Major Services Used

Service	Purpose in CI/CD
CodePipeline	Orchestrates the release pipeline
CodeCommit	Source code repository (Git)
CodeBuild	Builds source code, runs tests
CodeDeploy	Deploys to EC2, on-premises, Lambda, etc.
Elastic Beanstalk	Simplified deployment for web apps
ECR	Stores Docker container images
S3	Stores build artifacts
CloudWatch	Logs, metrics, alarms
SNS	Notifications (success/failure)
IAM	Access control for pipeline and services

7 Deployment Targets



8 Example Pipeline (Web Application)



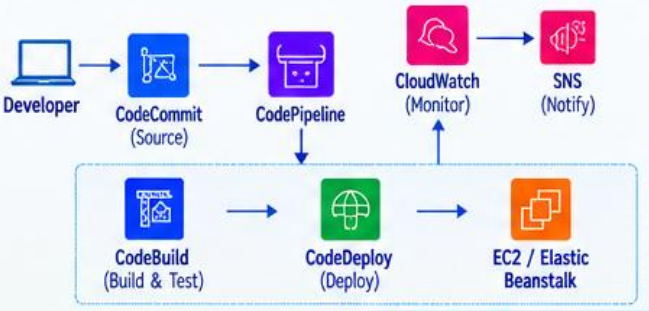
9 Steps to Create a CodePipeline (Console)

1. Create a source repository (CodeCommit or connect to GitHub).
2. Create a build project in CodeBuild (specify build commands, e.g., npm build).
3. (Optional) Create a deployment target (EC2/EB/ECS) and CodeDeploy app.
4. Open AWS CodePipeline → Create pipeline.
5. Choose pipeline name and service role (IAM).
6. Add Source stage → connect to repository.
7. Add Build stage → select CodeBuild project.
8. Add Deploy stage → select CodeDeploy / Elastic Beanstalk / ECS.
9. Configure optional manual approval, environment variables, artifacts.
10. Review and create pipeline.

10 Example buildspec.yml (Node.js)

```yaml
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 18
    commands:
      - npm install
  build:
    commands:
      - npm run build
  post_build:
    commands:
      - echo Build completed!
artifacts:
  files:
    - '**/*'
  base-directory: dist
```

11 Example pipeline architecture (with AWS services)



12 Best Practices

- ✅ Keep pipeline stages modular.
- ✅ Use automated tests before deployment.
- ✅ Use separate pipelines for Dev, Test, Prod.
- ✅ Enable manual approval for production.
- ✅ Store artifacts in S3.
- ✅ Use IAM roles with least privilege.
- ✅ Monitor using CloudWatch and set alarms.
- ✅ Use versioning and rollback for safe deployments.
- ✅ Integrate with infrastructure as code (CloudFormation / CDK).
- ✅ Delete unused resources to save cost.

13 Scenario-Based Question

Q: A company wants to automatically build, test and deploy a web application whenever developers push code to a GitHub repository. They also want to roll back to a previous version if the deployment fails. Which AWS services and pipeline stages would you use?

Answer:

- ✅ Use **AWS CodePipeline** with stages: Source (GitHub), Build (CodeBuild), Deploy (CodeDeploy or Elastic Beanstalk).
- ✅ Enable versioning and rollback in CodeDeploy / Elastic Beanstalk.
- ✅ Use CloudWatch and SNS for monitoring and notifications.

14 Key Takeaways

- ✅ CodePipeline automates the entire CI/CD release process.
- ✅ Integrates with multiple AWS services.
- ✅ Provides visibility, rollback and manual approval.
- ✅ Use with CodeBuild, CodeDeploy, Elastic Beanstalk, ECS, etc.
- ✅ Helps deliver software faster, more reliably and with fewer errors.

"Commit
Build
Test
Deploy
Repeat
Better Software!"

AWS Module 4 - Exam Cheat Sheet

Build · Deploy · Automate · Repeat · Scale

Focus on Concepts, Architecture, Use Cases and Exam Scenarios

Same - Concepts Bigger Opportunities!

1 AWS CloudFormation (Infrastructure as Code)

What is CloudFormation ?

- AWS CloudFormation is an Infrastructure as Code (IaC) service that lets you define and provision AWS resources using templates (JSON or YAML).
- It creates and manages a stack of AWS resources as a single unit.
- Helps you deploy, update and delete resources in a predictable and repeatable manner.
- Ideal for automated and consistent infrastructure deployment across environments.

"Write infrastructure once, deploy everywhere."

Key Components

Template	JSON or YAML file defining AWS resources.
Stack	Collection of resources created from a template.
Change Set	Shows the changes before applying (create/update).
Stack Update	Updates stack with modified template.
Rollback	Automatically reverts changes if update fails.
StackSet	Deploys stacks across multiple accounts/regions.

Architecture Diagram



Update Flow (Change Set & Rollback



How Templates Help ?

- ✅ Consistency across environments (Dev, Test, Prod)
- ✅ Repeatable deployments
- ✅ Version control (use with Git)
- ✅ Easy recovery and disaster recovery (redeploy same template)
- ✅ Multi-account / multi-region with StackSets

Steps to Create a Stack

1. Prepare template (YAML/JSON)
2. Open CloudFormation console
3. Create stack (choose template)
4. Configure parameters
5. Review and create (or use change set)
6. Monitor (Events tab)

Exam Scenario: A company wants same infrastructure in Dev, Test, Prod. How to do this?

Answer: Use a CloudFormation template (and StackSets for multi-region/account) to ensure consistent, repeatable deployment.

2 AWS Elastic Beanstalk (PaaS)

What is Elastic Beanstalk ?

- AWS Elastic Beanstalk is a fully managed Platform as a Service (PaaS) to deploy, manage and scale web applications.
- It automatically handles infrastructure provisioning, load balancing, scaling, monitoring and health management.
- You just upload your code and Elastic Beanstalk takes care of the rest.
- Supports multiple platforms: Java, .NET, PHP, Node.js, Python, Ruby, Go, Docker etc.

"Focus on your application code, not the infrastructure."

Key Features

- ✅ Automatic provisioning of AWS resources
- ✅ Integrated load balancing and auto scaling
- ✅ Health monitoring (with CloudWatch)
- ✅ Easy deployment (upload code)
- ✅ Supports multiple platforms
- ✅ Environment management (web server / worker)

Supported Platforms

Java · .NET · PHP · Node.js · Python · Ruby · Go · Docker

Architecture Diagram



Environment Types

Web Server Environment

- For web applications (HTTP/HTTPS)
- Includes load balancer and auto scaling
- Handles web traffic
- Suitable for most use cases

Worker Environment

- For background processing
- No load balancer
- Useful for long-running tasks (e.g., batch jobs, queues)

Deployment Steps

1. Prepare application code (zip)
2. Create application and environment
3. Choose platform (e.g., Python)
4. Configure (instance type, scaling, etc.)
5. Deploy and monitor (auto scaling, health)

Use Cases

- ✅ Deploy web applications quickly
- ✅ Handle variable traffic
- ✅ APIs and backend services
- ✅ Educational projects/prototypes
- ✅ Managed infrastructure for devs

Exam Scenario: A startup wants to deploy a Python web app. Dev, Test, Prod. managing servers should automate and more.

Answer: Use Elastic Beanstalk to deploy, manage, & scale code (auto handles/account)

3 AWS Cloud Development Kit (CDK)

What is AWS CDK ?

- AWS CDK is an open source Infrastructure as Code (IaC) framework to define AWS infrastructure using familiar programming languages.
- It synthesizes your code into a CloudFormation template, which is then deployed by CloudFormation.
- Provides higher-level abstractions (constructs) for AWS resources.
- Not a replacement for CloudFormation, but works with it.

"Use your favorite programming language to create AWS infrastructure."

Hierarchy of Constructs

App (The root of your CDK application)
↓
Stack (A deployment unit)
↓
Constructs (Building blocks: L1, L2, L3)
↓
AWS Resources (e.g., S3, Lambda, VPC)

Supported Languages

Python · TypeScript · JavaScript · Java · C# (.NET) · Go

CDK Workflow



Common CDK Commands

cdk init	Create a new CDK project
cdk bootstrap	Set up required resources in your AWS account/region
cdk synth	Generate CloudFormation template
cdk deploy	Deploys the stack to AWS (uses CloudFormation)
cdk diff	Show differences between deployed and current code
cdk destroy	Delete the stack and resources

Example (Python) - Create an S3 Bucket

```
from aws_cdk import App, Stack
from aws_cdk import aws_s3 as s3

class MyStack(Stack):
  def __init__(self, scope, id, **kwargs):
    super().__init__(scope, id, **kwargs)
    s3.Bucket(self, "MyBucket",
        versioned=True)
app = App()
MyStack(app, "MyStack")
app.synth()
```

CDK vs CloudFormation

- CDK = Code (Python/TypeScript etc.)
- Synthesizes to CloudFormation
- Higher-level abstractions
- CloudFormation = YAML/JSON (declarative)
- Both provide IaC, repeatability and consistency

* CDK is complementary, not a replacement replacement

Exam Scenario: You want to define infrastructure using Python and deploy it. Which service and command would you use?

Answer: Use AWS CDK. Write code and run "cdk deploy" (after cdk bootstrap).

4 AWS CodePipeline (CI/CD)

What is AWS CodePipeline ?

- AWS CodePipeline is a fully managed continuous integration and continuous delivery (CI/CD) service.
- It automates the release process for your applications whenever there is a code change.
- Coordinates build, test and deploy stages using other services (e.g., CodeCommit, CodeBuild, CodeDeploy, Elastic Beanstalk, etc.).
- Provides a visual workflow, release history and approval actions.
- Helps prevent unsuccessful versions from being deployed (through automated tests and manual approvals).

"Automate your software delivery, from commit to production."

CI/CD Terminology

Continuous Integration (CI)	Automatically build and test code.
Continuous Delivery (CD)	Automatically deploy to staging/production.
Pipeline	Workflow of stages and actions.
Source	Where code is stored (CodeCommit/GitHub).
Build	Compiles the code (CodeBuild).
Test	Runs automated tests (CodeBuild/others).
Deploy	Deploys to target (CodeDeploy / Beanstalk / ECS/ Lambda).
Artifact	Output (e.g., build files) stored in S3 between stages.

CI/CD Architecture



Major AWS Services

CodePipeline (Orchestrates) · CodeBuild (Build + Test) · CodeDeploy (Deploy) · CodeCommit / GitHub (Source) · S3 (Artifacts) · CloudWatch (Monitoring) · SNS (Notifications) · IAM (Access control)

Steps to Set Up (Console)

1. Create a pipeline
2. Choose source (CodeCommit/GitHub)
3. Add build stage (CodeBuild)
4. Add test stage (automated tests)
5. Add deploy stage (CodeDeploy/Beanstalk/etc.)
6. Configure notifications and monitoring

Use Cases

- ✅ Automate release for web apps
- ✅ Ensure only tested code is deployed
- ✅ Continuous delivery to Dev, Test, Prod
- ✅ Supports manual approval for production
- ✅ Reduces human errors

Exam Scenario: Code should be automatically built, tested and deployed when developers push to GitHub. How to design this?

Answer: Use AWS CodePipeline with Source (GitHub) → Build (CodeBuild) → Test → Deploy (CodeDeploy/Beanstalk) → Monitor (CloudWatch + SNS).

5 Comparison Table (CloudFormation vs CDK vs Elastic Beanstalk vs AWS CodePipeline

Feature / Aspect	CloudFormation	AWS CDK	Elastic Beanstalk	AWS CodePipeline
Purpose	Infrastructure as Code	Infrastructure as Code (in code)	Application deployment (PaaS)	CI/CD orchestration
Main job	Provision and manage AWS resources	Define infrastructure using programming language (synth to CFN)	Deploy and manage web applications	Automate build, test and deploy
Input	YAML/JSON template	Code (Python/TypeScript/Java etc.)	Application code (zip/war)	Source code (GitHub/CodeCommit)
Output / Role	Creates/updates AWS resources	Generates CloudFormation template and deploys resources	Running application environment with managed infrastructure	Automated release pipeline
Typical scenario	Same infrastructure in multiple envs	IaC using familiar language	Quickly deploy a web app without managing servers	Automate build, test, deploy on code change

6 Exam Scenarios & Answer Structure

1. Deploy same infrastructure in Dev, Test, Prod.
   → Use CloudFormation (and StackSets for multi-account/region).
2. Deploy a web application quickly without managing servers.
   → Use Elastic Beanstalk.
3. Define infrastructure using Python and deploy it.
   → Use AWS CDK (cdk deploy → synthesizes to CloudFormation).
4. Automatically build, test and deploy when code is pushed to GitHub.
   → Use AWS CodePipeline with CodeBuild, tests and deploy (CodeDeploy/Beanstalk).

7 Key Exam Takeaways

- ✅ CloudFormation = IaC (YAML/JSON template), consistent & repeatable.
- ✅ CDK = IaC in code (Python/TS/Java etc.), synthesizes to CloudFormation.
- ✅ Elastic Beanstalk = PaaS, deploy application code, manages infrastructure.
- ✅ CodePipeline = CI/CD service, automates build, test, deploy.
- ✅ Use Change Sets and Rollback for safe updates in CloudFormation.
- ✅ Beanstalk handles provisioning, scaling, load balancing and monitoring.
- ✅ CDK is not a replacement for CloudFormation, it works with it.
- ✅ Automated tests and approvals in CI/CD prevent bad deployments.
- ✅ Use CloudWatch and SNS for monitoring and notifications.

"Learn Deploy Automate Succeed"

AWS Built for Builders

AWS Elastic Load Balancer (ELB)

Distribute Traffic • Improve Availability • Scale Automatically

"A single endpoint for your application that distributes incoming traffic across multiple targets."

High Availability
Better Performance
Scalable Applications

1 What is AWS ELB ?

- AWS Elastic Load Balancing (ELB) automatically distributes incoming application traffic across multiple targets such as EC2 instances, containers (ECS), IP addresses, or Lambda functions.
- It improves application availability, fault tolerance, and scalability.
- It automatically scales with the number of targets.
- It performs health checks to ensure traffic is sent only to healthy targets.
- It integrates with Auto Scaling, CloudWatch, WAF and other AWS services.

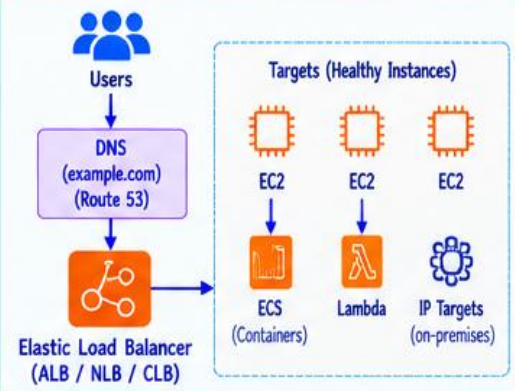
"ELB ensures that your application is highly available and responsive."

2 Types of Load Balancers

Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
• Works at Layer 7 (HTTP/HTTPS) • Routes traffic based on content (path, host, headers, query) • Best for web applications and microservices • Supports advanced routing (e.g., /api, /images) • Supports WebSockets, HTTP/2	• Works at Layer 4 (TCP/UDP) • Handles millions of requests with ultra-low latency • Best for high performance and real-time applications • Supports static IP addresses • Preserves source IP • Ideal for gaming, IoT, financial apps	• Legacy generation (EC2-Classic and VPC) • Supports Layer 4 and 7 (basic) • Use only for older applications • Limited features (compared to ALB/NLB)

Use ALB for most HTTP/HTTPS applications. Use NLB for high-performance TCP/UDP. Use CLB only for legacy apps.

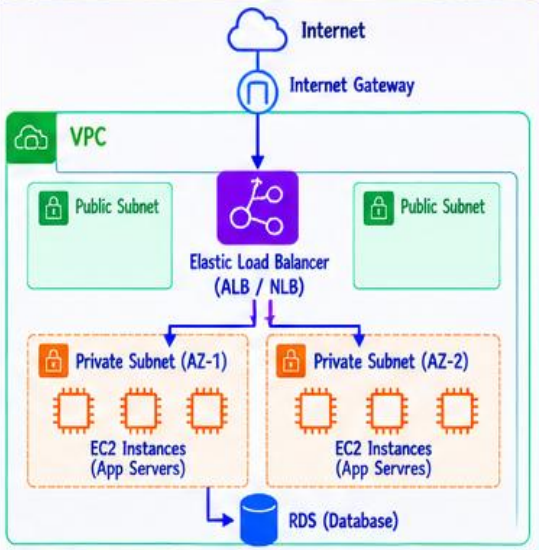
3 How ELB Works ? (High-Level Flow)



Key Steps

1. User sends request to a DNS name (Route 53).
2. ELB receives the request and performs health checks.
3. Traffic is distributed to healthy targets based on the configured routing rules.
4. If a target fails, traffic is automatically routed to other healthy targets.

4 ELB Architecture (In a VPC)



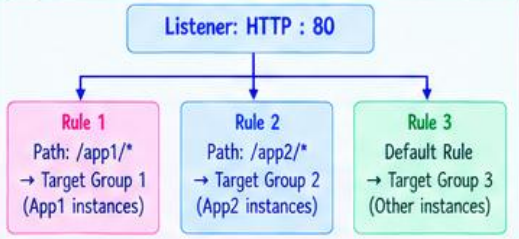
5 Key Features

- ✓ Distributes traffic across multiple targets (EC2, ECS, Lambda, etc.)
- ✓ Improves availability and fault tolerance
- ✓ Performs health checks (configurable)
- ✓ Integrates with Auto Scaling (adds/removes instances automatically)
- ✓ Supports multiple load balancer types (ALB, NLB, CLB)
- ✓ SSL/TLS termination (HTTPS)
- ✓ Supports cross-zone load balancing (enabled by default for ALB/NLB)
- ✓ CloudWatch metrics and access logs
- ✓ Integrates with AWS WAF for protection
- ✓ Scales automatically with traffic
- ✓ Supports sticky sessions (session affinity)

6 Listener, Target Group and Health Checks

Listener	• Checks for incoming connection requests • Configured on a port (e.g., 80, 443) • Forwards requests to a Target Group • Can have rules (only in ALB) based on path, host, headers
Target Group	• Collection of targets (EC2, IP, Lambda, ECS) • Defines protocol and port • Performs health checks on targets • Routes traffic only to healthy targets
Health Check	• Periodically checks the health of each target • If unhealthy, traffic is not sent to that target • Configurable path (e.g., /health), interval, timeout, thresholds

7 Routing in ALB (Example)



ALB can route traffic to different target groups based on URL path, host name, headers, or query strings.

8 Use Cases

- Web applications (load balance HTTP/HTTPS traffic)
- Microservices (route based on path/host)
- Real-time applications (gaming, streaming)
- Internal applications (using NLB)
- Disaster recovery and multi-AZ high availability
- Blue/Green or Canary deployments (with CodeDeploy)
- Load balancing containers (ECS/EKS)
- Load balancing Lambda functions (ALB)

9 Integration with Other AWS Services

- **Auto Scaling** (automatically adjusts instances)
- **Route 53** (DNS for your application)
- **CloudWatch** (Monitoring & metrics)
- **AWS WAF** (Protects from web attacks)
- **ACM** (SSL/TLS certificates)
- **ECS / EKS** (Containers)
- **Lambda** (Serverless functions)

10 Create an Application Load Balancer (ALB) – Steps

1. Go to EC2 Console → Load Balancers → Create Load Balancer
2. Choose Application Load Balancer and click Create
3. Configure basic settings (name, scheme: internet-facing/internal, IP type)
4. Select VPC and at least two subnets (different AZs)
5. Configure security group (allow HTTP/HTTPS)
6. Configure listeners (e.g., HTTP:80 or HTTPS:443)
7. Create a target group (choose target type, protocol, health check path)
8. Register targets (EC2 instances, IPs, etc.)
9. Review and create

11 Comparison of Load Balancers

Feature	ALB	NLB	CLB
Layer	7 (HTTP/HTTPS)	4 (TCP/UDP)	4 & 7 (Basic)
Use case	Web apps, microservices	High performance, TCP/UDP apps	Legacy applications
Routing	Content-based (path, host, header)	IP and port based	Basic (host/path)
Performance	High	Ultra high	Moderate
Supports Static IP	No	Yes	No
Source IP	X-Forwarded-For	Preserved	Preserved (some cases)
WebSockets	Yes	Yes	Limited
Best for	Modern applications	Real-time, low latency	Older applications

12 Exam Tips & Key Takeaways

- ✓ ALB → Layer 7 (HTTP/HTTPS), content-based routing
- ✓ NLB → Layer 4 (TCP/UDP), static IP, ultra-low latency
- ✓ CLB → Legacy (use only if required)
- ✓ Health checks ensure traffic goes only to healthy targets
- ✓ Integrate ELB with Auto Scaling for high availability
- ✓ ELB automatically scales and is multi-AZ by default
- ✓ Use ALB for most web applications
- ✓ Use NLB for high-performance or TCP/UDP traffic
- ✓ Use AWS WAF with ALB for security
- ✓ Monitor using CloudWatch and enable access logs (S3)

"Choose the right load balancer for your application needs and architect for high availability across multiple AZs."

AWS Application Load Balancer (ALB) – Create, Configure and Related Concepts

High Availability | Better Performance | Scalable Applications

"Distribute traffic intelligently and keep your application always available."

1 What is Application Load Balancer (ALB)?

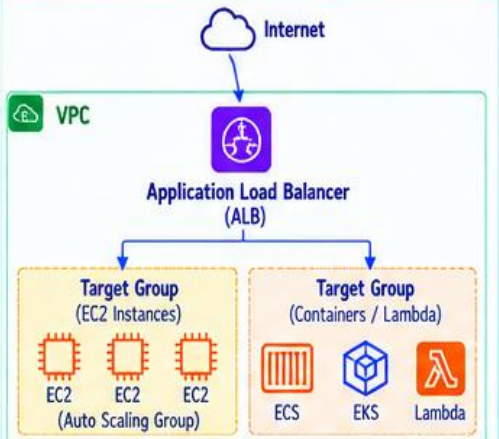
- Application Load Balancer (ALB) is a Layer 7 (HTTP/HTTPS) load balancer that distributes incoming traffic across multiple targets such as EC2 instances, containers (ECS), IP addresses, or Lambda functions.
- Routes traffic based on content (path, host, headers, query).
- Improves application availability, fault tolerance and scalability.
- Performs health checks to ensure traffic is sent only to healthy targets.
- Integrates with Auto Scaling, CloudWatch, AWS WAF, ACM, Route 53 and more.

"Use ALB for most HTTP/HTTPS applications (with advanced routing requirements)."

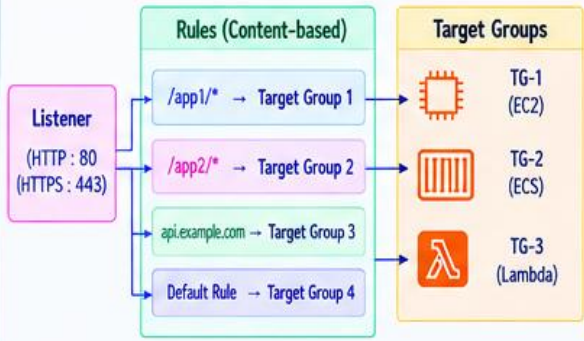
2 Types of Load Balancers

Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
• Layer 7 (HTTP/HTTPS)	• Layer 4 (TCP/UDP)	• Legacy generation
• Content-based routing (path, host, headers)	• Handles millions of requests with ultra-low latency	• Layer 4 and 7 (basic)
• Best for web applications and microservices	• Best for high performance and real-time applications	• Limited features
• Supports WebSockets	• Preserves source IP	• Use for older applications
• Integrates with WAF	• Use for non-HTTP traffic (TCP/UDP)	• Not recommended for new workloads
• Use for most modern apps		

3 ALB Architecture Diagram



4 How ALB Routes Traffic ? (Layer 7)



5 Step-by-Step: Create an Application Load Balancer

1. Go to EC2 Console → Load Balancers → Create Load Balancer → Application Load Balancer.
2. Configure Basic Settings: Name, Scheme (Internet-facing / Internal), IP type (IPv4/IPv6).
3. Select VPC and at least two subnets (different Availability Zones).
4. Configure Security Group: Allow inbound HTTP (80) or HTTPS (443).
5. Configure Listener: Choose protocol and port (e.g., HTTP:80 or HTTPS:443). For HTTPS, select an ACM certificate.
6. Create Target Group: Choose target type (Instance, IP, Lambda, or Container), protocol and health check path (e.g., /health).
7. Register Targets: Select the targets (EC2 instances / IPs / Lambda).
8. Review and Create the Load Balancer.

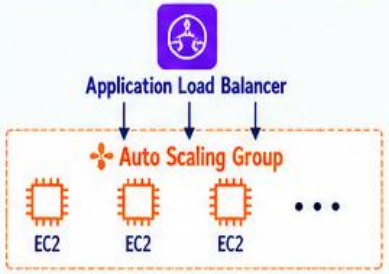
6 Health Checks

- ALB periodically checks the health of targets using a configured path (e.g., /health).
- Only healthy targets receive traffic.
- If a target is unhealthy, ALB stops sending traffic to it.

Common Health Check Settings	
Protocol	HTTP or HTTPS
Path	/health (custom)
Interval	30 seconds (default)
Timeout	5 seconds
Healthy threshold	5
Unhealthy threshold	2

7 Auto Scaling with ALB

- Attach the target group to an Auto Scaling Group (ASG).
- ALB distributes traffic across instances in the ASG.
- When demand increases, new instances are launched and automatically registered.
- When demand decreases, instances are terminated.



8 Scaling Policy (Important)

- A scaling policy defines when and how to add or remove instances in an Auto Scaling Group.
- Common policies:

Policy Type	Description
Target Tracking	Automatically scales to maintain a target metric (e.g., CPU 50%).
Step Scaling	Adds/removes instances based on metric thresholds.
Scheduled Scaling	Scales based on a schedule (e.g., business hours).

Common Metrics

- CPU Utilization
- Request Count per Target (ALB metric)
- Network In / Out
- Custom Application Metrics

9 Monitoring with CloudWatch

- Monitor ALB metrics to analyze performance and detect issues.
- Important ALB metrics:
  - RequestCount
  - TargetResponseTime
  - HTTPCode_Target_2XX / 4XX / 5XX
  - HealthyHostCount
  - UnHealthyHostCount
  - RejectedConnectionCount

Amazon CloudWatch

Use CloudWatch Alarms to trigger Auto Scaling policies or notify through SNS.

10 AWS CloudTrail (Audit & Governance)

- CloudTrail records API calls made in your AWS account.
- Helps in auditing, compliance and troubleshooting.
- For ALB, it logs actions like:
  - CreateLoadBalancer
  - DeleteLoadBalancer
  - CreateTargetGroup
  - RegisterTargets
  - ModifyLoadBalancerAttributes

AWS CloudTrail

Helps you analyze changes, detect unauthorized access and find the exact cause of issues.

11 Real-World Use Case

Scenario: A web application running on EC2 instances experiences high traffic. Some instances fail, and the application becomes slow.

Solution:

1. Use ALB to distribute traffic across multiple EC2 instances.
2. Configure health checks so failed instances are not used.
3. Use Auto Scaling to launch new instances when CPU is high.
4. Set a target tracking scaling policy (e.g., CPU 50%).
5. Monitor using CloudWatch and set alarms.
6. Enable CloudTrail for auditing changes.

Result: Improved application availability, better resource utilization, and automatic handling of traffic spikes.

12 Key Takeaways

- ✅ ALB is a Layer 7 load balancer for HTTP/HTTPS applications.
- ✅ Supports advanced routing (path, host, headers).
- ✅ Use target groups and health checks for high availability.
- ✅ Integrate ALB with Auto Scaling for automatic scaling.
- ✅ Use CloudWatch for monitoring and alarms.
- ✅ Use scaling policies (target tracking, step scaling, scheduled).
- ✅ Use CloudTrail for audit and governance.
- ✅ Choose ALB for modern applications; NLB for TCP/UDP; CLB for legacy.
- ✅ Properly configure security groups, subnets and listeners.
- ✅ Always use HTTPS with ACM for production workloads.

"Monitor, scale and govern – for a resilient and high-performing application on AWS."

Amazon EC2 Auto Scaling & Auto Scaling Group (ASG) – Configure, Scale, Recover

Handle changing workload • Improve availability • Optimize cost

"Right number of instances, Right time, Right cost!"

Scalable
Reliable
Cost-Effective

1 What is EC2 Auto Scaling ?

- Amazon EC2 Auto Scaling automatically adjusts the number of EC2 instances based on demand (traffic / load).
- It ensures that you have the right number of instances to handle changing workload.
- It works with CloudWatch metrics, scaling policies and launch templates.

"Automatically scale your application for high availability and optimal cost."

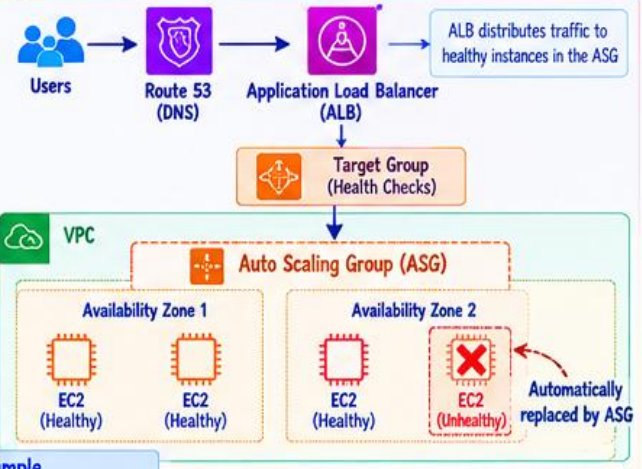
2 Benefits

- ✅ **High availability** – maintains desired capacity across multiple AZs.
- ✅ **Automatic replacement of unhealthy instances.**
- ✅ **Cost optimization** – scales in when demand **decreases** (no need to pay for unused instances).
- ✅ **Elasticity** – automatically scales out during traffic spikes.
- ✅ **Better performance** – handles increased load efficiently.

3 EC2 Auto Scaling vs Auto Scaling Group (ASG)

Aspect	EC2 Auto Scaling	Auto Scaling Group (ASG)
What is it?	AWS service that provides auto scaling capability.	A collection of EC2 instances that share the same scaling and management configuration.
Scope	Service (global/regional)	Specific group of instances
Purpose	Provides the feature (you create ASGs using it).	Maintains desired capacity, scales instances, replaces unhealthy instances.
Terminology	Service	Resource created in the service
Example	EC2 Auto Scaling (service)	WebServer-ASG (group)

4 Architecture Diagram (ALB + ASG)



5 Core Settings for an Auto Scaling Group

Minimum capacity	Minimum number of instances to run.
Desired capacity	Number of instances to maintain (initial).
Maximum capacity	Maximum number of instances allowed.
Launch template / configuration	AMI, instance type, security groups, user data, key pair, etc.
Subnets / AZs	Select at least two AZs for high availability.
Health checks	EC2 health checks or ELB health checks.
Target group	Attach to an ALB/NLB target group.

6 How Auto Scaling Works ? (Workflow)

1. **Monitor (CloudWatch)**
   - Monitor metrics (e.g., CPU, RequestCount, NetworkIn/Out).
   - CloudWatch alarms trigger scaling actions.
2. **Scale Out (increase instances)**
   - When demand increases (e.g., CPU > threshold).
   - ASG launches new EC2 instances.
   - Instances are registered with Target Group.
   - ALB starts sending traffic to new instances.
3. **Scale In (decrease instances)**
   - When demand decreases (e.g., CPU < threshold).
   - ASG terminates instances (graceful).
   - Maintains desired capacity within min/max.
4. **Replace Unhealthy**
   - If an instance fails health checks,
   - ASG automatically terminates it and launches a new instance.

7 Scaling Policies (Important)

Policy Type	Description	Example
Target Tracking	Automatically adjusts capacity to maintain a target metric (e.g., CPU 50%).	Keep average CPU at 50%. ASG adds/removes instances automatically.
Step Scaling	Adds/removes instances based on metric thresholds and step adjustments.	If CPU > 70% → add 2 instances If CPU > 90% → add 4 instances If CPU < 30% → remove 2 instances
Scheduled Scaling	Scales based on a fixed schedule (time/date).	Scale out at 9 AM (expected high traffic), scale in at 10 PM (low traffic).

Use a combination of policies for better control (e.g., target tracking + scheduled scaling)."

8 ALB + ASG Relationship

- ✅ **ALB** distributes incoming traffic to healthy instances in the ASG (using Target Group).
- ✅ **ASG** changes the number of instances based on demand.
- ✅ **ALB** performs health checks and only sends traffic to healthy instances.
- ✅ If an instance becomes unhealthy, ALB stops sending traffic and ASG replaces it.

ALB handles traffic distribution, ASG handles the instance count."

9 Step-by-Step: Create an Auto Scaling Group (Console)

1. Go to EC2 Console → Auto Scaling Groups → Create Auto Scaling group.
2. Choose launch template (or launch configuration).
3. Select instance type, AMI, security group, key pair, user data.
4. Select VPC and at least two subnets (different AZs).
5. Set minimum, desired and maximum capacity.
6. Attach to an existing or new load balancer target group (ALB).
7. Configure health check (EC2 or ELB).
8. Add scaling policies (target tracking / step / scheduled).
9. Review and create.

10 Exam Scenario (Based on Your Notes)

Problem	A web application experiences heavy traffic during the day and low traffic at night. Some EC2 instances may fail.
Proposed Design	Use an ALB with an ASG across multiple AZs.
Traffic Spike (Day)	Scale out when CPU > 70% (add instances).
Low Traffic (Night)	Scale in when CPU < 30% (remove instances).
Instance Failure	Automatically replaced by ASG (health check).
Result	Improved availability, handles changing workload, better resource utilization and lower cost.

11 Fixed EC2 vs Auto Scaling Group

Feature	Fixed EC2 Fleet	Auto Scaling Group
Instance count	Fixed	Automatically adjusts
Handles traffic spike	May fail or become slow	Scales out automatically
Availability	If instance fails, manual replacement	Automatically replaces unhealthy instances
Cost	Pay for idle instances	Scales in to reduce cost
Management	Manual	Automatic
Best use case	Steady, predictable load	Changing / unpredictable load

12 Answering Your Exam Questions

Q1 How can ASG be used to handle changing workload?

- Monitors metrics (e.g., CPU) using CloudWatch.
- Automatically increases instances when demand rises.
- Decreases instances when demand falls.
- Replaces unhealthy instances.
- Ensures application is always available and stable.

Q2 How does load balancing + ASG improve availability and resource utilization?

- ALB distributes traffic across multiple healthy instances.
- ASG ensures the right number of instances are running.
- If an instance fails, ALB stops sending traffic and ASG replaces it.
- Scales out during high traffic (better performance).
- Scales in during low traffic (cost optimization).

13 CloudWatch Monitoring (for ASG)

Amazon CloudWatch

- Monitor metrics: CPUUtilization, RequestCount, NetworkIn/Out, HealthyHostCount, etc.
- Create CloudWatch alarms (e.g., CPU > 70%).
- Alarms trigger scaling policies.
- Also monitor using ALB metrics (TargetResponseTime, HTTP 5XX).

14 Key Takeaways

- ✅ ASG automatically adjusts EC2 instances based on demand.
- ✅ Use multiple AZs for high availabiity.
- ✅ Configure health checks and scaling policies.
- ✅ ALB + ASG = High availability + Better resource utilization.
- ✅ Use CloudWatch for monitoring and alarms.
- ✅ Helps reduce cost and improve performance.

15 Memory Trick

"M D M S" for ASG Setup

- **M** – Min capacity
- **D** – Desired capacity
- **M** – Max capacity
- **S** – Scaling policy

Remember: Set the range and scaling rules!

16 Final Exam Quote



"Let Auto Scaling handle the peaks, so your application never sleeps."

Scalable Today. Reliable Tomorrow."

Amazon EC2 Auto Scaling & Auto Scaling Group (ASG)

Automatically scale | High availability | Fault tolerance | Cost optimization

> "Scale your infrastructure with demand, not guesswork."

1 What is EC2 Auto Scaling ?

- Amazon EC2 Auto Scaling automatically adjusts the number of EC2 instances based on demand (traffic / load).
- It ensures you have the right number of instances to handle changing workload.
- It works with CloudWatch metrics, scaling policies and launch templates.
- Improves application availability, fault tolerance and cost efficiency.

> "Automatically scale your application for high availability and optimal cost."

2 What is Auto Scaling Group (ASG) ?

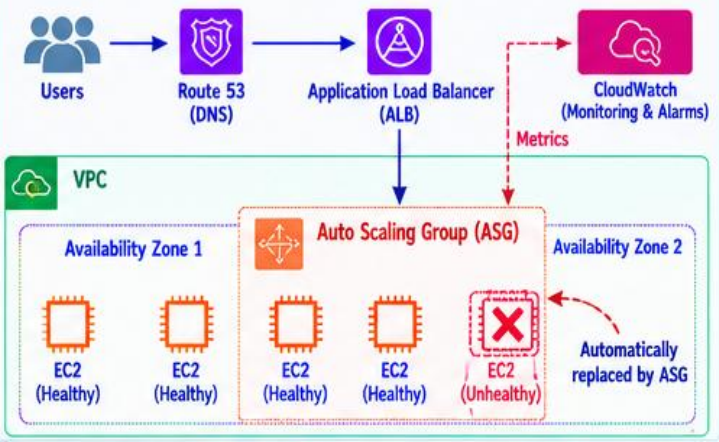
- An Auto Scaling Group (ASG) is a collection of EC2 instances that share the same scaling policies and configuration.
- Maintains desired capacity by automatically launching or terminating instances.
- Replaces unhealthy instances automatically (using health checks).
- Spreads instances across multiple Availability Zones (for high availability).
- Works with Load Balancer (ALB/CLB/NLB) to distribute traffic.

> "ASG ensures the right number of healthy instances are always running."

3 Key Components

Component	Description
Launch Template (or Launch Config)	Defines AMI, instance type, key pair, security groups, user data, etc.
Auto Scaling Group	Manages a group of EC2 instances and scaling policies.
Minimum Capacity	Minimum number of instances to run.
Desired Capacity	Ideal number of instances (initial).
Maximum Capacity	Maximum number of instances allowed.
Scaling Policy	Defines when and how to scale (target tracking, step scaling, scheduled).
Health Check	Checks instance health (EC2 or ELB).
Load Balancer	Distributes traffic to instances in ASG.
CloudWatch	Monitors metrics and triggers scaling.

4 Architecture Diagram (ALB + ASG)



5 How Auto Scaling Works ? (Workflow)

1. **Monitor** — CloudWatch monitors metrics (e.g., CPU, RequestCount).
2. **Evaluate** — If metric breaches threshold, scaling policy is triggered.
3. **Scale Out / In** — ASG launches or terminates instances based on policy.
4. **Replace Unhealthy** — If an instance fails health check, ASG terminates it and launches a new instance automatically.

6 Scaling Policies

Policy Type	Description	Example
Target Tracking	Automatically maintains a target value for a metric (e.g., CPU 50%).	Scale out if CPU > 50% Scale in if CPU < 50%
Step Scaling	Adds or removes instances based on metric thresholds and step adjustments.	If CPU > 70% → add 2 If CPU > 90% → add 4 If CPU < 30% → remove 2
Scheduled Scaling	Scales based on a fixed schedule (time/date).	Scale out at 9 AM Scale in at 10 PM

> Use a combination of policies for better control (e.g., target tracking + scheduled scaling).

7 Step-by-Step: Create an ASG (Console)

1. Go to EC2 Console → Auto Scaling Groups.
2. Choose Launch Template (or Launch Configuration).
3. Select AMI, instance type, key pair, security groups, user data.
4. Choose VPC and at least two subnets (different AZs).
5. Set Min, Desired and Max capacity.
6. Attach to a Load Balancer (ALB/CLB/NLB) with a target group.
7. Configure health check (EC2 or ELB).
8. Add scaling policies (target tracking / step / scheduled).
9. Choose notifications (SNS – optional).
10. Review and create.

8 CloudWatch Monitoring (for ASG)

Amazon CloudWatch

Important Metrics

- CPUUtilization
- RequestCount (for ALB)
- NetworkIn / NetworkOut
- HealthyHostCount / UnHealthyHostCount
- GroupInServiceInstances
- GroupTotalInstances
- ALB metrics (TargetResponseTime, HTTPCode_XX)

CloudWatch Alarms

- Trigger scaling policies (scale out/in)
- Notify via SNS (email, SMS)
- Monitor application health and performance

9 Use Cases

- Web applications (variable traffic)
- E-commerce (handle seasonal spikes)
- Real-time applications (gaming, streaming)
- APIs and microservices
- Disaster recovery (multi-AZ)
- Batch processing (scale on demand)

> "Perfect for applications with unpredictable or time-based traffic."

10 Scenario: Handling Traffic Spike

Q: A web application runs on EC2 behind an ALB. During a festival, traffic increases and some instances become slow. Design a solution.

Solution:

1. Use an Auto Scaling Group with Launch Template.
2. Set min: 2, desired: 4, max: 10 (example).
3. Use Target Tracking policy (CPU 50%).
4. Attach ASG to an ALB (across multiple AZs).
5. Enable CloudWatch monitoring and alarms.
6. Use Scheduled Scaling for known events (e.g., festivals).
7. Ensure unhealthy instances are replaced automatically.

Result: Application remains available, handles high traffic, and recovers automatically.

11 Fixed EC2 vs Auto Scaling Group

Feature	Fixed EC2	Auto Scaling Group
Instance count	Fixed	Dynamic (scales automatically)
Handles traffic spike	May fail / become slow	Scales out automatically
Availability	Manual recovery	Replaces unhealthy instances
Cost	Pay for idle instances	Scales in to reduce cost
Management	Manual	Automatic
Best use case	Stable, predictable workload	Changing / unpredictable workload

12 Key Takeaways

- ✓ ASG automatically adjusts EC2 instances based on demand.
- ✓ Use multiple AZs for high availability.
- ✓ Configure health checks and scaling policies.
- ✓ Integrate with ALB for traffic distribution.
- ✓ Use CloudWatch for monitoring and alarms.
- ✓ Choose the right scaling policy for your use case.
- ✓ Helps reduce cost and improve performance.
- ✓ Automatically replaces unhealthy instances.
- ✓ Use scheduled scaling for predictable events.

13 Final Exam Quote

> "Right number of instances, Right time, Right cost – that's the power of Auto Scaling!"

AWS CloudTrail – Complete Overview

Log • Monitor • Audit • Govern • Ensure Compliance

"Know who did what, when and from where in your AWS account."

Visibility drives Security and Better Decisions!

1 What is AWS CloudTrail ?

- AWS CloudTrail is a logging and monitoring service that records AWS API calls and account activity.
- It provides event history of actions taken by users, roles, and AWS services.
- Helps in security analysis, operational auditing, troubleshooting and compliance.
- It does not monitor resource configuration changes continuously (for that we use AWS Config).
- CloudTrail logs are stored in Amazon S3 and can be analyzed using Amazon CloudWatch, Athena, or other tools.

"CloudTrail gives you a record of your AWS activity for security, governance and compliance."

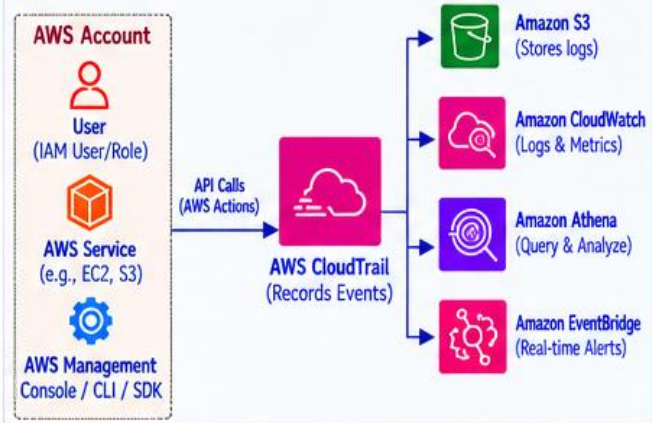
2 Key Features

- ✅ Records API calls for all AWS services (supported services).
- ✅ Logs who made the request, what action was taken, when, from where (IP), and the response.
- ✅ Supports Management Events, Data Events and Insights Events.
- ✅ Integrates with Amazon S3, CloudWatch Logs, EventBridge, AWS Config, and Amazon Athena.
- ✅ Helps in security monitoring, compliance (PCI, HIPAA, GDPR) and forensic analysis.
- ✅ Multi-region and organization-wide tracking (using AWS Organizations).
- ✅ Can deliver logs to a centralized S3 bucket across multiple accounts.

3 Types of Events

Event Type	Description	Examples	Enabled by Default
Management Events	Record management plane API calls (e.g., create, delete, modify).	RunInstances, CreateBucket, DeleteSecurityGroup	Yes
Data Events	Record data plane API calls on resources (like S3, Lambda).	GetObject, PutObject (Lambda Invoke, DynamoDB Read/Write)	No (must be enabled)
Insights Events	Identify unusual API activity using machine learning.	Unusual API call volume, credential compromise (behavioral anomalies)	No (must be enabled)

4 How CloudTrail Works ? (Architecture)



5 Event Log Structure (Sample)

```
{
   "eventVersion": "1.11",
   "userIdentity": {
      "type": "IAMUser",
      "arn": "arn:aws:iam::123456789012:user/ravi"
   },
   "eventTime": "2025-09-15T10:23:45Z",
   "eventSource": "ec2.amazonaws.com",
   "eventName": "RunInstances",
   "awsRegion": "ap-south-1",
   "sourceIPAddraes": "203.0.113.25",
   "requestParameters": { ... },
   "responseElements": { ... },
   "eventID": "abcd-1234-...",
   "eventType": "AwsApiCall",
   "managementEvent": true
}
```

Tells you who did what, when, from where and the result.

6 CloudTrail vs AWS Config vs CloudWatch

Feature	CloudTrail	AWS Config	CloudWatch
Purpose	Logs API calls (account activity)	Tracks resource configuration changes	Monitors metrics, logs and alarms
Data collected	API events (who did what)	Resource configuration history	Metrics, logs, custom logs
Continuous monitoring	No (event-based)	Yes (configuration state)	Yes (logs & metrics)
Use cases	Auditing, security analysis, compliance	Detect config changes, compliance	Operational monitoring, alerts

7 Creating a Trail (Steps)

1. Go to CloudTrail console.
2. Click Create trail.
3. Choose a name for the trail.
4. Select the S3 bucket (create a new one or use existing).
5. Choose event types: Management events (default), Data events (optional), Insights events (optional).
6. Enable log file integrity validation (optional).
7. Choose multi-region trail (recommended).
8. Add CloudWatch Logs (optional, for real-time monitoring).
9. Set up log file SSE encryption (optional, using KMS).
10. Review and create the trail.

8 Important Settings

Setting	Description
Trail type	Single-region or Multi-region (recommended)
Event type	Management events, Data events, Insights events
S3 bucket	Stores log files (enable versioning)
Log file integrity validation	Detects tampering with log files
CloudWatch Logs	Real-time log delivery and monitoring
SSE Encryption	Encrypt logs using AWS KMS
Include global service events	Logs for global services (e.g., IAM, STS, Route 53) are stored in us-east-1

9 Use Cases

- Security auditing (unauthorized activity)
- Compliance reporting (PCI, HIPAA, GDPR)
- Investigation & forensics (who did what, when)
- Detect unusual activity (using Insights)
- Governance in multi-account environments (AWS Organizations)
- Historical analysis and troubleshooting

"Essential for security, compliance and accountability in the AWS cloud."

10 Integration with Other AWS Services

- **Amazon S3** (Store logs)
- **Amazon CloudWatch** (Logs, Metrics, Alarms)
- **Amazon EventBridge** (Trigger real-time actions)
- **Amazon Athena** (Query logs with SQL)
- **AWS Config** (Track resource changes)
- **AWS Organizations** (Organization-wide trails)

CloudTrail works best when integrated with other AWS services to monitor, analyze and respond to events effectively.

11 Best Practices

- ✅ Enable multi-region CloudTrail.
- ✅ Log all management events (and enable data events if needed).
- ✅ Enable log file integrity validation.
- ✅ Store logs in a secure S3 bucket (enable versioning + encryption).
- ✅ Send logs to CloudWatch for real-time monitoring (optional).
- ✅ Use AWS Organizations to enable organization trail (across all accounts).
- ✅ Set up alerts using EventBridge for critical events (e.g., policy changes, security group open to 0.0.0.0/0).
- ✅ Regularly review logs using Athena or CloudWatch Logs Insights.
- ✅ Retain logs for a sufficient period (as per compliance needs).

12 Exam Tips & Common Questions

1. CloudTrail logs API calls, not resource configuration (that's AWS Config).
2. Management events are enabled by default; data events are disabled by default.
3. Global service events (IAM, STS, Route 53) are logged in us-east-1.
4. Use CloudTrail Insights to detect unusual activity.
5. CloudTrail logs are stored in Amazon S3.
6. For real-time alerts, integrate with CloudWatch or EventBridge.
7. Organization trail is used to collect logs from all accounts in an AWS Organization.
8. Know the difference between CloudTrail, AWS Config and CloudWatch.

13 Final Takeaway

"CloudTrail provides visibility, accountability and security by recording every API call made in your AWS environment. Use it with S3, CloudWatch, Athena and EventBridge to monitor, analyze and respond to AWS activity effectively."

Know your cloud activity. Secure your cloud journey.

AWS CloudWatch – Monitoring, Metrics, Logs, Alarms and Troubleshooting

Observe • Detect • Alert • Alert • Take Action • Keep your applications healthy

"Monitor today,
Solve problems before users feel them."

1 What is Amazon CloudWatch ?

- Amazon CloudWatch is a monitoring and observability service for AWS resources and applications.
- Collects and tracks metrics, logs and events in real time.
- Helps you monitor resource utilization, application performance, and operational health.
- You can set alarms, get notifications, and automate actions.
- Works across AWS services, on-premises servers, and custom applications (via agents).

"CloudWatch gives you visibility into your AWS environment."

2 Key Features

- ✓ Collects and stores metrics, logs and events
- ✓ Visualizes data with dashboards
- ✓ Create alarms and get notifications (SNS)
- ✓ Automatic monitoring for many AWS services (e.g., EC2, RDS, Lambda, ELB, etc.)
- ✓ Supports custom metrics and application logs
- ✓ Integrates with Auto Scaling, EventBridge, SNS and more
- ✓ Helps with troubleshooting and performance optimization
- ✓ Retention and storage options for logs
- ✓ Built-in insights with CloudWatch Logs Insights
- ✓ Monitors across AWS and on-premises (using CloudWatch Agent)

3 Components of CloudWatch

Metrics
- Time-series data about resources (e.g., CPUUtilization, RequestCount).
- Collected at 1-minute granularity (5-minute for some services).

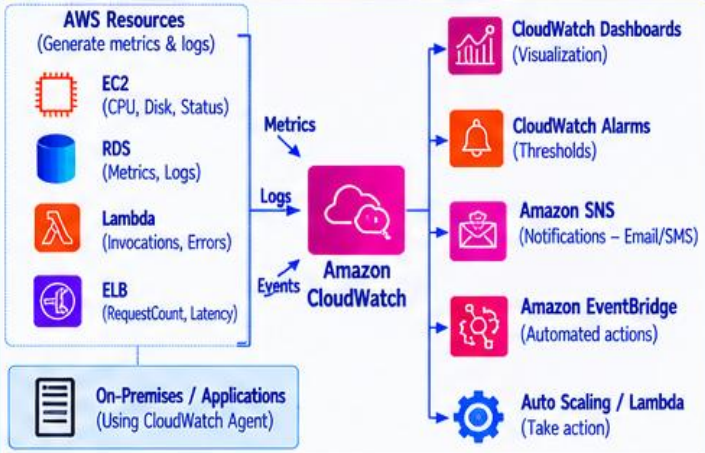
Logs
- Collects, stores and allows you to search log data (e.g., application logs, VPC flow logs, Lambda logs).
- Use CloudWatch Logs Insights to query logs.

Alarms
- Monitor metrics or log-based conditions.
- Trigger notifications (SNS) or automated actions (e.g., Auto Scaling).

Dashboards
- Visualize metrics, alarms and logs in one place.
- Create custom dashboards for your needs.

Events (EventBridge)
- React to changes in resources (e.g., instance state change).
- Can trigger automated actions (Lambda, SNS, etc.).

4 CloudWatch Architecture (High-Level)



5 Common Metrics (Examples)

AWS Service	Important Metrics
EC2	CPUUtilization, DiskReadBytes, DiskWriteBytes, NetworkIn/Out
RDS	CPUUtilization, DatabaseConnections, ReadLatency, WriteLatency
Lambda	Invocations, Errors, Duration, Throttles
ELB / ALB	RequestCount, TargetResponseTime, HTTPCode_Target_2XX / 4XX / 5XX
ECS	CPUUtilization, MemoryUtilization
S3	BucketSizeBytes, NumberOfObjects, AllRequests
Custom Application	Any business metric (e.g., ActiveUsers, Transactions) using CloudWatch Agent or PutMetricData API

6 CloudWatch Logs and Logs Insights

- CloudWatch Logs collects and stores log data.
- Supports log groups and log streams.
- Use CloudWatch Logs Insights to search and analyze logs with SQL-like queries.
- Helps detect errors, unusual activity and the exact cause of issues.

Example Logs Insights Query

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

7 Alarms and Notifications

- Create alarms on metrics (e.g., CPU > 70%).
- Define threshold, evaluation period and number of datapoints.
- When the alarm state is triggered, CloudWatch can:
  - Send notifications via Amazon SNS (email, SMS).
  - Execute automated actions (e.g., scale EC2 using Auto Scaling).
  - Trigger Lambda or EventBridge rules.

Alarm States

OK	ALARM	INSUFFICIENT DATA
Metric is within threshold	Metric is breaching the threshold	Not enough data to evaluate yet

8 Troubleshooting Workflow (From Metric to Root Cause)

1. Identify the issue – e.g., high latency or application not responding.
2. Check CloudWatch dashboard for key metrics (CPU, Memory, RequestCount, etc.).
3. If metric crosses threshold, check the CloudWatch alarm state.
4. Go to CloudWatch Logs and use Logs Insights to search for errors/exceptions.
6. Correlate timestamps between metric spike and log events.
6. Identify the exact cause (e.g., high CPU due to heavy requests, error in app code).
7. Take action – scale, restart, fix configuration, or update code.

"Metrics tell you that something is wrong.
Logs tell you why it is wrong."

9 Integration with Other AWS Services

- **SNS** (Notifications)
- **Auto Scaling** (Scale resources)
- **EventBridge** (Automate actions)
- **AWS Config** (Compliance)
- **CloudTrail** (Audit API calls)
- **Systems Manager** (Monitor on-premises)
- **X-Ray** (Distributed tracing)
- **Lambda** (Trigger actions)

10 Creating a CloudWatch Alarm (Steps)

1. Go to CloudWatch Console → Alarms → Create Alarm.
2. Select a metric (e.g., EC2 → CPUUtilization).
3. Choose statistic (e.g., Average) and period (e.g., 5 minutes).
4. Set threshold (e.g., CPU > 70%).
5. Configure alarm conditions (number of datapoints).
6. Choose an action (SNS notification or Auto Scaling).
7. Add name and description.
8. Review and create.

11 Best Practices

- ✓ Monitor key metrics for all critical resources.
- ✓ Create meaningful alarms (avoid alarm fatigue).
- ✓ Use dashboards for quick visibility.
- ✓ Centralize logs and use Logs Insights for analysis.
- ✓ Set up notifications via SNS.
- ✓ Use custom metrics for business KPIs.
- ✓ Enable detailed monitoring for EC2 (1-minute metrics).
- ✓ Set appropriate log retention to control costs.
- ✓ Integrate with EventBridge and Auto Scaling for automation.
- ✓ Use multiple regions and accounts (CloudWatch cross-account).

12 Common Use Cases

- Monitor application performance
- Detect and alert on failures
- Capacity and resource planning
- Audit and compliance reporting
- Troubleshoot issues (logs)
- Real-time operational visibility
- Monitor on-premises infrastructure
- Security monitoring (unauthorized activity)

13 Exam Tips & Key Takeaways

- ✓ CloudWatch is for monitoring (not for configuration change tracking – that's AWS Config).
- ✓ Know the difference between metrics, logs, alarms and events.
- ✓ Alarms can trigger SNS notifications or Auto Scaling actions.
- ✓ CloudWatch Logs Insights is useful for finding the exact cause of an issue.
- ✓ Detailed monitoring provides 1-minute metrics (default is 5 minutes).
- ✓ CloudWatch integrates with many AWS services (SNS, EventBridge, Auto Scaling, etc.).
- ✓ Use dashboards for better operational visibility.
- ✓ Be ready for scenario-based questions (e.g., high CPU, how to investigate?).

"Observe, Analyze, Act –
with CloudWatch, you are always one step ahead."

Amazon EC2 Auto Scaling & ASG | Amazon CloudWatch | AWS CloudTrail

Monitor • Scale • Audit • Build Reliable and Secure Applications on AWS

> "Monitor, Scale and Audit – The three pillars of a reliable cloud."

1 Amazon EC2 Auto Scaling – Overview

- Amazon EC2 Auto Scaling automatically adjusts the number of EC2 instances based on demand (traffic / workload).
- Helps maintain application availability, handles faults, and optimizes cost.
- Works with Auto Scaling Group (ASG), CloudWatch metrics and scaling policies.
- Common use cases: web applications, APIs, microservices, batch processing.

> "Right number of instances, right time, right cost."

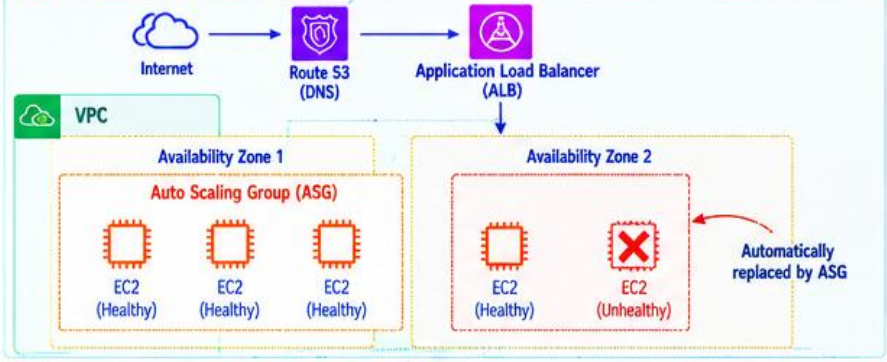
2 Auto Scaling Group (ASG)

- A collection of EC2 instances that share the same scaling policies and configuration.
- Automatically launches, terminates, and replaces unhealthy instances.
- Distributes instances across multiple Availability Zones (for high availability).

Parameter	Description
Minimum capacity	Minimum instances to run
Desired capacity	Ideal number of instances (initial)
Maximum capacity	Maximum instances allowed
Launch template	AMI, instance type, key pair, security group, user data, etc.
Health check	EC2 or ELB health checks

3 Architecture (ALB + ASG)

2 Architecture (ALB + ASG)



5 How Auto Scaling Works ? (Workflow)

1. Monitor metrics (CloudWatch)
2. Evaluate against scaling policy
3. Scale Out (launch instances) or Scale In (terminate instances)
4. Replace unhealthy instances (using health checks)

6 Step-by-Step: Create an ASG

1. Go to EC2 Console → Auto Scaling Groups → Create ASG.
2. Choose launch template / configuration.
3. Select instance type, security group, key pair, user data.
4. Select VPC and at least two subnets (different AZs).
5. Set min, desired, max capacity.
6. Attach a load balancer (ALB/NLB) target group.
7. Configure scaling policies (target tracking / step / scheduled).
8. Configure health checks and notifications.
9. Review and create.

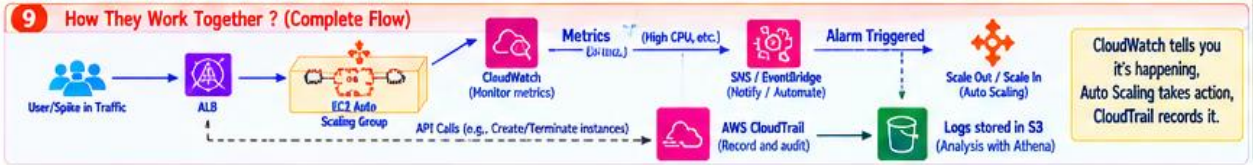
7 Benefits & Use Cases

- ✓ High availability and fault tolerance
- ✓ Automatic scaling for changing demand
- ✓ Cost optimization (scale in when not needed)
- ✓ Better performance and reliability
- ✓ Used for web apps, APIs, a-commerce, batch jobs

8 Common Metrics for ASG

- CPUUtilization (average)
- RequestCount (for ALB)
- NetworkIn / NetworkOut
- HealthyHostCount / UnHealthyHostCount
- GroupDesiredCapacity / GroupInServiceInstances

9 How They Work Together ? (Complete Flow)



1 Amazon CloudWatch – Overview

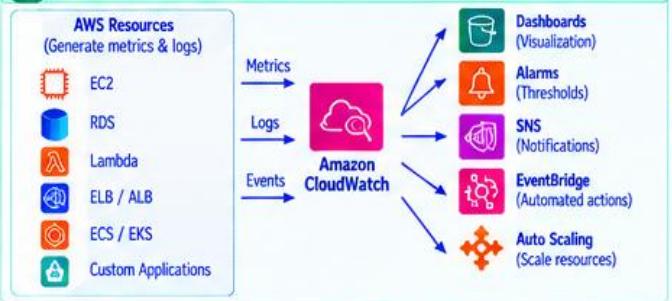
- Amazon CloudWatch is a monitoring and observability service for AWS resources and applications.
- Collects and tracks metrics, logs and events in real time.
- Helps you set alarms, get notifications and automate actions (e.g., Auto Scaling).
- Provides dashboards for visualization and tools for troubleshooting and performance optimization.

> "Monitor today, solve problems before users feel them."

2 Key Components

Metrics (Time-series data) | Logs (Application & system logs) | Alarms (Trigger actions) | Dashboards (Visualization) | Events/EventBridge (Automated actions)

3 CloudWatch Architecture



4 CloudWatch Logs & Logs Insights

- **CloudWatch Logs** collects and stores log data from AWS services and on-premises resources.
- Use **Logs Insights** to search and analyze logs with SQL-like queries.

Example Logs Insights Query

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

5 Creating a CloudWatch Alarm

1. Go to CloudWatch Console → Alarms → Create alarm.
2. Select metric (e.g., EC2 → CPUUtilization).
3. Set conditions (e.g., CPU > 70% for 5 minutes).
4. Choose action (SNS notification or Auto Scaling).
5. Add name and description.
6. Review and create.

6 Alarm States

OK	ALARM	INSUFFICIENT DATA
Metric is within threshold	Metric is breaching threshold	Not enough data to evaluate yet

7 Important Metrics (Examples)

EC2	CPUUtilization, DiskReadBytes, NetworkIn/Out
RDS	CPUUtilization, DatabaseConnections, ReadLatency
Lambda	Invocations, Errors, Duration, Throttles
ELB / ALB	RequestCount, TargetResponseTime, HealthyHostCount
Auto Scaling	GroupInServiceInstances, GroupDesiredCapacity
Custom App	Business metrics (e.g., ActiveUsers, Transactions)

1 AWS CloudTrail – Overview

- AWS CloudTrail is a logging and monitoring service that records all API calls and account activity.
- Provides event history of actions taken by users, roles and AWS services.
- Helps in security analysis, operational auditing, troubleshooting and compliance (e.g., PCI, HIPAA, GDPR).
- Logs are stored in Amazon S3 and can be analyzed using CloudWatch, Athena or other tools.

> "Know who did what, when and from where in your AWS account."

2 Types of Events

Event Type	Description	Example	Enabled by Default
Management Events	API calls on control plane (e.g., create, delete, modify)	RunInstances, CreateBucket	Yes
Data Events	API calls on data plane (e.g., S3 object-level)	GetObject, PutObject	No (must enable)
Insights Events	Detects unusual API activity (behavioral analysis)	Unusual API call pattern	No (must enable)

3 CloudTrail Architecture



4 Event Log Structure (Sample)

```
{
  "eventTime": "2025-09-15T10:23:452",
  "eventName": "RunInstances",
  "eventSource": "ec2.amazonaws.com",
  "awsRegion": "ap-south-1",
  "sourceIPAddress": "203.0.113.25",
  "userIdentity": {"type": "IAMUser", "arn":
  "arn:aws:iam::123456789012:user/ravi"},
  "requestParameters":
  "responseElements": (...
}
```

Tells you who did what, when, from where and the result.

5 Creating a Trail (Steps)

1. Go to CloudTrail Console → Create trail.
2. Choose trail name.
3. Select S3 bucket (create new or use existing).
4. Choose event types (management, data, insights).
5. Enable log file integrity validation (optional).
6. Choose multi-region trail (recommended).
7. Add CloudWatch Logs (optional).
8. Review and create.

6 Use Cases

- ✓ Security auditing and compliance
- ✓ Detect unauthorized activity
- ✓ Investigate and perform forensic analysis
- ✓ Monitor API activity across accounts
- ✓ Meet regulatory requirements (PCI, HIPAA, GDPR)

10 Exam Scenarios

1. A web application faces high traffic during a sale. CPU > 70%.
   → Use CloudWatch alarm with target tracking policy to scale out ASG.
2. An instance becomes unhealthy.
   → ASG automatically replaces it (uses health checks).
3. You need to find who terminated an EC2 instance.
   → Check CloudTrail logs (eventName: TerminateInstances).
4. You want to analyze application errors.
   → Use CloudWatch Logs and Logs Insights.
5. You need to detect unusual API activity.
   → Enable CloudTrail Insights and set CloudWatch alarm.

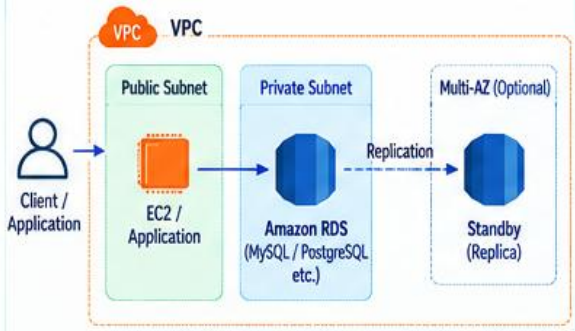
11 Key Takeaways

- ✓ ASG ensures high availability and automatic scaling.
- ✓ CloudWatch monitors metrics, logs and triggers alarms.
- ✓ CloudTrail records all API activity for auditing and compliance.
- ✓ Use CloudWatch alarms to automate scaling with ASG.
- ✓ Integrate services (SNS, EventBridge, S3, Athena) for monitoring and analysis.
- ✓ Use multi-AZ, multi-region and proper IAM permissions.
- ✓ Together, these services help you build secure, reliable and cost-effective applications on AWS.

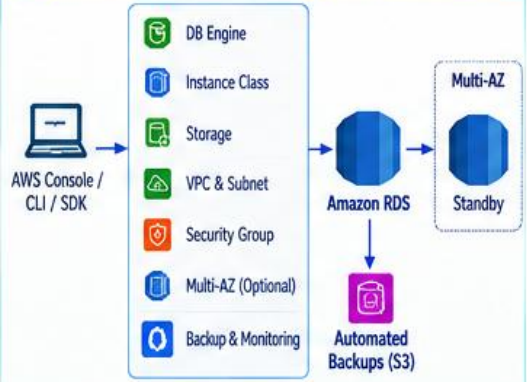
"Observe with CloudWatch, Scale with Auto Scaling, Audit with CloudTrail – Run in the cloud with confidence."

Database Services

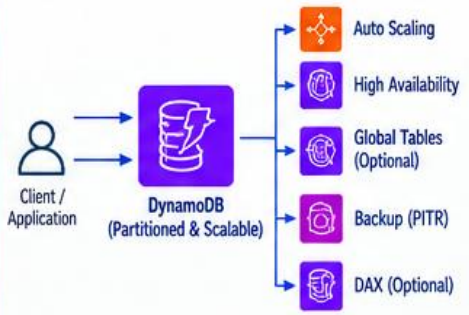
1 Amazon RDS – Architecture



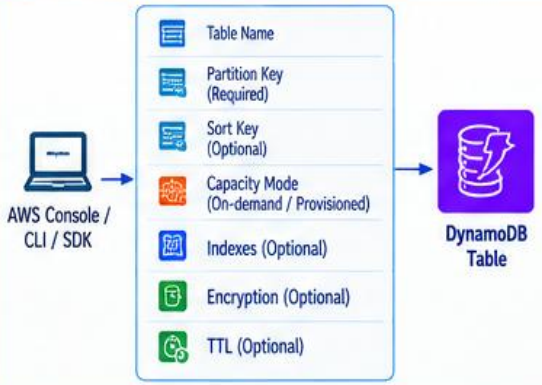
2 Create an Amazon RDS – Configuration Architecture



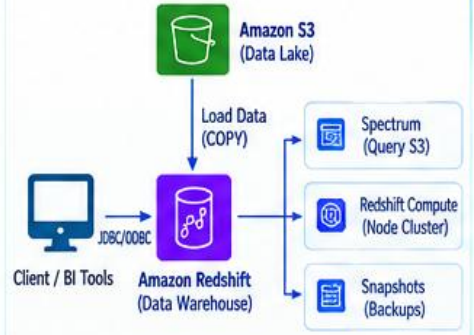
3 Amazon DynamoDB – Architecture



4 Create DynamoDB Table – Configuration Architecture



5 Amazon Redshift – Architecture

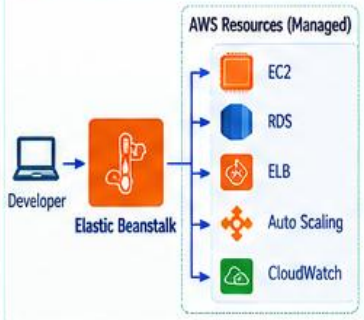


Developing Tools

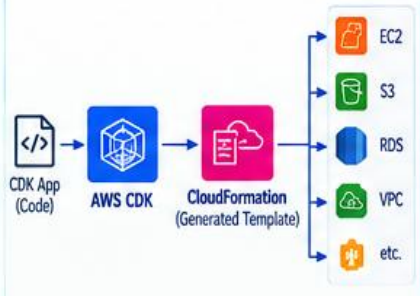
6 AWS CloudFormation – Architecture



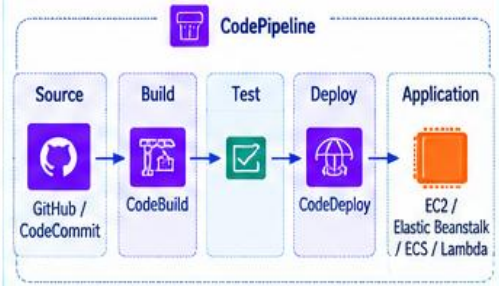
7 Elastic Beanstalk – Architecture



8 AWS CDK – Architecture

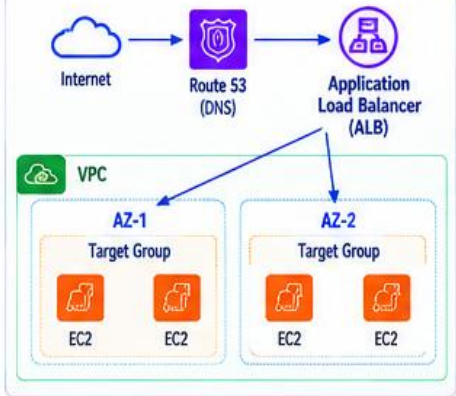


9 AWS CodePipeline (CI/CD) – Architecture



Management & Governance

10 AWS Elastic Load Balancer (ALB) – Architecture



11 Create an Application Load Balancer – Configuration



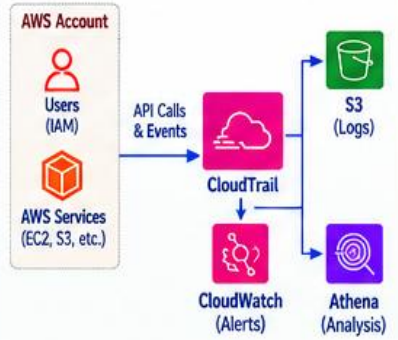
12 EC2 Auto Scaling & ASG – Architecture



13 CloudWatch – Architecture



14 CloudTrail – Architecture



Design | Deploy | Monitor | Scale | Govern – with AWS